



MA95 Rexx Interface - Message Handles and Properties Extension

Version 1.1 - Draft 2

Take Note!

Before using this User's Guide and the product it supports, be sure to read the general information under "Notices".

This edition applies to the following product:

Version 1.1 of MA95: A Rexx Interface to IBM MQ (message-property sample integration draft)
and to any subsequent releases and modifications until otherwise indicated in new editions.

© **Copyright IBM Corporation 1997, 2010. All rights reserved.**

US Government Users Restricted Rights - Use, duplication or disclosure restricted by GSA ADP
Schedule Contract with IBM Corporation.

Contents

Contents.....	iii
Figures.....	ix
Tables.....	x
Notices.....	xi
Preface.....	xii
Prerequisites.....	xii
Acknowledgements.....	xii
Other SupportPacs.....	xii
Chapter 1. Introduction.....	1
Chapter 2. Installing the SupportPac.....	3
Installing on Windows.....	4
Installing the DLLs.....	4
Compiling the Code for Windows.....	4
Installing on z/OS.....	6
Compiling the Code for z/OS.....	6
TSO Support.....	7
Chapter 3. Interface Design Philosophy.....	8
Chapter 4. General Points.....	9
Compatibility with Previous SupportPacs.....	9
Return Codes.....	10
Last Operation.....	10
Return Code Naming.....	10
Message Lengths.....	11
Header and Event processing.....	11
ZLIST.....	12
Stem Variables.....	14
Rexx Execs.....	15
Trace.....	16
Chapter 5. Handling MQ Descriptors.....	17
The Object Descriptor.....	19
The Message Descriptor.....	20
The Get Message Option Structure.....	22
The Put Message Options Structure.....	23
The Variable Length String Structure.....	24
Specifying parameters for command interface (RXMQC).....	25
Chapter 6. Thread Support.....	26
Initialization.....	26

<i>Termination</i>	26
<i>Connection and Disconnection</i>	26
<i>Access scope</i>	26
<i>Shared Variables</i>	26
Chapter 7. The Interface	27
<i>Initialization</i>	27
Description.....	27
Parameters.....	27
Call.....	27
Additional Interface Return Codes and Messages.....	27
Example.....	27
<i>Thread Initialization</i>	28
Description.....	28
Parameters.....	28
Call.....	28
Additional Interface Return Codes and Messages.....	28
Example.....	28
<i>Termination</i>	29
Description.....	29
Parameters.....	29
Call.....	29
Additional Interface Return Codes and Messages.....	29
Example.....	29
<i>RXMQCONN</i>	30
Description.....	30
Parameters.....	30
Call.....	30
Additional Interface Return Codes and Messages.....	30
Example.....	30
<i>RXMQDISC</i>	32
Description.....	32
Parameters.....	32
Call.....	32
Additional Interface Return Codes and Messages.....	32
Example.....	32
<i>RXMQOPEN</i>	33
Description.....	33
Parameters.....	33
Call.....	33
Additional Interface Return Codes and Messages.....	33
Example.....	34
<i>RXMQCLOS</i>	35
Description.....	35
Parameters.....	35
Call.....	35
Additional Interface Return Codes and Messages.....	35
Example.....	36
<i>RXMQINQ</i>	37
Description.....	37
Parameters.....	37
Call.....	37
Additional Interface Return Codes and Messages.....	37
Example.....	38
<i>RXMQSET</i>	39

Description.....	39
Parameters.....	39
Call.....	39
Additional Interface Return Codes and Messages.....	39
Example.....	40
<i>RXMQSUB</i>	41
Description.....	41
Parameters.....	41
Call.....	41
Additional Interface Return Codes and Messages.....	41
Example.....	41
<i>RXMQCMIT</i>	41
Description.....	41
Parameters.....	42
Call.....	42
Additional Interface Return Codes and Messages.....	42
Example.....	42
<i>RXMQBACK</i>	43
Description.....	43
Parameters.....	43
Call.....	43
Additional Interface Return Codes and Messages.....	43
Example.....	43
<i>RXMQGET</i>	44
Description.....	44
Parameters.....	44
Call.....	44
Additional Interface Return Codes and Messages.....	45
Example.....	46
<i>RXMQPUT</i>	47
Description.....	47
Parameters.....	47
Call.....	47
Additional Interface Return Codes and Messages.....	47
Example.....	49
<i>RXMQPUT1</i>	50
Description.....	50
Parameters.....	50
Call.....	50
Additional Interface Return Codes and Messages.....	51
Example.....	53
<i>RXMQC</i>	54
Description.....	54
Parameters.....	54
Additional Interface Return Codes and Messages.....	54
Examples.....	55
<i>RXMQBRWS</i>	56
Description.....	56
Parameters.....	56
Call.....	56
Additional Interface Return Codes and Messages.....	56
Example.....	57
<i>RXMQHXT</i>	58
Description.....	58
Parameters.....	58
Call.....	58
Additional Interface Return Codes and Messages.....	58

Extracted information.....	60
<i>RXMQEVNT</i>	63
Description.....	63
Parameters.....	63
Call.....	63
Usage Notes.....	64
Additional Interface Return Codes and Messages.....	65
Example.....	66
<i>RXMQTM</i>	67
Description.....	67
Parameters.....	67
Call.....	68
Additional Interface Return Codes and Messages.....	68
Trigger information.....	70
Examples.....	71
Appendix A. ISPF Interface.....	72
Appendix B. Sample REXX execs.....	75
Appendix C. Message properties.....	76
<i>Message handle usage</i>	76
<i>Message property option and descriptor structures</i>	77
<i>The Create Message Handle Options Structure (MQCMHO)</i>	77
Description.....	77
Stem mapping and initial values.....	77
Options.....	77
Example.....	77
<i>The Delete Message Handle Options Structure (MQDMHO)</i>	78
Description.....	78
Stem mapping and initial values.....	78
Options.....	78
Example.....	78
<i>The Set Message Property Options Structure (MQSMPO)</i>	78
Description.....	78
Stem mapping and initial values.....	78
Options.....	79
Example.....	79
<i>The Property Descriptor Structure (MQPD)</i>	79
Description.....	79
Stem mapping and initial values.....	79
Field details.....	79
Example.....	80
<i>The Inquire Message Property Options Structure (MQIMPO)</i>	80
Description.....	80
Stem mapping and initial values.....	80
Options.....	80
Example.....	81
<i>The Delete Message Property Options Structure (MQDMPO)</i>	81
Description.....	81
Stem mapping and initial values.....	81
Options.....	81
Example.....	81
<i>The Message Handle To Buffer Options Structure (MQMHBO)</i>	81
Description.....	81
Stem mapping and initial values.....	81

Options.....	82
Example.....	82
<i>The Buffer To Message Handle Options Structure (MQBMHO).....</i>	<i>82</i>
Description.....	82
Stem mapping and initial values.....	82
Options.....	82
Example.....	83
<i>RXMQMH.....</i>	<i>83</i>
Description.....	83
Parameters.....	83
Call.....	83
Additional Interface Return Codes and Messages.....	83
Usage Notes.....	83
Example.....	83
<i>RXMQDMH.....</i>	<i>83</i>
Description.....	83
Parameters.....	84
Call.....	84
Additional Interface Return Codes and Messages.....	84
Usage Notes.....	84
Example.....	84
<i>RXMQSMP.....</i>	<i>84</i>
Description.....	84
Parameters.....	84
Call.....	84
Additional Interface Return Codes and Messages.....	84
Usage Notes.....	85
Example.....	85
<i>RXMQIMP.....</i>	<i>85</i>
Description.....	85
Parameters.....	85
Call.....	86
Additional Interface Return Codes and Messages.....	86
Usage Notes.....	86
Example.....	86
<i>RXMQDMP.....</i>	<i>87</i>
Description.....	87
Parameters.....	87
Call.....	87
Additional Interface Return Codes and Messages.....	87
Usage Notes.....	87
Example.....	87
<i>RXMQMBF.....</i>	<i>87</i>
Description.....	87
Parameters.....	87
Call.....	88
Additional Interface Return Codes and Messages.....	88
Usage Notes.....	88
Example.....	88
<i>RXMQBMH.....</i>	<i>88</i>
Description.....	88
Parameters.....	88
Call.....	89
Additional Interface Return Codes and Messages.....	89
Usage Notes.....	89
Example.....	89

Appendix D. Message property samples.....	90
<i>Sample 1 - Put and get one property using message handles.....</i>	<i>90</i>
Purpose.....	90
Execution flow.....	90
Complete Rexx sample.....	90
Detailed explanation.....	91
Expected validation.....	92
<i>Sample 2 - Enumerate several received properties.....</i>	<i>93</i>
Purpose.....	93
Execution flow.....	93
Complete Rexx sample.....	93
Detailed explanation.....	94
Expected validation.....	95
<i>Sample 3 - Convert properties to MQRFH2 and reconstruct them.....</i>	<i>96</i>
Purpose.....	96
Execution flow.....	96
Complete Rexx sample.....	96
Detailed explanation.....	97
Expected validation.....	97

Figures

Figure 1. TSO Batch JCL.....7

Figure 2. ZLIST and Event processing..... 13

Figure 3. A Trigger Monitor..... 71

Figure 4. A Rexx Triggered Process.....71

Figure 5. ISPF Exec (MA95T1)..... 72

Figure 6. ISPF Panel (MA95P1)..... 73

Figure 7. ISPF Panel (success)..... 74

Figure 8. ISPF Panel (failure)..... 74

Tables

Table 1. Function Names.....	9
Table 2. Object Descriptor (MQOD) Mappings.....	19
Table 3. Message Descriptor (MQMD) Mappings.....	20
Table 4. Get Message Option (MQGMO) Mappings.....	22
Table 5. Put Message Options (MQPMO) Mappings.....	23
Table 6. RXMQC Function Parameter Mappings.....	54
Table 7. Transmission Queue Message Header (MQXQH) Mappings.....	60
Table 8. Dead Letter Queue Message Header (MQDLH) Mappings.....	61
Table 9. Trigger Component (MQTM/MQTMC2) Mappings.....	70

Notices

The following paragraph does not apply in any country where such provisions are inconsistent with local law.

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore this statement may not apply to you.

References in this publication to IBM products, programs, or services do not imply that IBM intends to make these available in all countries in which IBM operates.

Any reference to an IBM licensed program or other IBM product in this publication is not intended to state or imply that only IBM's program or other product may be used. Any functionally equivalent program that does not infringe any of the intellectual property rights may be used instead of the IBM product. Evaluation and verification of operation in conjunction with other products, except those expressly designated by IBM, is the user's responsibility.

IBM may have patents or pending patent applications covering subject matter in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries, in writing, to the IBM Director of Licensing, IBM Corporation, 500 Columbus Avenue, Thornwood, New York 10594, USA.

The information contained in this document has not been submitted to any formal IBM test and is distributed AS IS. The use of the information or the implementation of any of these techniques is a customer responsibility and depends on the customer's ability to evaluate and integrate them into the customer's operational environment. While each item has been reviewed by IBM for accuracy in a specific situation, there is no guarantee that the same or similar results will be obtained elsewhere. Customers attempting to adapt these techniques to their own environments do so at their own risk.

The following terms are trademarks of the International Business Machines Corporation in the United States and/or other countries:

WebSphere® MQ

IBM®

AIX®

MVS™

z/OS®

The following terms are trademarks of the Microsoft Corporation in the United States and/or other countries:

Windows® 95, 98, Me

Windows NT, 2000, XP

~~Preface~~

This MA95 SupportPac provides a Rexx Interface for IBM WebSphere MQ Version 6.0 and later for Windows and z/OS. It permits the usage of MQ functions within the Rexx Environment.

This interface is different to that described in the *WebSphere MQ Application Programming Reference Version 6.0 (SC34-6596)* book, as the API is customised for the Rexx environment. However, with a few exceptions, all the function described in the APR is available. Some extensions to the API are also provided to ease the usage of the interface.

The interface described within this SupportPac, should not be taken to be part of the official MQ Product API, nor should the interface itself be considered part of the official MQ product.

Prerequisites

The SupportPac requires:

- z/OS Version 1.6 or later plus WebShere MQ for z/OS Version 6 or 7
- Windows 2000 or Windows XP or Windows 2003 or later plus IBM Object Rexx for Windows Interpreter Edition plus WebShere MQ for Windows Version 6 or 7

What is in this SupportPac:

- REXX Function Pack load module provides Rexx support for WebSphere MQ for z/OS.
- Two DLLs which provide Rexx support for WebSphere MQ for Windows. One DLL supports access to a local Queue Manager, whilst the other DLL provides client access to a server Queue Manager.
- Rexx Execs which demonstrate usage of the interface.
- Source code of SupportPac modules as a reference for creating similar interfaces.
- This paper which documents the interface.

Acknowledgements

This SupportPac is based on the MA18, MA19, MA77 and MA78 SupportPacs written by Robert Harris of IBM Hursley.

Other SupportPacs

This SupportPac replaces the following outdated SupportPacs:

MA18	A SupportPac providing this interface for Rexx and MQSeries for MVS/ESA
MA19	A SupportPac containing a Rexx interface for Rexx and MQ for MVS/ESA for the issuing of MQSC commands.
MA77	A SupportPac providing this interface for Rexx and MQSeries for Windows NT
MA78	A SupportPac containing a Rexx interface to MQSeries for Windows NT for the issuing of MQSC commands.

Chapter 1. Introduction

This SupportPac provides a Rexx interface, within the z/OS and Windows environment, for WebSphere Message Queueing access.

One **IRXFUSER** load module is provided for z/OS environment. It is the so-called user REXX function pack which, when placed to the commonly available load library, allows the user to call all the provided interface functions without additional preparations. This load module is automatically placed in memory during LOGON, and all interface functions become readily available for use.

Alternative setup may be used, if it is not possible to exploit REXX function pack advantages. SupportPac load library contains alias names for every exported function, and user should issue **TSOLIB** command against this library to make them available for REXX procedures. The first call to **RXMQINIT** function takes care of loading the executable into memory, and the last call to **RXMQTERM** unloads the interface.

Two Dynamically Linkage Libraries (DLLs) are provided for Windows environment:

RXMQN A DLL that provides access to a Queue Manager located on the same workstation

RXMQT A DLL that provides access to a remote Queue Manager located on the different workstation via WebSphere MQ Client

Both DLLs contain exactly the same function, and has the same interface, however, both of them cannot be used in one REXX program simultaneously, because they are linked with different MQ API libraries. See below how to properly install the interface.

A full implementation of the API as described in the *WebSphere MQ Application Programming Reference Version 6.0 SC34-6596* (referenced below as APR) is provided, so this book will be needed to use the Rexx Interface. However, there are a few restrictions:

- *MQINQ* only permits a single attribute to be examined, as support for multiple access is too complicated in the Rexx environment
- *MQSET* only permits the setting of a single attribute
- *MQCONN* and *MQBEGIN* functions are not implemented

In addition to the standard API functions, the Rexx Interface provides a number of extensions to the API to ease the coding with the Exec:

- Browse function is provided
- Header Extraction function is provided to split up a message from a Transmission Queue or a Dead letter Queue into its components
- Event Interpretation function is provided to split up a message from an Event Queue into its components
- Trigger Message function is provided to split up a Trigger message from an Initiation Queue and to generate/parse execution parameters
- Rexx Interface for manipulation of MQ objects (the function that is provided via the MQSC command).

Implementation of the last function is according with the *WebSphere MQ Script (MQSC) Command Reference Version 6.0 SC34-6597*, so this book will be needed to use the Rexx Interface. However, bear in mind:

- The + - line extender symbols are not required or supported
- The underlying function is provided by use of the PCF ESCAPE command (see *WebSphere MQ*

MA95: A Rexx Interface to WebSphere MQ

Programmable Command Formats and Administration Interface Version 6.0 SC34-6598 for details)

- Some non-printable characters are returned.

Chapter 2. Installing the SupportPac

Take the following actions to install the SupportPac

1. Save the file **MA95.zip** to a temporary directory.
2. Uncompress using any unzip program keeping the original directory structure. This will produce the following directories:

MA95\	
bin\	contains the binary ready-to-use files of SupportPac
doc\	contains this MA95.pdf file that you are reading
MVS\	contains MVS specific files
ASM\	MVS Assembler source module (REXX function package)
Include\	MVS specific include files to use during recompile
JCL\	job samples for recompiling and running tests
samples\	contains sample REXX programs to show the interface usage
src\	contains source code of SupportPac modules common to Windows and MVS
Windows\	contains Windows specific files
Compile\	contains files necessary to compile on Windows
Runtests\	contains batch files for running samples
Util\	contains files necessary to recreate REXX constants definitions

Installing on Windows

Take the following actions to install the SupportPac on Windows:

Copy **rxmqn.dll** or **rxmqt.dll** from **MA95\bin** into a suitable directory contained in the **PATH** entry of the System -> Advanced -> Environment Variables (obtained via the Control Panel). If you want to restrict usage, then place in the User variables section, but it is recommended to place DLL in the System Variables section. Do not try to place both DLLs in the same commonly accessed directory. If both DLLs are called from the same REXX program the results are unpredictable.

Installing the DLLs

To use the Rexx WebSphere MQ function within a Rexx Exec, the relevant DLL must be made known to Rexx, and then the interface initialized.

To load the Local Queue Manager interface, code:

```
rcc = RxFuncAdd('RXMQINIT', 'RXMQN', 'RXMQINIT' )
rcc = RXMQINIT()
```

To load the Client/Server interface, code:

```
rcc = RxFuncAdd('RXMQINIT', 'RXMQT', 'RXMQINIT' )
rcc = RXMQINIT()
```

Once this initialization sequence has been done, then the Rexx MQ interface is ready for use.

Please note these operational characteristics:

- The **RxFuncAdd** operations need only be done once for the whole of the Object Rexx environment.
- The **RXMQINIT** call need to be done for each REXX EXEC run (ie: each Process using the interface).
- The **RXMQTERM** call removes the definitions of the functions (so, if two EXECs were running together, the second would start failing after the first issued the **RXMQTERM** call). Consequently, these may be omitted under normal circumstances.

In a thread based environment, a thread should be initialized via **RXMQCONS** (**RXMQINIT** should only be called in the initial process thread)

Compiling the Code for Windows

If you want to add function to the interface (or bring it up to current WebSphere MQ level), then you may choose to compile the code. You will need the Object Rexx Toolkit along with the WebSphere MQ libraries and includes.

Two batch files are provided to compile the new version of the interface using the Microsoft Development Studio for Visual C++:

- **MA95\Windows\Compile\make_n.bat** to create a DLL that provides access to a Queue Manager located on the same workstation
- **MA95\Windows\Compile\make_t.bat** to create a DLL that provides access to a remote Queue Manager located on the different workstation via WebSphere MQ Client

Before starting the compile ensure that the C++ source code and headers in **MA95\src** are in the INCLUDE path as well as WebSphere MQ and Object Rexx Toolkit headers and library files.

Another task you may want to do is upgrading WebSphere MQ constants defined in REXX to a higher level. These constants are defined in **MA95\src\RXMQCONS.hpp** file which can be recreated using the following instructions:

- Place current WebSphere MQ include files **CMQC.H** and **CMQCFC.H** to the same directory as **MA95\Windows\Util\makecons.bat**
- Make changes, if desired, to **MA95\Windows\Util\rxmqcons.cfg** which contains the list of prefixes of constants to be pulled out of the WebSphere MQ include files

- Run the batch file
- Recompile the interface DLLs (as described above) or MVS function pack (as described below)

Installing on z/OS

Take the following actions to install the SupportPac from the unpacked **MA95.ZIP** file:

- **MA95\bin\MA95.xmt** contains the load library to be transferred to z/OS system. It needs to be transferred to the destination TSO system as a sequential binary file with a record format of FB 80.
- To send the file via ftp open the session with the target host. Ensure the **BINARY** option is set, then use the following commands:
quote site fixrecfm 80
put MA95.xmt MA95xmit
- On TSO, issue this command:
receive indsnam(MA95XMIT)
- When prompted for a filename, reply
dsn(MA95.LOAD)
- This creates a PDS called **mvsuserid.MA95.LOAD** with SupportPac executables
- Use ISPF 3.2 to delete the **MA95XMIT** file
- Use ISPF 3.3 to copy all members from **mvsuserid.MA95.LOAD** into the load library of your choice. If you want other users to access the MA95 interface, select a common access load library (like LINKLIB concatenation). SupportPac is automatically recognized by TSO as a REXX function pack during LOGON, and interface functions become readily available for use. Otherwise use a private load library, which may be made accessible from your TSO/ISPF sessions by the use of **TSOLIB ACTIVATE DSNAM(MA95.LOAD)** command.
- To permit tracing to appear, ensure that the following DD-statement is in your TSO LOGON Procedure (or dynamically issue a TSO **ALLOCATE FI(SYSPRINT) SYSOUT** command)
//SYSPRINT DD SYSOUT=*

The MVS file names have been described are an example. Please use whatever conventions are suitable for your installation.

To use the Rexx WebSphere MQ function within a Rexx Exec, nothing special has to be done to make it known to Rexx (compared to Windows **RxFuncAdd** call).

Compiling the Code for z/OS

If you want to add function to the interface (or bring it up to current WebSphere MQ level), then you may choose to compile the code. You will need IBM C/C++ Compiler and Language Environment and High Level Assembler for z/OS along with the WebSphere MQ libraries and includes. Another task you may want to do is upgrading WebSphere MQ constants defined in REXX to a higher level. The latter task is described in Compiling the Code for Windows on page 4.

To assist with re-building the code, sample jobs are provided in **MA95\MVS\JCL** directory. They are:

- **ASM95** – use this one to assemble REXX function package part of SupportPac
- **CC95** – use this one to compile C++ part of SupportPac
- **LKED95** – use this job to bind Assembler, C++ parts and WebSphere MQ libraries into load module.

These sample jobs refer to WebSphere MQ, C/C++ and Language Environment libraries (which names you may need to update for your installation) as well as SupportPac libraries you will need to create. It is recommended to use PDSE type libraries. Suggested names and attributes for them are as follows:

- **WMQ.MA95.ASM** – RECFM=FB, LRECL=80
- **WMQ.MA95.CC** – RECFM=VB, LRECL=255
- **WMQ.MA95.OBJ** – RECFM=FB, LRECL=80
- **WMQ.MA95.LOAD** – RECFM=U, LRECL=32760

First, you will need to transfer the source code from **MA95\src** to **WMQ.MA95.CC** on your z/OS system. Then transfer MVS specific code from **MA95\MVS\ASM** to **WMQ.MA95.ASM** and **MA95\MVS\Include** to

WMQ.MA95.CC. Now make necessary changes to the code and submit 3 sample jobs to build the load module.

TSO Support

TSO Batch Support

The **MA95** interface will run in Batch mode via **IKJEFT01**, using a setup such as:

```
//LIB          EXEC PGM=IEBGENER
//*
//*              Create the exec library
//*
//SYSUT2        DD      DSN=&&LIB(SILLY),DISP=(NEW,PASS),
//                  UNIT=SYSDA,
//                  SPACE=(CYL,(1,1,10)),
//                  DCB=(DSORG=PO,RECFM=FB,LRECL=80,BLKSIZE=800)
//SYSPRINT       DD      DUMMY
//SYSIN DD       DUMMY
//SYSUT1        DD      DATA,DLM='##'
/* A Silly Exec */
RXMQINIT()
say 'QM1 is very very very very silly'
RXMQTERM()
exit 0
/* End of SILLY exec */
##
/*
//RUN           EXEC PGM=IKJEFT01
//SYSPROC       DD      DSN=&&LIB,DISP=SHR
//STEPLIB       DD      DSN=WMQ.MA95.LOAD,DISP=SHR
//SYSTSPT       DD      SYSOUT=*,DCB=(RECFM=F,LRECL=132,BLKSIZE=132)
//SYSPRINT      DD      SYSOUT=*
//SYSUDUMP      DD      SYSOUT=*
//SYSTSIN       DD      *
SILLY
/*
```

Figure 1. TSO Batch JCL

TSO Native Support

When running within a native TSO Exec (ie: one issued outside of the ISPF environment, or via the TSO command within ISPF), the Rexx processor is attached to TSO as a separate TCB. Consequently, the connection to the Queue Manager will only last throughout the lifetime of the Exec. If processing is interrupted via PA1, then the Rexx processor TCB is terminated, and so the MQ Step termination routines will be driven to terminate all extant accesses.

TSO Split Screen Support

When running within a ISPF Split Screen, the Rexx processor is attached to each part as a separate TCB. Consequently, the same considerations apply as in “TSO Native Support,” so the Queue Manager connection, and the Rexx variables cannot be shared between the two halves.

Chapter 3. Interface Design Philosophy

The Rexx MQ Interface API differs from that defined in the APR. This is because the call-type of API is not suitable for the Rexx environment. This has been replaced with a set of verbs that use Rexx Stem variables to contain the relevant information.

The opportunity has also been taken to remove some parameters due to the restriction that a single Windows thread or z/OS task (Exec in the Rexx environment) can only communicate with a single Queue Manager. Additionally, in order to simplify coding, Input and Output versions of object are provided (this saves deleting and rebuilding things like Message descriptors which are updated by a MQ Verb).

As part of the initialization call, a number of MQ Constants (as described in *WebSphere MQ Constants Version 6.0 SC34-6607*) are defined to the Rexx workspace. Thus, you will be able to code options according to the descriptions in this book. However, these values are not protected against change, so you should avoid using your own variables starting with MQ.

The command interface takes a string of characters in the format of a text command as described in the MQ Command reference. This is then placed in the Command Input Queue in the format of a PCF ESCAPE command. The Command server processes the command, and returns the results in a dynamically generated ReplyToQ (called **RXMQC.***) based upon the *ModelReplyToQ* specified. Each message is returned as an element of a Rexx Stem variable, with the number of messages being in stem.0 in the usual Rexx fashion. These returned messages may have non-printable characters in them.

Chapter 4. General Points

Compatibility with Previous SupportPacs

The previous SupportPacs, namely, MA18/19 for MVS/ESA and MA77/78 for Windows, used different names for the same implemented WebSphere MQ functions. That is why it was not possible to write a single REXX exec to execute in both environments.

In order to provide this capability the function names were unified to provide for running the same REXX exec unchanged on z/OS and Windows. The MA95 Common Function names are presented below in the table below. Sample REXX execs which run equally in both environments are provided in **MA95\samples** directory and explained more in on page 75.

To maintain compatibility with previous SupportPacs the old function names are still serviced, so that existing REXX execs are not required to be changed. For example, the calls to **RXMQN.../RXMQT...** in Windows and **RXMV...** in z/OS are supported and should provide the same results as new Common Functions names. The parameter lists remain unchanged.

In the rest of this document only the new Common Function names (**RXMQxxxx**) are described. See the table below for relationship between new and old function names specified in older manuals.

Table 1. Function Names

MQ Function Name	MA95 Common Function	MA77/78 Windows Function	MA18/19 MVS/ESA Function	Page
Standard MQ functions:				
<i>MQBACK</i>	RXMQBAC	RXMQNBAC	RXMQVBAC	43
<i>MQCLOSE</i>	RXMQCLOS	RXMQNCLOSE	RXMQVCLOSE	35
<i>MQCMIT</i>	RXMQCMIT	RXMQNCMIT	RXMQVCMIT	41
<i>MQCONN</i>	RXMQCONN	RXMQNCONN	RXMQVCONN	30
<i>MQDISC</i>	RXMQDISC	RXMQNDISC	RXMQVDISC	32
<i>MQGET</i>	RXMQGET	RXMQNGET	RXMQVGET	44
<i>MQINQ</i>	RXMQINQ	RXMQNIQ	RXMQVINQ	37
<i>MQOPEN</i>	RXMQOPEN	RXMQNOPEN	RXMQVOPEN	33
<i>MQPUT</i>	RXMQPUT	RXMQNPUT	RXMQVPUT	47
<i>MQPUT1</i>	RXMQPUT1	--	--	50
<i>MQSET</i>	RXMQSET	RXMQNSET	RXMQVSET	39
Extension functions:				
Initialization	RXMQINIT	RXMQNIQ	RXMQVINQ	27
Thread initialization	RXMQCONS	RXMQNCONS	RXMQVCONS	28
Termination	RXMQTERM	RXMQNTERM	RXMQVTERM	29
Command function	RXMQC	RXMQCNC	RXMQVC	54
Browse	RXMQBRS	RXMQNBROWSE	RXMQVBROWSE	56
Header Extraction	RXMQHXT	RXMQNHXT	RXMQVHXT	58
Event Extraction	RXMQEVNT	RXMQNEVENT	RXMQVEVENT	63
Trigger Extraction	RXMQTM	RXMQNTM	RXMQVTM	67
Universal function	--	--	RXMQV	--

MQ Function Name	MA95 Common Function	MA77/78 Windows Function	MA18/19 MVS/ESA Function	Page
caller				
MQSUB	RXMQSUB	RXMQNSUB	RXMQVSUB	
MQCRTMH	RXMQMH	RXMQNMH	RXMQVMH	
MQDLTMH	RXMQDMH	RXMQNDMH	RXMQVDMH	
MQSETMP	RXMQSMP	RXMQNSMP	RXMQVSMP	
MQINQMP	RXMQIMP	RXMQNIMP	RXMQVIMP	
MQDLTMP	RXMQDMP	RXMQNDMP	RXMQVDMP	
MQMHBUF	RXMQMBF	RXMQNMBF	RXMQVMBF	
MQBUFMH	RXMQBMH	RXMQNBMH	RXMQVBMH	

Warning. Some minor incompatibilities may still exist between the old and new SupportPacs due to optimizations made during the code merge. For example, small changes were made to improve EVENT queue message parsing algorithm (see RXMQEVNT on page 63). If you encounter a problem, please contact the authors to get workaround or have your suggestion included in the next release of SupportPac.

Return Codes

All the **RXMQ** functions return a standard Rexx return string. This is structured so that the numeric Return Code (which may be negative) is obtained by a **word(RCC, 1)** call.

The Return Code for an operation can be negative to show that the interface has detected an error, otherwise it will be the MQ Completion Code (not the uninformative Reason Code).

The Return String is in text format as follows:

Word 1 Return Code
Word 2 MQ Completion Code (or 0 if successful)
Word 2 MQ Reason Code (or 0 if successful)
Word 4 **RXMQ** function being run
Word > OK or an helpful error message

Last Operation

In addition, the current (ie: the settings last set) values are available in these variables:

RXMQ.LASTRC current operation Return Code
RXMQ.LASTCC current operation MQ Completion Code
RXMQ.LASTAC current operation MQ Reason Code
RXMQ.LASTOP current operation **RXMQ** function name
RXMQ.LASTMSG current operation Return String

Return Code Naming

A set of variables called **RXMQ.RCMAP.nn** are also placed in the workspace, where nn is the MQ Reason Code. These variables can be used to turn a return code number into the defining string.

Thus:

```
rcc = '2048 2 2048 RXMQPUT ERROR'
interpret 'fcs = RXMQ.RCMAP.'word(rcc,1)
/* fcs = MQRC_PERSISTENT_NOT_ALLOWED */
```

Message Lengths

When a *MQGET* is pershaped, if the buffer size is too small for the message, then the returned message length is the **real** length of the message, not the smaller size which fits in the buffer (see *DataLength* for *MQGET* in the APR).

Consequently, if you specify a too small a message length, and do not take any notice of the return code indicating truncation, then the length of the message in stem.0 will be **bigger** than the message in stem.1. This will result in a mysterious loss of data in the message.

Header and Event processing

Functions **RMQHXT** and **RXMQEVNT** will take messages and split them up into the contained components. These exploded components may clash with those for the Message Descriptor (or other like things). Therefore, use different **stem.** names to avoid this possibility.

ZLIST

One of the problems with REXX **stem.** variables is that it is difficult to know what components (things after the **.**) are associated with the **stem.** You have to know which ones might be around, and then test with something like:

```
if ( stem.comp1 <> 'STEM.COMP1' ) then say 'comp1 =/'stem.comp1 '/'
if ( stem.comp2 <> 'STEM.COMP2' ) then say 'comp2 =/'stem.comp2 '/'
```

To get around this problem, the **output** descriptors will contain a component called **ZLIST**. **ZLIST** will contain a list of words, each word a component name which is attached to the stem variable. You can then use the Rexx words (to get the number of elements) and word (to extract the component name) functions to manipulate the stem. variable. **ZLIST** does not contain itself (ie: **ZLIST** is not within **stem.ZLIST**).

The presence of an item in **ZLIST** implies that the relevant Stem.Component is **defined** as a Rexx Variable. However, the contents may be null (a length of zero or set to **' '**) depending upon what the underlying MQ object contains.

This facility is not of much use for the **RXMQOPEN**, **RXMQGET** and **RXMQPUT** calls (wherein **ZLIST** is provided for the Output Object Descriptor, Output Message Descriptor, Output Get Message Options and Output Put Message Options) as the contents of the Output **stem.** variable is of fixed format. However, it can be used to display the **stem.** variable and can also be useful in copying operations.

For **RXMQHXT** and **RXMQEVNT** processing, **ZLIST** is of variable format, containing things relevant to the Message or Event being processed. **ZLIST** for **RXMQHXT** processing contains components 0 and 1 (the original message) as well as **NAME** and **TYPE**. For **RXMQEVNT** processing, **NAME**, **TYPE** and **REA** are always present; the rest of the list will depend upon the event being processed.

For example to display an Object Descriptor:

```
drop iod. ; drop ood.
iod.on = 'N1'
iod.ot = MQOT_Q

rcc = RXMQOPEN('iod.', mqoo_inquire, 'h1', 'ood.')
say 'RC=' rcc 'H=' h1
do j=1 to words(ood.zlist) ; k = word(ood.zlist,j) ; say k '/'ood.k '/' ; end
```


ZLIST can be used for Event processing:

```

drop bm. ; drop ed.
rcc = RXMQBRWS( he, 'bm.')
say 'Browse RC=' rcc

rcc = RXMQEVNT('bm.', ' ed.')
say 'Event RC =' rcc
say '.zlist /'ed.zlist/'

/* Protect against bad function by being very cautious! */
if ( (ed.zlist <> 'ED.ZLIST') & (words(ed.zlist) <> 0) ) then ,
  do j=1 to words(ed.zlist)
    k = word(ed.zlist,j)
    say 'ed.'k' /'ed.k'/'
  end
end

/* I'm only interested in Channel Stopped Events */
/*
/* However, do not want to access undefined
/* components.
/*
/*

if ( ed.name = 'CHANNEL_STOPPED' ) then do
  uvars = 'Q_MGR_NAME CHANNEL_NAME REASON_QUALIFIER ERROR_IDENTIFIER'
  do i=1 to words(uvars)
    uv = word(uvars,i)
    if ( wordpos(uv,ed.zlist) <> 0 ) then ,
      say uv '= <'ed.uv'>'
  end
end

/* So, if PN is not set within the Event
/* (it's an optional parameter), it will
/* not be accessed.
/*

```

Figure 2. ZLIST and Event processing

Stem Variables

As described in “” on page 17, Stem variables are extensively used in this interface. A Stem variable is one that has various bits separated by dots (such as **a.b.c**). Everything after the first dot is called a component; so in the above example, **a** is the Stem variable, and **b** & **c** are components.

You should be aware that you can cause conflicts if you use Rexx variables with the same name as components. This is because Rexx will **substitute** the values of component names as if they were variables before usage.

```
a.1 = 15
a.2 = 3

b = 2
say a.b    /* -> 3 due to substitution */
```

This can cause problems if you use any of the returned component names from this utility as native variables because you will get an 'unknown' setting due to the substitution.

```
qn = 'QM1'
ud = 'some userish data'

rcc = RXMQ...(data_which_will_set_.qn=A , 'out.')

say out.qn  /* tries to resolve out.QM1          */
            /*          -> A                    */
            /* as the utility does the substitution */

say out.ud  /* tries to resolve out.some userish data */
            /* -> a Rexx error due to invalid var name */
```

Unless you are deliberately doing this sort of processing, I suggest you avoid using variables which are returned as components.

Rexx Execs

An Object Rexx exec in the Windows environment behaves differently from that in the z/OS environment due to the way the function is placed in regard to the operating system. One of these is in the way one runs an exec.

Under Windows a Rexx exec is run via

```
REXX exec.rexx
```

whereas under z/OS the exec is run via

```
exec
```

to make the latter style available under Windows, you can code up a one line batch file as follows:

```
@REXX exec.rexx %1 %2 %3 %4 %5 %6 %7 %8 %9
```

Trace

Tracing is provided by settings in the **RXMQTRACE** Rexx variable. Note that the tracing is sent to the currently open STDOUT stream, and some of the settings can produce a lot of output. The settings are:

CONN	<i>MQCONN</i>
DISC	<i>MQDISC</i>
OPEN	<i>MQOPEN</i>
CLOSE	<i>MQCLOSE</i>
GET	<i>MQGET</i>
PUT	<i>MQPUT</i>
PUT1	<i>MQPUT1</i>
INQ	<i>MQINQ</i>
SET	<i>MQSET</i>
CMIT	<i>MQCMIT</i>
BACK	<i>MQBACK</i>
COM	<i>MQSC</i>
BRO	Browse extension
HXT	Header extraction extension
EVENT	Event expansion extension
TM	Trigger message extension
MMD	Rexx stem var -> MQMD
MOD	Rexx stem var -> MQOD
MPO	Rexx stem var -> MQPMO
MGO	Rexx stem var -> MQGMO
BMD	MQMD -> Rexx stem var
BOD	MQOD -> Rexx stem var
BPO	MQPMO -> Rexx stem var
BGO	MQGMO -> Rexx stem var
SK	Return Code processing
TR	Thread based processing
INIT	Initialization processing (including RXMQCONS)
TERM	Deregistration processing
*	Trace everything!!!

So, to trace Gets and Puts, one would code

```
RXMQTRACE = 'PUT GET'
```

Chapter 5. Handling MQ Descriptors

The API defined for MQ in the APR Manual uses various structures to pass information both into and out of the Queue Manager. These structures are:

MQOD	The Object Descriptor, used by <i>MQOPEN</i> and <i>MQPUT1</i> verbs to specify the MQ Object being processed, and return various attributes of the accessed item
MQMD	The Message Descriptor, used by <i>MQGET</i> and <i>MQPUT</i> verbs to specify (for the <i>MQPUT</i>) attributes for the emplaced message, and return these attributes (for the <i>MQGET</i>)
MQGMO	This structure controls the operation of the <i>MQGET</i> verb
MQPMO	This structure controls the operation of the <i>MQPUT</i> verb
MQCHARV	This structure represents variable length strings used in MQOD structure

These structures are input/output for the MQ Verbs.

In order to supply these structures to the underlying MQ Verbs within this Rexx MQ Interface, Rexx **stem** variables are used. In order to reduce complexity, and enhance the ease of usage of the interface, **separate** Stem variables are used for input and output. This reduces the complexity of the Rexx code, as the input Stem variable may be reused without completely reinitializing it.

This approach allows, for simple applications, the initial setup of the stem variables representing the requested options; these are then repeatedly reused, the output versions simply not being accessed.

The **structure** of the stem variables is **fixed**. That means that the name of the stem variable (before the dot) can be chosen by the caller, whilst the latter part (after the dot) is fixed by the interface. The things after the dot are called the **Components** of the stem variable.

The normal Rexx rules apply to these Stem variables, in particular they are case invariant (Rexx treats all variables as being of Upper case), and substitution may occur within the name. Therefore, take care to avoid using variables that could clash with the naming conventions of these interface requirements (see "Stem Variables" on page 14).

When supplying these stem variables to the interface, you have to pass the **name** of the stem variable (including the trailing dot). Thus, one would normally specify this information as a literal (**RXMQ...** (..., '**AGMO.**', ...)).

However, you are at liberty to use the normal Rexx substitutions on an interface call (so **Z = 'AGMO.'**; **RXMQ...** (..., **z**) is correct), and even abandon the stem variable convention completely (but this will lead to unwieldy execs). This abandonment, however, does not apply to one of the **RXMQOPEN** parameters.

When you build the stem variable, component abbreviations for the full name of the relevant structure's field is used (eg: **CCID** for *CodedCharSetId*) to improve legibility of the Exec. You only specify those fields of interest - the others should be **omitted**. The omitted components will default to the relevant settings as defined in the APR (usually a value or nulls).

However, although some fields of the descriptors are only used for input or output, this interface will utilize all of the information within the Stem variable - even if it is not used by the underlying MQ code (such as the Destination Count fields within the **PMO** descriptor - these are not used by the underlying MQ code, but this interface will process them if so supplied).

When the interface returns a structure to the exec, in the named Stem variable, **all** the components (fields) will be placed within the stem.structure.

The actual settings for these component variables are documented in the MQ APR to which you should refer. As the interface places within the Rexx workspace all **MQ_** numeric values, the stem components can be set using the normal MQ conventions (eg: **stem.PER = MQMD_NOT_PERSISTENT**). The interface does not check that the values are relevant for the field.

In the case of text fields, the interface will truncate supplied data that is too long for the MQ structure without notification. Fields that are to be null should not be supplied to the interface, and are returned as nulls ('').

Actual message data to/from the Queue Manager is passed via the usual Rexx convention (see “Message Lengths” on page 11 for a warning about truncation):

stem.0 contains the length of the data

stem.1 contains the message data

Functions **RXMQHXT** and **RXMQEVNT** will take messages and split them up into the contained components. These exploded components may clash with those for the Message Descriptor (or other like things). Therefore, use different stem. names to avoid this possibility.

ZLIST processing (see “ZLIST” on page 12) is available for the **Output** Stems representing a **MQOD**, **MQMD**, **MQGMO** or **MQPMO**. If present within an Input Stem. variable, **ZLIST** is ignored.

The Object Descriptor

The Object descriptor is used by **RXMQOPEN** and **RXMQPUT1** calls (*MQOPEN* and *MQPUT1* verbs) to specify the MQ Object being processed, and return various attributes of the accessed item.

If you are accessing a Queue, then the short cut form of **RXMQOPEN/RXMQPUT1** can be used, and so the Object Descriptor is only of interest upon completion of the call. The only interesting part of the **MQOD** in this case is the name of the 'real' queue generated when a Model queue is opened.

Table 2. Object Descriptor (MQOD) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
Version 1			
<i>Version</i>	.VER	I	MLONG
<i>ObjectType</i>	.OT	I	MLONG
<i>ObjectName</i>	.ON	B	MQCHAR48
<i>ObjectQMgrName</i>	.OQM	B	MQCHAR48
<i>DynamicQueueName</i>	.DQN	I	MQCHAR48
<i>AlternateUserID</i>	.AUID	I	MQCHAR12
Version 2			
<i>RecsPresent</i>	.RP	I	MLONG
<i>KnownDestCount</i>	.KDC	O	MLONG
<i>UnknownDestCount</i>	.UDC	O	MLONG
<i>InvalidDestCount</i>	.IDC	O	MLONG
Version 3			
<i>AlternateSecurityID</i>	.ASID	I	MQBYTE40
<i>ResolvedQName</i>	.RQN	O	MQCHAR48
<i>ResolvedQMgrName</i>	.RQMN	O	MQCHAR48
Version 4			
<i>ObjectString</i>	.OS.	I	MQCHARV
<i>SelectionString</i>	.SS.	I	MQCHARV
<i>ResObjectString</i>	.ROS.	O	MQCHARV
<i>ResolvedType</i>	.RT	O	MLONG

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names
- Components **RP** and **ASID** are supported only in Windows

The Message Descriptor

The Message Descriptor details the type of the message being processed. It also has a meaning where messages are obtained from a queue - whereat it is used to select messages for obtention from the queue. The interface does not check that combinations of components are valid.

As separate versions of a Message Descriptor are required by the interface for Input and Output on each call, the input **MQMD** can be reused for subsequent accesses. Components omitted will take the defaults as defined in the APR.

Table 3. Message Descriptor (MQMD) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
Version 1			
<i>Version</i>	.VER	I	MQLONG
<i>Report</i>	.REP	O/I	MQLONG
<i>MsgType</i>	.MSG	O/I	MQLONG
<i>Expiry</i>	.EXP	O/I	MQLONG
<i>Feedback</i>	.FBK	O/I	MQLONG
<i>Encoding</i>	.ENC	O/I	MQLONG
<i>CodedCharSetId</i>	.CCSI	O/I	MQLONG
<i>Format</i>	.FORM	O/I	MQCHAR8
<i>Priority</i>	.PRI	O/I	MQLONG
<i>Persistence</i>	.PER	O/I	MQLONG
<i>MsgId</i>	.MSGID	B/B	MQBYTE24
<i>CorrelId</i>	.CID	B/I	MQBYTE24
<i>BackoutCount</i>	.BC	B/-	MQLONG
<i>ReplyToQ</i>	.RTOQ	O/I	MQCHAR48
<i>ReplyToQMgr</i>	.RTOQM	O/I	MQCHAR48
<i>UserIdentifier</i>	.UID	O/B	MQCHAR12
<i>AccountingToken</i>	.AT	O/B	MQBYTE32
<i>ApplyIdentityData</i>	.AID	O/B	MQCHAR32
<i>PutApplType</i>	.PAT	O/B	MQLONG
<i>PutApplName</i>	.PAN	O/B	MQCHR28
<i>PutDate</i>	.PD	O/B	MQCHAR8
<i>PutTime</i>	.PT	O/B	MQCHAR8
<i>ApplOriginData</i>	.AOD	O/B	MQCHAR4
Version 2			
<i>GroupId</i>	.GID	B/B	MQBYTE24
<i>MsgSeqNumber</i>	.MSN	B/B	MQLONG
<i>Offset</i>	.OFF	B/B	MQLONG
<i>MsgFlags</i>	.MF	B/B	MQLONG

Structure name	Stem. Component	Input, Output or Both	Format
<i>OriginalLength</i>	.0L	O/B	MQLONG

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names

The Get Message Option Structure

The Get Message Option Structure requests what message is to be obtained from a queue via the *MQGET* verb. As it is updated by this operation, **RXMQGET** uses an Input and Output Stem variable to hold this information.

Table 4. Get Message Option (MQGMO) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
Version 1			
<i>Version</i>	.VER	I	MLONG
<i>Options</i>	.OPT	I	MLONG
<i>WaitInterval</i>	.WAIT	I	MLONG
<i>ResolvedQueueName</i>	.RQN	O	MQCHAR48
Version 2			
<i>MatchOptions</i>	.MOPT	I	MLONG
<i>GroupStatus</i>	.GS	O	MGCHAR
<i>SegmentStatus</i>	.SS	O	MGCHAR
<i>Segmentation</i>	.SEG	O	MGCHAR
Version 3			
<i>MsgToken</i>	.MT	B	MQBYTE16
<i>ReturnedLength</i>	.RL	O	MLONG
Version 4			
<i>MsgHandle</i>	.MH	I	MQHMSG

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names

The Put Message Options Structure

The Put Message Option Structure requests what type of message is to be placed in a queue via the *MQPUT* or *MQPUT1* verb. As it is updated by this operation, **RXMQPUT** and **RXMQPUT1** uses an Input and Output Stem variable to hold this information.

Table 5. Put Message Options (MQPMO) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
Version 1			
<i>Version</i>	.VER	I	MQLONG
<i>Options</i>	.OPT	I	MQLONG
<i>Timeout</i>	.TIME	I	MQLONG
<i>Context</i>	.CON	I	HOBj
<i>KnownDestCount</i>	.KDC	O	MQLONG
<i>UnKnownDestCount</i>	.UDC	O	MQLONG
<i>InvalidDestCount</i>	.IDC	O	MQLONG
<i>ResolvedQueueName</i>	.RQN	O	MQCHAR48
<i>ResolvedQueueMgrName</i>	.RQMN	O	MQCHAR48
Version 2			
<i>RecsPresent</i>	.RP	I	MQLONG
Version 3			
<i>OriginalMsgHandle</i>	.OMH	I	MQHMSG
<i>NewMsgHandle</i>	.NMH	B	MQHMSG
<i>Action</i>	.ACT	I	MQLONG
<i>PubLevel</i>	.PL	I	MQLONG

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names
- Component **RP** is supported only in Windows.

The Variable Length String Structure

The Variable Length String Structure (**MQCHARV**) designates character string of variable length and specific character set identifier.

Actual string data to/from the Queue Manager is passed via the usual Rexx convention:

stem.0 contains the length of the string

stem.1 contains the string itself

stem. name is constructed of the name of the higher structure and the name of this structure component, like, for example, OD.ROS.1 stands for ReturnedObjectString in Object Descriptor structure. Before the call appropriate OD.ROS.0 variable should be set to the biggest of string length and buffer size. During the call VSBufSize is set to OD.ROS.0 and VSLength is set to actual string length. String buffer is automatically created and it's pointer is placed to **MQCHARV** VSPtr. On return from the call VSLength value is placed to OD.ROS.0 variable, returned string is placed to OD.ROS.1 and buffer is released.

Coded character set identifier is assigned to **stem.CCSI** variable.

Specifying parameters for command interface (RXMQC)

The interface requires some parameters to control operation:

- The name of the queue Manager (if omitted, the default Queue Manager is accessed)
- The name of the Command Input Queue (defaults to **SYSTEM.ADMIN.COMMAND.QUEUE**)
- The name of the model ReplyToQ (defaults to **SYSTEM.MQSC.REPLY.QUEUE**)
- A Timeout for the operation (defaults to 5 seconds)

In order to supply this information to the underlying MQ Verbs within this Rexx MQ Interface, Rexx **stem** variables are used.

The **structure** of the stem variables is **fixed**. That means that the name of the stem variable (before the dot) can be chosen by the caller, whilst the latter part (after the dot) is fixed by the interface. The things after the dot are called the **Components** of the stem variable.

The normal Rexx rules apply to these Stem variables, in particular they are case invariant (Rexx treats all variables as being of upper case), and substitution may occur within the name. Therefore, take care to avoid using variables that could clash with the naming conventions of these interface requirements.

When supplying these stem variables to the interface, you have to pass the **name** of the stem variable (including the trailing dot). Thus, one would normally specify this information as a literal (**RXMQC(... , 'ASTEM. ', ...)**).

However, you are at liberty to use the normal Rexx substitutions on an interface call (so **Z = 'ASTEM. '**; **RXMQC(... , Z)** is correct), and even abandon the stem variable convention completely (but this will lead to unwieldy execs). This abandonment, however, does not apply to one of the **RXMQC** parameters.

When you build the stem variable, component abbreviations for the full name of the relevant structure's field is used (eg: **CQ** for Command Queue) to improve legibility of the Exec. You only specify those fields of interest - the others should be **omitted**. The omitted components will default to the relevant settings as defined in the interface.

Actual message data in response to the Command is passed via the usual Rexx convention:

stem.0 contains the length of the data

stem.1 contains the data

~~Chapter 6. Thread Support~~

Some non-IBM Versions of Rexx which run in the Windows environment may support Rexx activity within threads. IBM Object Rexx for Windows does not support threads. There is no multitask support in z/OS REXX environment, so all the thread-based issues in this documentation will be applicable only for Windows Rexx/Threaded environment.

This utility supports access to the API in a Rexx/Threaded environment; however, this function is untested and usage is at your own risk.

Initialization

The initialization of via **RXMQINIT** should be done under the **Process** (main) thread. This will ensure that the functions are available for all the threads which are subsequently created within the process.

Within each thread, **RXMQINIT** **should not** be called (as this would remove all Global information stored within the interface). The thread should be initialized with a **RXMQCONS** call which simply creates the MQ mappings within the Rexx Variable space for the thread.

Termination

The usage of **RXMQTERM** will remove all access to the interface whomsoever is using it with the Process. If this is done before all threads within the Process have stopped accessing MQ facilities, this will result in errors (as the function will affect threads other than the issuing one).

No special processing needs to be done at thread termination, as MQ facilities will terminate any current access within the thread.

Connection and Disconnection

Within the thread, the **RXMQCONN** and **RXMQDISC** call should be made as normal to establish and terminate access to the Queue Manager. Each Thread can contact a different Queue Manager. The scope of the Connection is only within the issuing Thread. Access made in the original processes 'main' thread do **not** spread to the subsequent threads within the process.

When the **RXMQDISC** call is issued, all access to the Queue Manager within the issuing thread are effected. The graceful processing described in "Termination" on page 29 applies to all accesses within the current thread.

Access scope

The handle returned by the **RXMQCONN** and used for most of the access functions of the interface has a scope of the issuing thread. This is policed within the interface to prevent MQ errors.

Functions like **RXMQCMIT** and **RQMQBACK** have a scope of **all** accessed MQ objects **within** the issuing thread; they do not effect accesses outside the thread.

Shared Variables

The interface implicitly assumes that Rexx Variables within a thread **are not shared** across threads. If your version of Rexx supports shared Rexx variables across threads, then ensure that access is suitably restricted (ie: the variables used are unique to the threads) across these threads.

Chapter 7. The Interface

The functions provided by this Rexx MQ interface roughly follow those provided by the underlying MQ API, with some extensions and the calls required by Rexx to initialize the interface.

All the parameters specified for a call are required; none can be omitted.

When the interface detects an error, a **negative** return code will be provided as the first word in the return string. These are documented with the associated message under the individual calls.

Initialization

Description

This function initializes the interface, defines all the functions for Rexx usage, and places a number of the **MQ*** constants into the Rexx workspace. These constants are described in *WebSphere MQ Constants Version 6.0 SC34-6607*. See **MA95\src\RXMQCONS.hpp** for a list of the defined constants.

The **RXMQINIT** call needs to be done within each EXEC. In a thread-based environment, issue this call only in the 'owning' thread for the process.

Parameters

None

Call

```
rcc = RXMQINIT()
```

Additional Interface Return Codes and Messages

None

Example

```
rcc = RXMQINIT()
```

Thread Initialization

Description

This function places a number of the **MQ*** constants into the Rexx workspace for a thread. These constants are described in *WebSphere MQ Constants Version 6.0 SC34-6607*. See **MA95\src\RXMQCONS.hpp** for a list of the defined constants. This function can be called when there is no Queue Manager activity.

The **RXMQCONS** call needs to be the first call within a thread to setup the **MQ*** constants. It can be called generally to do the same function.

Parameters

None

Call

```
rcc = RXMQCONS()
```

Additional Interface Return Codes and Messages

None

Example

```
rcc = RXMQCONS()
```


Termination

Description

This function simply removes the access to the interface functions from Rexx. It does **not** initiate MQ Termination processing. If a prior **RXMQDISC** has not been done, then the usual End-of-Process MQ function will (eventually) stop access to the Queue Manager.

The **MQ*** definitions are left in the Rexx workspace, so that new commands can be composed using the 'real' notations.

As the **RXMQTERM** call removes the definitions of the functions, if two EXECs were running together, the second would start failing after the first issued the **RXMQTERM** call. Consequently, this call should be omitted under normal circumstances.

In a thread-based environment, issue this call **only** in the 'owning' thread for the process when all other accesses have ended.

Parameters

None

Call

```
rcc = RXMQTERM()
```

Additional Interface Return Codes and Messages

None

Example

```
rcc = RXMQTERM()
```

RXMQCONN

Description

This function connects the Rexx Interface to the Queue Manager. Note that there is a MQ restriction such that only one Queue Manager can be contacted from a Windows thread or z/OS task (the Rexx processor, in this case).

This call **has** to be made after the **RXMQINIT** call, and only be made once (unless a **RXMQDISC** is made).

Owing to the above restriction, the Queue Manager Handle returned by the use of *MQCONN* within the interface is not a useful thing, and so is not returned to the Rexx Exec.

Parameters

1. The name of the Queue Manager to connect to. (Input only).

Call

```
rcc = RXMQCONN( QM )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQCONN Bad number of parms

Explanation You must specify only **one** parameter to **RXMQCONN**; this parameter being the name of the Queue Manager to contact.

-2 0 0 RXMQCONN Null QM name

Explanation The Queue Manager name supplied contained only nulls, not a proper name.

-3 0 0 RXMQCONN Zero length QM name

Explanation The Queue Manager Name supplied was of zero length (ie: '').

-4 0 0 RXMQCONN QM name too long

Explanation The maximum length of a valid Queue Manager Name is **MQ_Q_MGR_NAME_LENGTH** bytes.

-5 0 0 RXMQCONN No available Q objects

Explanation The number of concurrent MQ Objects accessed with the process has reached its limit.

-95 0 0 RXMQCONN Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

-96 0 0 RXMQCONN Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

-98 0 0 RXMQCONN Already Connected to a QM

Explanation The current thread has already Connected to a Queue Manager

-99 0 0 RXMQCONN UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
rcc = RXMQCONN( 'QM1' )
```

This call will contact the local Queue Manager called **QM1**. If this Queue Manager is not defined, or not

running, then the call will fail.

If the **RXMQCONN** Client/Server interface is being used, then the MQ Client/Server Communications must be configured and active for the Server Queue Manager to be contacted. A Client/Server *MQCONN* takes longer to complete than a local contact.

RXMQDISC

Description

This function disconnects (*MQDISC*) from the currently connected Queue Manager. As an extension to the function, the interface will issue a *MQCLOSE(...,MQCO_NONE)* for any still open queue accessed via the interface (this is to cope with Rexx Tracing, and so give the user a simple way of 'gracefully' exiting when in test mode).

Parameters

None.

Call

```
rcc = RXMQDISC()
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQDISC Bad number of parms

Explanation You cannot specify any parameters to this call.

-95 0 0 RXMQDISC Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under *CSingleLock::Unlock*).

-96 0 0 RXMQDISC Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under *CSingleLock::Lock*).

-98 0 0 RXMQDISC Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQDISC UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
rcc = RXMQDISC()
```

This call will disconnect from the currently accessed Queue Manager doing a *MQCLOSE(None)* on any Queues still open at this point.

If the **RXMQDISC** Client/Server interface is being used, then the MQ Client/Server Communications will stop after this call has successfully completed.

RXMQOPEN

Description

This verb provides access to a MQ Object via a *MQOPEN* call. Upto 100 Objects can be accessed via this interface in any one process (ie: 100 spread throughout all the threads owned by the process). Although one will normally be accessing a Queue, any of the allowed MQ objects can be accessed.

Parameters

1. The name of a Stem variable (including the dot) specifying the Object Descriptor for the MQ Object to access. This is an input only field. The format of this Stem variable is described in “The Object Descriptor” on page 19.

If the name given does not end in a dot, then the data is taken to be the name of a Queue (or Model Queue) to access. This short cut removes the requirement to fully format up a stem variable for 'normal' Queue access; but note that you supply the name of the Queue, not the name of the variable containing the name of the Queue.

2. The *MQOPEN* Options (as described in the APR). This is an input only field, and should resolve into a number (not the name of a field containing the Options).

3. The name of a variable to contain a handle for the MQ Object being accessed. This is an output field, and should be the name of the field to receive the handle.

The handle returned is **not** the handle returned by the underlying *MQOPEN* verb; this latter value is not accessible outside of the interface. This handle must be quoted on all subsequent accesses to the Object.

4. The name of a Stem variable (including the dot) into which is placed the Object Descriptor returned by the underlying *MQOPEN* verb. This is an output only field.

The format of this Stem variable is described in “The Object Descriptor” on page 19; **ZLIST** processing is provided.

Call

```
rcc = RXMQOPEN( 'Stem.Input.OD.', OpenOptions, 'VarHandle', 'Stem.Output.OD.' )
```

or

```
rcc = RXMQOPEN( QueueName , OpenOptions, 'VarHandle', 'Stem.Output.OD.' )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQOPEN Bad number of parms

Explanation You **must** specify four parameters to the **RXMQOPEN** call.

-2 0 0 RXMQOPEN Null Input OD/Qname

Explanation A null has been supplied for the first parameter, the name of a stem variable for an input Open Descriptor or the name of a Queue to access.

-3 0 0 RXMQOPEN Zero length Input OD/Qname

Explanation No value has been keyed for the first parameter, the name of a stem variable for an input Open Descriptor or the name of a Queue to access.

-4 0 0 RXMQOPEN Null options

Explanation A null has been supplied for the second parameter, a number representing the Open Options. To specify No Options, supply a 0.

-5 0 0 RXMQOPEN Zero length options

Explanation No value has been keyed for the second parameter, a number representing the Open Options. To specify No Options, supply a 0.

-6 0 0 RXMQOPEN Null handle name

Explanation A null has been supplied for the third parameter, the name of a variable which will be set to

the obtained handle for the accessed MQ Object.

-7 0 0 RXMQOPEN Zero length handle name

Explanation No value has been keyed for the third parameter, the name of a variable which will be set to the obtained handle for the accessed MQ Object.

-8 0 0 RXMQOPEN Null Output OD

Explanation A null has been supplied for the forth parameter, the name of a stem variable which will be set to the obtained Object Descriptor for the accessed MQ Object.

-9 0 0 RXMQOPEN Zero length Output OD

Explanation No value has been keyed for the forth parameter, the name of a stem variable which will be set to the obtained Object Descriptor for the accessed MQ Object.

-10 0 0 RXMQOPEN No available Q objects

Explanation The limit of MQ Objects supported by this interface has been reached.

-95 0 0 RXMQOPEN Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

-96 0 0 RXMQOPEN Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

-98 0 0 RXMQOPEN Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQOPEN UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
opts = MQOO_INQUIRE      + MQOO_INPUT_SHARED      ,  
      + MQOO_BROWSE       + MQOO_SAVE_ALL_CONTEXT ,  
      + MQOO_FAIL_IF QUIESCING
```

```
rcc = RXMQOPEN( N1, opts, 'hn1', 'od.' )
```

This call opens the Queue **N1** for a Browse access, and permits the inquiry of the queue's attributes. If the open succeeds, then the variable **hn1** is set to the handle for subsequent access to **N1**, and the stem variable **od.** is set to the contents of the Object Descriptor for **N1** (eg: **od.ON = 'N1'**).

```
iod.OT = MQOT_Q  
iod.ON = 'N1'
```

```
rcc = RXMQOPEN( 'iod.', MQOO_BROWSE+MQOO_INQUIRE, 'hn1', 'ood.' )
```

This example shows how the Queue **N1** would be accessed if the full Object Descriptor method is used to specify the MQ Object to be accessed.

RXMQCLOS

Description

This verb stops access to a MQ Object, using the underlying *MQCLOSE* verb.

Parameters

1. The Handle for the object obtained from a prior **RXMQOPEN** call. This is an input parameter. After this call completes, the handle is no longer valid for use.
2. The Close options. This is an input parameter representing the type of *MQCLOSE* operation to be pershaped.

Call

```
rcc = RXMQCLOS( handle, CloseOptions )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQCLOS Bad number of parms

Explanation You **must** specify two parameters to the **RXMQCLOS** call.

-2 0 0 RXMQCLOS Null handle

Explanation A null has been supplied for the first parameter, the handle representing the MQ object.

-3 0 0 RXMQCLOS Zero length handle

Explanation No value has been keyed for the first parameter, the handle representing the MQ object.

-4 0 0 RXMQCLOS Null options

Explanation A null has been supplied for the second parameter, a number representing the Close Options. To specify No Options, supply a 0.

-5 0 0 RXMQCLOS Zero length options

Explanation No value has been keyed for the second parameter, a number representing the Close Options. To specify No Options, supply a 0.

-6 0 0 RXMQCLOS Handle out of range

Explanation The value of the handle supplied is not in the known range for a handle within the interface.

-7 0 0 RXMQCLOS Invalid handle

Explanation The handle specified does not relate to an accessed MQ Object.

-95 0 0 RXMQCLOS Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

-96 0 0 RXMQCLOS Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

-97 0 0 RXMQCLOS Handle not owned by Current Thread

Explanation The object referred to by the given handle was not accessed by the current Thread (ie: it was **RXMQOPEN**ed by another thread).

-98 0 0 RXMQCLOS Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQCLOS UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
rcc = RXMQCLOSE( hn1, MQCO_NONE )
```

This call closes the object referred to by the handle specified in the **hn1** variable, with no special closing actions being requested.

RXMQINQ

Description

This call will inquire upon a **single** attribute of a MQ object. This is a difference between this interface and the function of the underlying *MQINQ* verb.

The relevant data is returned in character format, so numeric attributes need not be converted for Rexx usage. The requested attribute is specified via **MQIA_**, **MQCA_**... variables.

Parameters

1. The **handle** for the object obtained from a prior **RXMQOPEN** call, whereat the object was opened for Inquiry. This is an input parameter.
2. The **Attribute** number to be inquired upon (setting starting with **MQIA**, **MQCA_**...). This is an input parameter.
3. The name of a variable into which will be returned the current setting of the desired attribute. Numeric attributes (like Maximum Message Size) are presented as decimal strings (so '17' might be returned rather than '11'x). This is an output parameter.

Call

```
rcc = RXMQINQ ( handle, Attribute, VarAttributeValue )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQINQ Bad number of parms

Explanation You **must** specify three parameters to the **RXMQINQ** call.

-2 0 0 RXMQINQ Null handle

Explanation A null has been supplied for the first parameter, the handle representing the MQ object.

-3 0 0 RXMQINQ Zero data handle

Explanation No value has been keyed for the first parameter, the handle representing the MQ object.

-4 0 0 RXMQINQ Null data input attr

Explanation A null has been supplied for the second parameter, a number representing representing the attribute of the MQ object to be obtained.

-5 0 0 RXMQINQ Zero data input attr

Explanation No value has been keyed for the second parameter, a number representing the attribute of the MQ object to be obtained.

-6 0 0 RXMQINQ Null output attr

Explanation A null has been supplied for the third parameter, the name of a variable to receive the value of the requested attribute.

-7 0 0 RXMQINQ Zero length output attr

Explanation No value has been keyed for the third parameter, the name of a variable to receive the value of the requested attribute.

-8 0 0 RXMQINQ No attribute supplied

Explanation No value was supplied for the attribute under consideration.

-9 0 0 RXMQINQ Attribute out of valid range

Explanation The value of the attribute under consideration was outside of the ranges defined for Integer and Character attributes.

-10 0 0 RXMQINQ Handle out of range

Explanation The value of the handle supplied is not in the known range for a handle within the interface.

-11 0 0 RXMQINQ Invalid handle

Explanation The handle specified does not relate to an accessed MQ Object.

-95 0 0 RXMQINQ Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

-96 0 0 RXMQINQ Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

-97 0 0 RXMQINQ Handle not owned by Current Thread

Explanation The object referred to by the given handle was not accessed by the current Thread (ie: it was RXMQOPENed by another thread).

-98 0 0 RXMQINQ Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQINQ UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
rcc = RXMQINQ( hn1, MQIA_MAX_MSG_LENGTH, 'maxmsg' )  
/* maxmsg = 3109856 */
```

This call obtains the current Maximum Message Length attribute for the queue referenced by the handle contained in **hn1**. In this case, the **maxmsg** variable is set to 3109856, the value of the desired attribute.

RXMQSET

Description

This call will set a **given** attribute of a MQ object. This is a difference between this interface and the underlying *MQSET* verb, whereat many attributes can be manipulated in a single execution.

The relevant data is specified in character format, so numeric attributes need not be converted for interface usage. The attribute is specified via **MQIA_**, **MQCA_**... variables.

Parameters

1. The **handle** for the object obtained from a prior **RXMQOPEN** call, whereat the object was opened for Setting. This is an input parameter.
2. The **Attribute** Number to be set (starting with **MQIA_**, **MQCA_**...). This is an input parameter.
3. The value of the attribute which is to be set in the MQ Object. Numeric attributes (like Trigger Depth) are specified as a normal Rexx decimal string (so use '17' rather than '11'x). This is an input parameter.

Call

```
rcc = RXMQSET( handle, Attribute, AttributeSetting )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQSET Bad number of parms

Explanation You **must** specify three parameters to the **RXMQSET** call.

-2 0 0 RXMQSET Null handle

Explanation A null has been supplied for the first parameter, the handle representing the MQ object.

-3 0 0 RXMQSET Zero data handle

Explanation No value has been keyed for the first parameter, the handle representing the MQ object.

-4 0 0 RXMQSET Null data attribute

Explanation A null has been supplied for the second parameter, a number representing representing the attribute of the MQ object to be set.

-5 0 0 RXMQSET Zero data attribute

Explanation No value has been keyed for the second parameter, a number representing the attribute of the MQ object to be set.

-6 0 0 RXMQSET Null setting

Explanation A null has been supplied for the third parameter, the name of a variable to receive the value of the requested attribute.

-7 0 0 RXMQSET Zero length setting

Explanation No value has been keyed for the third parameter, the name of a variable to receive the value of the requested attribute.

-8 0 0 RXMQSET No attribute supplied

Explanation No value was supplied for the attribute under consideration.

-9 0 0 RXMQSET Attribute out of valid range

Explanation The value of the attribute under consideration was outside of the ranges defined for Integer and Character attributes.

-10 0 0 RXMQSET Handle out of range

Explanation The value of the handle supplied is not in the known range for a handle within the interface.

-11 0 0 RXMQSET Invalid handle

Explanation The handle specified does not relate to an accessed MQ Object.

-95 0 0 RXMQSET Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

-96 0 0 RXMQSET Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

-97 0 0 RXMQSET Handle not owned by Current Thread

Explanation The object referred to by the given handle was not accessed by the current Thread (ie: it was **RXMQOPENed** by another thread).

-98 0 0 RXMQSET Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQSET UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
rc = RXMQSET( hn1, MQIA_TRIGGER_DEPTH, 21)
```

This call sets the Trigger Depth for the Queue specified by **hn1** (which must have been opened with Set access) to 21 messages.

RXMQSUB

Description

This call creates a subscription to a topic using the MQSUB verb. The function returns the object handle used to receive publications in the second parameter. It can also return the subscription handle in an optional fourth parameter.

The three parameter form is kept unchanged for compatibility with earlier Rexx Execs. When the fourth parameter is not supplied, RXMQSUB behaves as it did before.

Parameters

1. The name of a Stem variable containing the Subscription Descriptor. This is an input/output parameter.
2. The name of the Rexx variable which will receive the object handle returned by MQSUB. This handle is used by RXMQGET to receive publications.
3. The name of a Stem variable into which the updated Subscription Descriptor will be placed.
4. Optionally, the name of the Rexx variable which will receive the subscription handle returned by MQSUB.

Call

```
rcc = RXMQSUB( 'Stem.SD.' , handle , 'Stem.OSD.' )
```

or

```
rcc = RXMQSUB( 'Stem.SD.' , handle , 'Stem.OSD.' , subhandle )
```

Additional Interface Return Codes and Messages

```
-1 0 0 RXMQSUB Bad number of parms
```

Explanation You must specify three or four parameters to RXMQSUB.

```
-6 0 0 RXMQSUB Null subscription handle var
```

Explanation A null has been supplied for the optional fourth parameter, the name of a Rexx variable into which the subscription handle is to be returned.

```
-7 0 0 RXMQSUB Zero subscription handle var
```

Explanation No value has been keyed for the optional fourth parameter, the name of a Rexx variable into which the subscription handle is to be returned.

```
-10 0 0 RXMQSUB No available Q objects
```

Explanation The limit of MQ Objects supported by this interface has been reached. When the optional subscription handle is requested, two internal handle slots are required.

Example

```
isd.VER = MQSD_VERSION_1
isd.OPT = MQSO_CREATE + MQSO_NON_DURABLE + MQSO_FAIL_IF_QUIESCING
isd.OS.0 = 16
isd.OS.1 = 'sport/football/#'
rcc = RXMQSUB('isd.', hpub, 'osd.', hsub)
say 'publication handle=' hpub
say 'subscription handle=' hsub
```

The handle hpub is used by RXMQGET to get publications. The handle hsub is the subscription handle returned by MQSUB. Both are MA95 handles and can be closed by RXMQCLOS when no longer required.

RXMQCMIT

Description

This verb will issue a *MQCMIT* verb. It syncpoints the current Queue Manager accesses. Note that this operation affects **all** the currently accessed queues which have extant operations within Unit of Work control **within** the current thread (ie: it does not effect other threads within the process).

Parameters

None

Call

```
rcc = RXMQCMIT()
```

Additional Interface Return Codes and Messages

```
-1 0 0 RXMQCMIT Bad number of parms
```

Explanation You cannot specify any parameters to this call.

```
-98 0 0 RXMQCMIT Not Connected to a QM
```

Explanation The current thread is not connected to a Queue Manager

```
-99 0 0 RXMQCMIT UNKNOWN FAILURE
```

Explanation Some unknown error has occurred!

Example

```
rcc = RXMQCMIT()
```

The accesses to all currently accessed queues (that are within Unit of Work control) are committed. Accesses outside of UOW control are unaffected by this call.

RXMQBACK

Description

This verb will issue a *MQBACK* verb. It rolls back the current Queue Manager accesses. Note that this operation affects **all** the currently accessed queues which have extant operations within Unit of Work control **within** the current thread (ie: it does not effect other threads within the process).

Parameters

None

Call

```
rcc = RXMQBACK()
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQBACK Bad number of parms

Explanation You cannot specify any parameters to this call.

-98 0 0 RXMQBACK Not Connected to a QM

Explanation The current thread is not connected to a Queue Manager

-99 0 0 RXMQBACK UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
rcc = RXMQBACK()
```

The accesses to all currently accessed queues (that are within Unit of Work control) are rolledback. Accesses outside of UOW control are unaffected by this call.

RXMQGET

Description

This call will obtain a message from a Queue, using the underlying *MQGET* verb. All the abilities of this verb are supported by this interface.

A quick way of issuing Browse calls is provided by “RXMQBRWS” on page 56.

Parameters

1. The Handle for the Queue obtained from a prior **RXMQOPEN** call, whereat the Queue was opened for Input (or Browse) access. This is an Input parameter.
2. The name of a Rexx Stem variable (including the dot) into which the obtained message will be placed. This is an input/output parameter. Upon the call, Component 0 must contain the Maximum length of the message to be received. After the call, Component 0 will contain the length of the message received (or would have been received if the initial setting was 0) and Component 1 will contain the obtained message (if any). See “Message Lengths” on page 11 for a warning about truncation.
3. The name of a Stem variable (including the dot) containing the Input Message Descriptor describing the Message to be obtained from the Queue. This is an input parameter.
4. The name of a Stem variable (including the dot) into which will be returned a Message Descriptor describing the message obtained by the call. This is an output parameter, so **ZLIST** processing is provided.
5. The name of a Stem variable (including the dot) containing the Get Message Options for the operation. This is an input parameter.
6. The name of a Stem variable (including the dot) into which will be placed the updated Get Message Options resulting from the call. This is an output parameter, so **ZLIST** processing is provided.

Call

```
rcc = RXMQGET( handle, 'Stem.Message.' , 'Stem.Input.MD.' , 'Stem.Output.MD.' ,  
               'Stem.Input.GMO.' , 'Stem.Output.GMO.' )
```


Additional Interface Return Codes and Messages

-1 0 0 RXMQGET Bad number of parms

Explanation You must specify six parameters to the **RXMQGET** call.

-2 0 0 RXMQGET Null handle

Explanation A null has been supplied for the first parameter, the handle representing the MQ object.

-3 0 0 RXMQGET Zero data handle

Explanation No value has been keyed for the first parameter, the handle representing the MQ object.

-4 0 0 RXMQGET Null data stem var

Explanation A null has been supplied for the second parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-5 0 0 RXMQGET Zero data stem var

Explanation No value has been keyed for the second parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-6 0 0 RXMQGET Null Input MsgDesc

Explanation A null has been supplied for the third parameter, the name of a Stem Variable containing the Input Message Variable for the operation.

-7 0 0 RXMQGET Zero length Input MsgDesc

Explanation No value has been keyed for the third parameter, the name of a Stem Variable containing the Input Message Variable for the operation.

-8 0 0 RXMQGET Null Output MsgDesc

Explanation A null has been supplied for the forth parameter, the name of a Stem Variable into which will be placed the resulting Message Descriptor from the operation.

-9 0 0 RXMQGET Zero length Output MsgDesc

Explanation No value has been keyed for the forth parameter, the name of a Stem Variable into which will be placed the resulting Message Descriptor from the operation.

-10 0 0 RXMNGET Null input GMO

Explanation A null has been supplied for the fifth parameter, the name of a Stem Variable containing the Get Message Options for the operation.

-11 0 0 RXMNGET Zero length input GMO

Explanation No value has been keyed for the fifth parameter, the name of a Stem Variable containing the Get Message Options for the operation.

-12 0 0 RXMNGET Null output GMO

Explanation A null has been supplied for the sixth parameter, the name of a Stem Variable into which will be placed the resulting Get Message Options from the operation.

-13 0 0 RXMQGET Zero length output GMO

Explanation No value has been keyed for the sixth parameter, the name of a Stem Variable into which will be placed the resulting Get Message Options from the operation.

-14 0 0 RXMQGET Handle out of range

Explanation The value of the handle supplied is not in the known range for a handle within the interface.

-15 0 0 RXMQGET Invalid handle

Explanation The handle specified does not relate to an accessed MQ Object.

-16 0 0 RXMQGET malloc failure, RC(<errno>)

Explanation An attempt to acquire storage for the number of bytes specified in the Message.0 (2nd parameter) variable failed. The return code is errno code from the malloc call.

-17 0 0 RXMQGET Zero length input data buffer

Explanation The Message.0 (2nd parameter) was zero, indicating no message to process.

-97 0 0 RXMQGET Handle not owned by Current Thread

Explanation The object referred to by the given handle was not accessed by the current Thread (ie: it was **RXMQOPENED** by another thread).

-98 0 0 RXMQGET Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQGET UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
message.0 = 100
message.1 = ''

igmo.opt = MQGMO_WAIT + MQGMO_SYNCPOINT + MQGMO_FAIL_IF QUIESCING
igmo.wait = 1

imd.MSGID = ''
imd.CID = ''

rcc = RXMQGET( hn1, 'message.', 'imd.', 'omd.', 'igmo.', 'ogmo.' )

/* on return, say...

message.0 = 13
message.1 = 'WMQ rules OK!'

omd.msg = MQMT_DATAGRAM
omd.PER = MQPER_PERSISTENT
...
ogmo.rqn = 'N1'

*/
```

This call destructively obtains the next message from the Queue. The message can be upto 100 bytes long - a bigger message is not obtained (as the options do not specify **MQGMO_ACCEPT_TRUNCATED_MSG**). The obtained message (which will not physically be removed from the Queue until a Syncpoint is issued, as it is obtained under Unit Of Work control) is 13 bytes long, and is persistent.

RXMQPUT

Description

This call will place a message into a Queue, using the underlying *MQPUT* verb. All the abilities of this verb are supported by this interface.

Parameters

1. The Handle for the Queue obtained from a prior **RXMQOPEN** call, whereat the Queue was opened for Output access. This is an Input parameter.
2. The name of a Rexx Stem variable (including the dot) containing the message to be placed on the Queue. This is an input parameter. Component 0 must contain the length of Component 1, which is the message to be put into the Queue.
3. The name of a Stem variable (including the dot) containing the Input Message Descriptor describing the Message to be placed on the Queue. This is an input parameter.
4. The name of a Stem variable (including the dot) into which will be returned a Message Descriptor describing the message placed by the call. This is an output parameter, so **ZLIST** processing is provided.
5. The name of a Stem variable (including the dot) containing the Put Message Options for the operation. This is an input parameter.
6. The name of a Stem variable (including the dot) into which will be placed the updated Put Message Options resulting from the call. This is an output parameter, so **ZLIST** processing is provided.

Call

```
rcc = RXMQPUT( handle, 'Stem.Message.', 'Stem.Input.MD.', 'Stem.Output.MD. ',  
                'Stem.Input.PMO.', 'Stem.Output.PMO.' )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQPUT Bad number of parms

Explanation You **must** specify six parameters to the **RXMQPUT** call.

-2 0 0 RXMQPUT Null handle

Explanation A null has been supplied for the first parameter, the handle representing the MQ object.

-3 0 0 RXMQPUT Zero data handle

Explanation No value has been keyed for the first parameter, the handle representing the MQ object.

-4 0 0 RXMQPUT Null data stem var

Explanation A null has been supplied for the second parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-5 0 0 RXMQPUT Zero data stem var

Explanation No value has been keyed for the second parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-6 0 0 RXMQPUT Null Input MsgDesc

Explanation A null has been supplied for the third parameter, the name of a Stem Variable containing the Input Message Variable for the operation.

-7 0 0 RXMQPUT Zero length Input MsgDesc

Explanation No value has been keyed for the third parameter, the name of a Stem Variable containing the Input Message Variable for the operation.

-8 0 0 RXMQPUT Null Output MsgDesc

Explanation A null has been supplied for the forth parameter, the name of a Stem Variable into which will be placed the resulting Message Descriptor from the operation.

-9 0 0 RXMQPUT Zero length Output MsgDesc

Explanation No value has been keyed for the forth parameter, the name of a Stem Variable into which will be placed the resulting Message Descriptor from the operation.

-10 0 0 RXMQPUT Null input PMO

Explanation A null has been supplied for the fifth parameter, the name of a Stem Variable containing the Put Message Options for the operation.

-11 0 0 RXMQPUT Zero length input PMO

Explanation No value has been keyed for the fifth parameter, the name of a Stem Variable containing the Put Message Options for the operation.

-12 0 0 RXMQPUT Null output PMO

Explanation A null has been supplied for the sixth parameter, the name of a Stem Variable into which will be placed the resulting Put Message Options from the operation.

-13 0 0 RXMQPUT Zero length output PMO

Explanation No value has been keyed for the sixth parameter, the name of a Stem Variable into which will be placed the resulting Put Message Options from the operation.

-14 0 0 RXMQPUT Handle out of range

Explanation The value of the handle supplied is not in the known range for a handle within the interface.

-15 0 0 RXMQPUT Invalid handle

Explanation The handle specified does not relate to an accessed MQ Object.

-16 0 0 RXMQPUT malloc failure, RC(<errno>)

Explanation An attempt to acquire storage for the number of bytes specified in the data.0 (2nd parameter) variable failed. The return code is errno code from the malloc call.

-17 0 0 RXMQPUT Zero length input data buffer

Explanation The data.0 (2nd parameter), the Stem variable containing the length of message to be sent, was zero or not numeric.

-18 0 0 RXMQPUT Data length is not equal to specified value

Explanation The data.0 (2nd parameter), the Stem variable containing the length of message to be sent, is not equal to data.1 actual message length.

-19 0 0 RXMQPUT Context handle out of range

Explanation The value of the handle supplied in **PMO.CON** is not in the known range for a handle within the interface.

-20 0 0 RXMQPUT Invalid Context handle

Explanation The handle specified in **PMO.CON** does not relate to an accessed MQ Object.

-96 0 0 RXMQPUT Context handle not owned by current thread

Explanation The object referred to by the given **PMO.CON** handle was not accessed by the current Thread (ie: it was **RXMQOPENed** by another thread).

-97 0 0 RXMQPUT Handle not owned by current thread

Explanation The object referred to by the given handle was not accessed by the current Thread (ie: it was **RXMQOPENed** by another thread).

-98 0 0 RXMQPUT Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQPUT UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
message.1 = 'WMQ''s wonderful interface!'
message.0 = LENGTH(message.1)

ipmo.opt   =      MQPMO_NO_SYNCPOINT      + MQPMO_NO_CONTEXT  ,
              + MQPMO_FAIL_IF QUIESCING

imd.MSG     = MQMT_DATAGRAM
imd.per     = MQPER_NOT_PERSISTENT

rcc = RXMQPUT( hn1, 'message.', 'imd.', 'omd.', 'ipmo.', 'opmo.' )

/* on return, say.....

omd.PD     = 20080831
...

opmo.rqn = 'N1'

*/
```

This call places the given non-persistent message on the Queue outside of a Unit of Work.

RXMQPUT1

Description

This call will open the Queue, place one message into it and close the queue using the underlying *MQPUT1* verb. All the abilities of this verb are supported by this interface.

Parameters

1. The name of a Stem variable (including the dot) specifying the Object Descriptor for the MQ Object to access. This is an input only field.

The format of this Stem variable is described in “The Object Descriptor” on page 19.

If the name given does not end in a dot, then the data is taken to be the name of a Queue (or Model Queue) to access. This short cut removes the requirement to fully format up a stem variable for 'normal' Queue access; but note that you supply the name of the Queue, not the name of the variable containing the name of the Queue.

2. The name of a Stem variable (including the dot) into which is placed the Object Descriptor returned by the underlying *MQOPEN* verb. This is an output only field.

The format of this Stem variable is described in “The Object Descriptor” on page 19; **ZLIST** processing is provided.

3. The name of a Rexx Stem variable (including the dot) containing the message to be placed on the Queue. This is an input parameter. Component 0 must contain the length of Component 1, which is the message to be put into the Queue.

4. The name of a Stem variable (including the dot) containing the Input Message Descriptor describing the Message to be placed on the Queue. This is an input parameter.

5. The name of a Stem variable (including the dot) into which will be returned a Message Descriptor describing the message placed by the call. This is an output parameter, so **ZLIST** processing is provided.

6. The name of a Stem variable (including the dot) containing the Put Message Options for the operation. This is an input parameter.

7. The name of a Stem variable (including the dot) into which will be placed the updated Put Message Options resulting from the call. This is an output parameter, so **ZLIST** processing is provided.

Call

```
rcc = RXMQPUT1( 'Stem.Input.OD.', 'Stem.Output.OD.', 'Stem.Message.',  
               'Stem.Input.MD.', 'Stem.Output.MD.', 'Stem.Input.PMO.',  
               'Stem.Output.PMO.' )
```

or

```
rcc = RXMQPUT1( Queue Name, 'Stem.Output.OD.', 'Stem.Message',  
               'Stem.Input.MD.', 'Stem.Output.MD.', 'Stem.Input.PMO', 'Stem.Output.PMO' )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQPUT1 Bad number of parms

Explanation You must specify seven parameters to the **RXMQPUT1** call.

-2 0 0 RXMQPUT1 Null Input OD/Qname

Explanation A null has been supplied for the first parameter, the name of a stem variable for an input Object Descriptor or the name of a Queue to access.

-3 0 0 RXMQPUT1 Zero length Input OD/Qname

Explanation No value has been keyed for the first parameter, the name of a stem variable for an input Object Descriptor or the name of a Queue to access.

-4 0 0 RXMQPUT1 Null Output OD

Explanation A null has been supplied for the second parameter, the name of a stem variable which will be set to the obtained Object Descriptor for the accessed MQ Object.

-5 0 0 RXMQPUT1 Zero length Output OD

Explanation No value has been keyed for the second parameter, the name of a stem variable which will be set to the obtained Object Descriptor for the accessed MQ Object.

-6 0 0 RXMQPUT1 Null data stem var

Explanation A null has been supplied for the third parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-7 0 0 RXMQPUT1 Zero data stem var

Explanation No value has been keyed for the third parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-8 0 0 RXMQPUT1 Null Input MsgDesc

Explanation A null has been supplied for the fourth parameter, the name of a Stem Variable containing the Input Message Variable for the operation.

-9 0 0 RXMQPUT1 Zero length Input MsgDesc

Explanation No value has been keyed for the fourth parameter, the name of a Stem Variable containing the Input Message Variable for the operation.

-10 0 0 RXMQPUT1 Null Output MsgDesc

Explanation A null has been supplied for the fifth parameter, the name of a Stem Variable into which will be placed the resulting Message Descriptor from the operation.

-11 0 0 RXMQPUT1 Zero length Output MsgDesc

Explanation No value has been keyed for the fifth parameter, the name of a Stem Variable into which will be placed the resulting Message Descriptor from the operation.

-12 0 0 RXMQPUT1 Null input PMO

Explanation A null has been supplied for the sixth parameter, the name of a Stem Variable containing the Put Message Options for the operation.

-13 0 0 RXMQPUT1 Zero length input PMO

Explanation No value has been keyed for the sixth parameter, the name of a Stem Variable containing the Put Message Options for the operation.

-14 0 0 RXMQPUT1 Null output PMO

Explanation A null has been supplied for the seventh parameter, the name of a Stem Variable into which will be placed the resulting Put Message Options from the operation.

-15 0 0 RXMQPUT1 Zero length output PMO

Explanation No value has been keyed for the seventh parameter, the name of a Stem Variable into which will be placed the resulting Put Message Options from the operation.

-16 0 0 RXMQPUT1 No available Q objects

Explanation The limit of MQ Objects supported by this interface has been reached.

-17 0 0 RXMQPUT1 malloc failure, RC(<errno>)

Explanation An attempt to acquire storage for the number of bytes specified in the data.0 (3rd parameter) variable failed. The return code is errno code from the malloc call.

-18 0 0 RXMQPUT1 Zero length input data buffer

Explanation The data.0 (3rd parameter), the Stem Variable containing the length of message to be sent, was zero or not numeric.

-19 0 0 RXMQPUT1 Data length is not equal to specified value

Explanation The data.0 (3rd parameter), the Stem variable containing the length of message to be sent, is not equal to data.1 actual message length.

-20 0 0 RXMQPUT1 Context handle out of range

Explanation The value of the handle supplied in **PMO.CON** is not in the known range for a handle within the interface.

-21 0 0 RXMQPUT1 Invalid Context handle

Explanation The handle specified in **PMO.CON** does not relate to an accessed MQ Object.

-95 0 0 RXMQPUT1 Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

-96 0 0 RXMQPUT1 Mutex <n> Acquisition failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

-97 0 0 RXMQPUT1 Context handle not owned by current thread

Explanation The object referred to by the given **PMO.CON** handle was not accessed by the current Thread (ie: it was **RXMQOPENed** by another thread).

-98 0 0 RXMQPUT1 Not Connected to a QM

Explanation The current thread was not connected to a Queue Manager

-99 0 0 RXMQPUT1 UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Example

```
message.1 = 'WMQ''s wonderful interface!'
message.0 = LENGTH(message.1)

ipmo.opt   =      MQPMO_NO_SYNCPOINT   + MQPMO_NO_CONTEXT   ,
             + MQPMO_FAIL_IF QUIESCING

imd.MSG     = MQMT_DATAGRAM
imd.per     = MQPER_NOT_PERSISTENT

rcc = RXMQPUT1( 'iod.', 'ood.', 'message.', 'imd.', 'omd.', 'ipmo.', 'opmo.' )

/* on return, say.....

omd.PD     = 20080831
...

opmo.rqn   = 'N1'

*/
```

This call places the given non-persistent message on the Queue outside of a Unit of Work.

```
message.1 = 'WMQ''s wonderful interface!'
message.0 = LENGTH(message.1)

ipmo.opt   =      MQPMO_NO_SYNCPOINT   + MQPMO_NO_CONTEXT   ,
             + MQPMO_FAIL_IF QUIESCING

imd.MSG     = MQMT_DATAGRAM
imd.per     = MQPER_NOT_PERSISTENT

rcc = RXMQPUT1( N1, 'ood.', 'message.', 'imd.', 'omd.', 'ipmo.', 'opmo.' )

/* on return, say...

omd.PD     = 20080831
...

opmo.rqn   = 'N1'

*/
```

This call places the given non-persistent message on the Queue outside of a Unit of Work using Queue Name as a parameter in **RXMQPUT1** function

RXMQC

Description

This function issues a **MQSC** command to a Queue Manager, returning the results. Each invocation checks whether the connection to Queue Manager exists. If the connection exists, the function uses it. If the connection does not exist, the function creates a connection to Queue Manager. If the connection is created by this function, it is released after the call.

As **RXMQC** operates within the Rexx environment, all the Rexx variables used are available for use within ISPF in the normal fashion. See on page 72 for an example.

Parameters

1. The controlling parameters (Input only)
either The name of the Queue Manager to connect to
or The name of a Stem. variable containing the name of the Queue Manager (**.QM**), the name of Command Queue (**.CQ**), the name of Model ReplyToQ (**.RQ**) and the Timeout (**.TO**)
2. The command to issue
3. The name of a Stem. variable into which the results of the **command** will be placed

Call

```
rcc = RXMQC (qmname, 'command', 'Stem.Reply.' )
```

or

```
rcc = RXMQC ('Stem.Params.', 'command', 'Stem.Reply.' )
```

Table 6. RXMQC Function Parameter Mappings

Name	Stem. Component	Input, Output or Both	Format
<i>Queue Manager</i>	.QM	I	MQCHAR48
<i>Command Queue</i>	.CQ	I	MQCHAR48
<i>Model ReplyToQ</i>	.RQ	I	MQCHAR48
<i>Timeout</i>	.TO	I	MQLONG

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- A timeout of 0 is an eternal wait

Additional Interface Return Codes and Messages

-1 0 0 RXMQC Bad number of parms

Explanation You **must** specify three parameters to **RXMQC**

-2 0 0 RXMQC Null parms

Explanation A null has been supplied for the first parameter, the name of a stem variable which will be set to the controlling parms for the operation.

-3 0 0 RXMQC Zero parms

Explanation No value has been keyed for the first parameter, the name of a stem variable which will be set to the controlling parms for the operation.

-4 0 0 RXMQC Null command var

Explanation A null has been supplied for the second parameter, the text of the command to issue.

-5 0 0 RXMQC Zero command var

Explanation No value has been keyed for the second parameter, the text of the command to issue.

-6 0 0 RXMQC Null response stem var

Explanation A null has been supplied for the third parameter, the name of a stem variable which will contain the results of the command.

-7 0 0 RXMQC Zero response stem var

Explanation No value has been keyed for the third parameter, the name of a stem variable which will contain the results of the command.

-8 0 0 RXMQC No command supplied

Explanation The length of the second parameter is zero, so no command was given.

-9 0 0 RXMQC Too big a command supplied

Explanation The length of the second parameter is bigger than 5000 bytes, which is the maximum supported Command length.

-10 0 0 RXMQC malloc failure, RC(<errno>)

Explanation An attempt to acquire storage for the response buffer. The return code is errno code from the malloc call.

-99 0 0 RXMQC UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Examples

```
res.0 = 0
rcc = RXMQC('QM1', 'DISPLAY QUEUE(*) TYPE(LOCAL)', 'res.' )
```

This call will contact the local Queue Manager called **QM1** and ask for a list of all the local Queues. The returned information is placed in the **res.n** stem variables, with **res.0** indicating the number of elements.

```
parm.QM = 'QM1'
parm.TO = 0
com = 'ping channel(CWTONBYTN)'
res.0 = 0
rcc = RXMQC('parm.', com, 'res.' )
```

This call will contact the remote Queue Manager called **QM1** and issue a ping on the given channel. The interface will wait until a reply to the ping is received.

RXMQBRWS

Description

This call is an extension to the MQ API as documented in the APR. This call will obtain the next message from a Queue via a **Browse** operation, using the underlying Browse function of the *MQGET* verb.

As this call is designed to be simple way to browse messages on a Queue, no Get Message Options or Message Descriptors are available. If access to these is required, then use the base “RXMQGET” on page 44.

Similarly, the position of the Browse cursor cannot be manipulated.

Parameters

1. The Handle for the Queue obtained from a prior **RXMQOPEN** call, whereat the Queue was opened for Browse access. This is an Input parameter.
2. The name of a REXX Stem variable (including the dot) into which the obtained message will be placed. This is an input/output parameter. Upon the call, Component **0** must contain the Maximum length of the message to be received. After the call, Component **0** will contain the length of the message received (or would have been received if the initial setting was 0) and Component **1** will contain the obtained message (if any). See “Message Lengths” on page 11 for a warning about truncation.

Call

```
rcc = RXMQBRWS( handle, 'Stem.Message.' )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQBRWS Bad number of parms

Explanation You **must** specify two parameters to the **RXMQBRWS** call.

-2 0 0 RXMQBRWS Null handle

Explanation A null has been supplied for the first parameter, the handle representing the MQ object.

-3 0 0 RXMQBRWS Zero data handle

Explanation No value has been keyed for the first parameter, the handle representing the MQ object.

-4 0 0 RXMQBRWS Null data stem var

Explanation A null has been supplied for the second parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-5 0 0 RXMQBRWS Zero data stem var

Explanation No value has been keyed for the second parameter, the name of a Stem Variable containing the maximum length of message to be obtained.

-6 0 0 RXMQBRWS Handle out of range

Explanation The value of the handle supplied is not in the known range for a handle within the interface.

-7 0 0 RXMQBRWS Invalid handle

Explanation The handle specified does not relate to an accessed MQ Object.

-8 0 0 RXMQBRWS malloc failure, RC(<errno>)

Explanation An attempt to acquire Storage for the number of bytes specified in the Message.0 (2nd parameter) variable failed. The return code is errno code that for the malloc call, and will usually result if the message.0 value is not numeric.

-9 0 0 RXMQBRWS Zero length input data buffer

Explanation The Message.0 (2nd parameter) was zero, indicating no message to process

-95 0 0 RXMQBRWS Mutex <n> Release failed rc <n>

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to release the lock for the given Reason (documented under CSingleLock::Unlock).

```
-96 0 0 RXMQBRWS Mutex <n> Acquisition failed rc <n>
```

Explanation The anonymous private Mutex used internally to single-thread object-based logic has failed to acquire for the given Reason (documented under CSingleLock::Lock).

```
-97 0 0 RXMQBRWS Handle not owned by Current Thread
```

Explanation The object referred to by the given handle was not accessed by the current Thread (ie: it was RXMQOPENed by another thread).

```
-98 0 0 RXMQBRWS Not Connected to a QM
```

Explanation The current thread was not connected to a Queue Manager .

```
-99 0 0 RXMQBRWS UNKNOWN FAILURE
```

Explanation Some unknown error has occurred!

Example

```
message.0 = 100
message.1 = ''

rcc = RXMQBRWS ( hn1, 'message.')

/* on return, say... message.0 = 2 ; message.1 = 'M1' */

message.0 = 100
message.1 = ''

rcc = RXMQBRWS ( hn1, 'message.')

/* on return, say... message.0 = 8 ; message. 1 = '>>>M2<<<' */
```

This example shows how a Browse is used to scan a Queue; observe that the message. Stem variable is cleared before each use.

RXMQHXT

Description

This call will take a message obtained from a Transmission Queue or a Dead Letter Queue (identified by the relevant header in the message) and split it up into its components.

This Header Extraction, therefore, permits the obtention of the 'real' message and an explanation of the control data associated with it.

The message to be split up is specified in the usual way as the name of the first stem. variable; with component **0** representing the length of the message which is supplied in component **1**. See "Message Lengths" on page 11 for a warning about truncated messages used with this function.

The Extracted data is placed in the second stem. variable (whose name is supplied); with component **0** representing the length of the 'actual' message which is placed in component **1**. The associated data is placed in other components, as shown in Table 7. Transmission Queue Message Header (MQXQH) Mappings on page 60 and in the Table 8. Dead Letter Queue Message Header (MQDLH) Mappings on page 61. It is **not** recommended that the input and output stem variables are the same (as this might lose information in the case of an error and additionally the component names clash with those generated as part of the Message descriptor).

In order to identify the type of header extracted, a component called **TYPE** is also created, taking the value of **XQH** or **DLH** (this is also provided in the **NAME** component).

Parameters

1. The name of a Rexx Stem variable (including the dot) containing a message to be splitup. This is an input parameter. Upon the call, Component **0** must contain the length of the message in Component **1**; the message must have been obtained from a Transmission Queue or a Dead Letter Queue. See "Message Lengths" on page 11 for a warning about truncation.
2. The name of a Rexx Stem variable (including the dot) into which the splitup message will be placed. This is an input/output parameter. After the call, Component **0** will contain the length of the 'actual' message and Component **1** will contain the 'actual' message (if any). Other components will be created (as documented in the Table 7. Transmission Queue Message Header (MQXQH) Mappings on page 60 and in the Table 8. Dead Letter Queue Message Header (MQDLH) Mappings on page 61) to return the extracted Header information from the input message. **ZLIST** processing is provided for this Stem variable.

Call

```
rcc = RXMQHXT( 'Stem.Message.', 'Stem.Splitup.' )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQHXT Bad number of parms

Explanation You **must** specify two parameters to the **RXMQHXT** call.

-2 0 0 RXMQHXT Null input stem var

Explanation A null has been supplied for the first parameter, the name of a Stem. variable representing the message to be splitup.

-3 0 0 RXMQHXT Zero input stem var

Explanation No value has been keyed for the first parameter, the name of a Stem. variable representing the message to be splitup.

-4 0 0 RXMQHXT Null output stem var

Explanation A null has been supplied for the second parameter, the name of a Stem. variable representing the splitup message.

-5 0 0 RXMQHXT Zero output stem var

Explanation No value has been keyed for the second parameter, the name of a Stem. variable

representing the splitup message.

-6 0 0 RXMQHXT No input data

Explanation The input Stem.0 was zero, indicating no message to process

-7 0 0 RXMQHXT Zero input data

Explanation The length of the input Stem.1 was zero, indicating no message to process

-8 0 0 RXMQHXT Cannot locate Header

Explanation The input Stem.0 was ≤ 3 , indicating no header in the message

-9 0 0 RXMQHXT Cannot find Header

Explanation The length of the input Stem.1 was ≤ 3 , indicating no header in the message

-10 0 0 RXMNHXT Unknown Header

Explanation The first 4 bytes of the input Stem.1 was not **DLH** or **XQH**, so the message did not come from a Dead Letter Queue or a Transmission Queue, and so cannot be splitup

-11 0 0 RXMNHXT Too short for a DLH (<n> bytes!)

Explanation Although the input Stem.1 looked like a DLH, Stem.0 was too small for the message to originate from a Dead Letter Queue, and so cannot be splitup

-12 0 0 RXMNHXT Too short for a XQH (<n> bytes!)

Explanation Although the input Stem.1 looked like a XQH, Stem.0 was too small for the message to originate from a Transmission Queue, and so cannot be splitup

-13 0 0 RXMHXT malloc failure, RC(<errno>)

Explanation An attempt to acquire Storage for the number of bytes specified in the input Stem.0 (1st parameter) variable failed. The return code is errno code from the malloc call.

-14 0 0 RXMQPUT1 Data length is not equal to specified value

Explanation The Stem.0 (1st parameter), the Stem variable containing the length of message to be splitup, is not equal to Stem.1 actual message length.

-99 0 0 RXMQHXT UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Extracted information

Transmission Queue Messages

Table 7. Transmission Queue Message Header (MQXQH) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
<i>Actual message length</i>	.0	O	MQLONG
<i>Actual message</i>	.1	O	MQCHAR
<i>RemoteQName</i>	.RQN	O	MQCHAR48
<i>RemoteQMgrName</i>	.RQM	O	MQCHAR48
<i>MsgDesc</i>	.xxx	O	MQMD1

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names
- See MsgDesc component names in the Table 3. Message Descriptor (MQMD) Mappings

• Dead Letter Queue Messages

Table 8. Dead Letter Queue Message Header (MQDLH) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
<i>Actual message length</i>	.0	O	MLONG
<i>Actual message</i>	.1	O	MQCHAR
<i>Reason</i>	.REA	O	MLONG
<i>DestinationQMgrName</i>	.DQM	O	MQCHAR48
<i>DestinationQName</i>	.DQN	O	MQCHAR48
<i>Encoding</i>	.ENC	O	MLONG
<i>CodedCharSetId</i>	.CCSI	O	MLONG
<i>Format</i>	.FORM	O	MQCHAR8
<i>PutApplType</i>	.PAT	O	MLONG
<i>PutApplName</i>	.PAN	O	MQCHAR28
<i>PutDate</i>	.PD	O	MQCHAR8
<i>PutTime</i>	.PT	O	MQCHAR8

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names

Example

```
/* A message has been obtained such that ... */  
  
message.0 = 438  
  
message.1 = <XQH>1234567890  
  
/* Clear the result variable */  
  
drop x.  
  
/* Split the message */  
  
rcc = RXMQHXT ( 'message.', 'x.' )  
  
/* on return, the following (and more) are set */  
  
say x.0 /* 10 */  
say x.1 /* 1234567890 */  
say x.RQM /* QM1 */  
say x.RQN /* CP1 */  
say x.PER /* 1 */  
say x.TYPE /* XQH */
```

This example shows how a message obtained from a Transmission Queue is splitup, showing information extracted from the **XQH** and the actual message being transmitted.

RXMQEVNT

Description

This call will take a message obtained from an Event Queue and split it up into its components. In general the default system queues called are

SYSTEM.ADMIN.QMGR.EVENT,
SYSTEM.ADMIN.CONFIG.EVENT,
SYSTEM.ADMIN.PERFM.EVENT and
SYSTEM.ADMIN.CHANNEL.EVENT

This Event Extraction, therefore, permits the detection of the event and an explanation of the control data associated with it.

The message to be split up is specified in the usual way as the name of a stem. variable; with component **0** representing the length of the message which is supplied in component **1**. See “Message Lengths” on page 11 for a warning about truncated messages used with this function. This message will have come from a prior **RXMQBRWS** or **RXMQGET** operation.

The Extracted data is placed in another stem. variable (whose name is supplied), with the various components containing information about the event. Each component name provided is equal to event attribute constant name, which are described in *WebSphere MQ Monitoring WebSphere MQ Version 6.0 SC34-6593*. For ease of reading and to save space constant name prefix (like **MQCA_** or **MQIA_**) is omitted. It is **not** recommended that the input and output stem variables are the same (as this might loose information in the case of an error and additionally the component names clash with those generated as part of the Message descriptor). Observe that some information is held in the event message's Message Descriptor (like Date and Time), so obtaining the message should be done via a Browse-type of **RXMQGET** rather than the **RXMQBRWS** call which does not return the Message Descriptor if this type of information is required.

In order to identify the type of event extracted, a component called **TYPE** is created and set to **EVENT**, and another called **NAME** which interprets the Event (it is presentd in the same way as described above for the component names).

Information about Events is discussed in *WebSphere MQ Monitoring WebSphere MQ Version 6.0 SC34-6593* book which you should use to interpret the expansion.

Warning

The PCF Documentation on events sometimes does not agree with what is actually recorded in the Event Message. Please take care in this arena, and treat deviations from the Documentation pragmatically (ie: raise an APAR, but process as this interface returns). A general usage should test each component to discover whether or not this information is returned. Alternatively, use **ZLIST** processing (as described in “ZLIST” on page 12). A returned component may be null (or have a zero length) if the Event Field is present without any data.

Parameters

- 1.The name of a Rexx Stem variable (including the dot) containing an event message to be splitup.
This is an input parameter. Upon the call, Component **0** must contain the length of the message in Component **1**; the message must have been obtained from an Event Queue. See “Message Lengths” on page 11 for a warning about truncation.
- 2.The name of a Rexx Stem variable (including the dot) into which the splitup message will be placed.
This is an input/output parameter. After the call, components will be created (as described above) to return the extracted event information from the input message. **ZLIST** processing is provided for this Stem variable.

Call

```
rcc = RXMQEVNT( 'Stem.Message.', 'Stem.Splitup.' )
```

Usage Notes

Bear in mind the following when using **RXMQEVNT**:

- A component is returned when the relevant parameter is present in the PCF Event Message. The returned data may consist of binary zeros, a null string ("") or all spaces if the contents do not exist (this is due to the way MQ builds the PCF Event message). Certain Rexx processors object to long strings of Binary zeros, so *you have been warned!*
- The PCF Event documentation may differ from the data actually returned. Always use **ZLIST** processing to see what is going on!
- The **EID**, **AEDI1**, **AEDI2** and **CED** fields are not returned as numbers, but rather in Hex. This will aid problem determination for these Channel error codes.
- There may be more than one **CED** field. In this case, **.CED.0** will contain the number of fields, with the data being in **.CED.n**
- The Date and Time of an Event is **not** held within the event, but in the Message Descriptor for the event.
- **.TYPE** is set to **'EVENT'** for all events.

Additional Interface Return Codes and Messages

-1 0 0 RXMQEVNT Bad number of parms

Explanation You **must** specify two parameters to the **RXMQEVNT** call.

-2 0 0 RXMQEVNT Null input stem var

Explanation A null has been supplied for the first parameter, the name of a Stem. variable representing the message to be splitup.

-3 0 0 RXMQEVNT Zero input stem var

Explanation No value has been keyed for the first parameter, the name of a Stem. variable representing the message to be splitup.

-4 0 0 RXMQEVNT Null output stem var

Explanation A null has been supplied for the second parameter, the name of a Stem. variable representing the splitup message.

-5 0 0 RXMQEVNT Zero output stem var

Explanation No value has been keyed for the second parameter, the name of a Stem. variable representing the splitup message.

-6 0 0 RXMQEVNT No input data

Explanation The input Stem.0 was zero, indicating no message to process

-7 0 0 RXMQEVNT Zero input data

Explanation The length of the input Stem.1 was zero, indicating no message to process

-8 0 0 RXMQEVNT Cannot locate Header

Explanation The input Stem.0 was ≤ 3 , indicating no header in the message

-9 0 0 RXMQEVNT Cannot find Header

Explanation The length of the input Stem.1 was ≤ 3 , indicating no header in the message

-10 0 0 RXMQEVNT Not an Event Header

Explanation The first 4 bytes of the input Stem.1 was not <MQCFH_EVENT>, so the message did not come from an Event Queue, and so cannot be splitup

-11 0 0 RXMQEVNT Too short for an Event (<n> bytes!)

Explanation Although the input Stem.1 looked like an Event Message, Stem.0 was too small for the message to originate from an Event Queue, and so cannot be splitup

-12 0 0 RXMQEVNT Unknown Event Category (<n>)

Explanation Although the input Stem.1 looked like an Event Message, the PCF Command field did not contain a recognisable event category, and so the message cannot be splitup

-13 0 0 RXMQEVNT Unknown Event Type (<n>)

Explanation Although the input Stem.1 looked like an Event Message, the PCF Reason field did not contain a recognisable event identifier, and so the message cannot be splitup

-14 0 0 RXMQEVNT No elements in the Event

Explanation Although the input Stem.1 looked like an Event Message, there were no PCF fields within the Message, and so the message cannot be splitup

-15 0 0 RXMQEVNT malloc failure, RC(<errno>)

Explanation An attempt to acquire Storage for the number of bytes specified in the input Stem.0 (1st parameter) variable failed. The return code is errno code that for the malloc call, and will usually result if the input Stem.0 value is not numeric or negative integer.

-16 0 0 RXMQPUT1 Data length is not equal to specified value

Explanation The Stem.0 (1st parameter), the Stem variable containing the length of message to be splitup, is not equal to Stem.1 actual message length.

```
-99 0 0 RXMQEVNT UNKNOWN FAILURE
```

Explanation Some unknown error has occurred!

Example

```
/* A message has been obtained such that ... */
message.0 = n
message.1 = <EVENT Header><Event Data>

/* Clear the result variable */
drop x.

/* Split the message */
rcc = RXMQEVNT ( 'message.', 'x.' )

/* on return, the following (and more) are set */

say x.TYPE           /* EVENT */
say x.NAME           /* CHANNEL_STOPPED */
say x.REA            /* 2283 */
say x.Q_MGR_NAME     /* QM1 */
say x.CHANNEL_NAME   /* SYSTEM.ADMIN.SVRCONN */
say x.REASON_QUALIFIER /* 8 */
say x.ERROR_IDENTIFIER /* 20009519 */
```

This example shows how a message obtained from **SYSTEM.ADMIN.CHANNEL.EVENT** is splitup, showing the information relating to the Channel Stop Event.

See Figure 2 on page 13 for an example using **ZLIST** processing to cope with the variable format component names.

```
/* Explurge an Event */

message.0 = n
message.1 = <EVENT Header><Event Data>
drop x.
rcc = RXMQEVNT ( 'message.', 'x.' )

/* Testing the returned information */

say x.TYPE      /* EVENT */
say x.NAME      /* INGET */
say x.REA       /* 2016 */

if ( x.qn <> 'X.QN' ) then say x.qn /* works - returned comp */
if ( x.BQN <> 'X.BQN' ) then say x.bqn /* fails - not in event */
```

This example shows how the components of an exploded Event message can be tested to fully extract all the returned information if **ZLIST** processing is not used.

ZLIST processing is also useful to cope with situations where an event String Field is defined, but set to all binary zeros. These can easily be changed into blanks (with space truncation) as follows:

```
message.0 = n
message.1 = <EVENT Header><Event Data>
drop x.
rcc = RXMQEVNT ( 'message.', 'x.' )

do i=1 to words(x.zlist)
  ts = word(x.zlist,i)
  x.ts = translate(x.ts, ' ', '00'x)
  x.ts = strip(x.ts, 'B')
end
```

RXMQTM

Description

This call will take a message obtained from an Initiation Queue (a Trigger Message) and split it up into its components. It will also parse the data passed to a started Rexx Exec (via a MQ Trigger Monitor).

This processing, therefore, permits the obtention of the control information associated with a Trigger: whether this is in the format of a MQ Message (garnered from an Initiation Queue) or passed as parameters to a Rexx Exec (as the Triggered Process).

The action of this function is controlled by the format of its first parameter, in particular whether or not it ends in a **dot**.

- If it **ends** in a dot, then **RXMQTM** is processing a message derived from an Initiation Queue.

The message to be processed is specified in the usual way as the name of a stem. variable; with component **0** representing the length of the message which is supplied in component **1**. See “Message Lengths” on page 11 for a warning about truncated messages used with this function.

This is called Message Mode.

- If it does **not end** in a dot, then **RXMQTM** is processing the parameter data passed via a Trigger Monitor to the Rexx Exec which is acting as a Triggered Process (ie: replaces the initial *parse arg* processing). It is the actual data, **not** a variable name that is supplied (ie: a substituted variable, not the variable name).

This is called Data Mode.

The Extracted data is placed in another stem. variable (whose name is supplied); with components representing the various sub-fields of the Trigger Message or Trigger parms.

Sub-fields which are all blanks (or start with a Binary Zero) are not extracted. **ZLIST** processing (see “ZLIST” on page 12) is provided so that the various extant components can be determined.

In Message Mode (a Trigger Message provided to **RXMQTM** in a Stem. variable) an additional component (not in **ZLIST**) called **PL** is provided which is the Parameter list for a process to be invoked by the reception of the Trigger Message in the Initiation Queue (if the current thread is connected to a Queue Manager, its name will be present in **.PL**). You should ensure that this component is not truncated in any way (as this will may well effect the activity of the process which uses it).

You can, therefore, use a Rexx Exec as the Triggered Process, extracting the supplied information using **RXMQTM** in Data Mode.

The use of Message Mode permits the coding of your own Trigger Monitor (recall the Trigger Messages only get placed in an Initiation Queue if the priorities are right, the process exists, and the Initiation Queue is Open for Getting) in Rexx (see Figure 3 on page 71), and Data Mode permits the use of Rexx Execs as Triggered Processes (see Figure 4 on page 71).

Parameters

1. This parameter takes one of these formats:

In Message Mode The name of a Rexx Stem variable (including the dot) containing a message to be splitup. This is an input parameter. Upon the call, Component **0** must contain the length of the message in Component **1**; the message must have been obtained from an Initiation Queue. See “Message Lengths” on page 11 for a warning about truncation.

In Data Mode The actual data (not a variable name) representing the **MQTMC2** structure which is used to initiate a Triggered Process.

2. The name of a Rexx Stem variable (including the dot) into which the extracted data will be placed. This is an input/output parameter. After the call, components will be created (as documented in Table 9. Trigger Component (MQTM/MQTMC2) Mappings on page 70) to return the extracted information. **ZLIST** processing is provided for this Stem variable. In the case of Message Mode, component **PL** will contain an area suitable for use by a Triggered Process as its parameters.

Call

Message Mode:

```
rcc = RXMQTM( 'Stem.Message.', 'Stem.Splitup.' )
```

Data Mode:

```
rcc = RXMQTM( MQTMC2_data , 'Stem.Splitup.' )
```

Additional Interface Return Codes and Messages

-1 0 0 RXMQTM Bad number of parms

Explanation You **must** specify two parameters to the **RXMQTM** call.

-2 0 0 RXMQTM Null input stem var

Explanation A null has been supplied for the first parameter, the name of a Stem. variable representing the message to be splitup or data representing a **MQTMC2** structure to be parsed.

-3 0 0 RXMQTM Zero input stem var

Explanation No value has been keyed for the first parameter, the name of a Stem. variable representing the message to be splitup or data representing a **MQTMC2** structure to be parsed.

-4 0 0 RXMQTM Null output stem var

Explanation A null has been supplied for the second parameter, the name of a Stem. variable representing the splitup data.

-5 0 0 RXMQTM Zero output stem var

Explanation No value has been keyed for the second parameter, the name of a Stem. variable representing the splitup data.

-6 0 0 RXMQTM No input data

Explanation The input Stem.0 was zero, indicating no message to process

-7 0 0 RXMQTM Zero input data

Explanation The length of the input Stem.1 was zero, indicating no message to process

-8 0 0 RXMQTM Cannot locate Header

Explanation The input Stem.0 was <= 3, indicating no header in the message

-9 0 0 RXMQTM Cannot find Header

Explanation The length of the input Stem.1 was <= 3, indicating no header in the message

-10 0 0 RXMQTM Zero input data

Explanation The length of the input data was zero, indicating no **MQTMC2** structure to process

-11 0 0 RXMQTM Cannot find Header

Explanation The length of the input data was <= 3, indicating no header in the alleged **MQTMC2** structure

-12 0 0 RXMQTM Unknown Header

Explanation The first 4 bytes of the input Stem. 1 or data was not '**TM** ', so the message did not come from an Initiation Queue or a Triggered Process' parameter, and so cannot be splitup

-13 0 0 RXMQTM Unknown Version

Explanation The second 4 bytes of the input Stem.1 or data was not that for a recognised Version field, so the message did not come from an Initiation Queue or a Triggered Process' parameter, and so cannot be splitup

-14 0 0 RXMQTM Header mismatch (1<>1)

Explanation Message mode was selected, but the StrucId field was not correct

-15 0 0 RXMQTM Version mismatch (1<>1)

Explanation Message mode was selected, but the Version field was not correct

-16 0 0 RXMQTM Header mismatch (2<>C)

Explanation Data mode was selected, but the StrucId field was not correct

-17 0 0 RXMQTM Version mismatch (2<>C)

Explanation Data mode was selected, but the Version field was not correct

-18 0 0 RXMQTM Too short for a TM (<n> bytes!)

Explanation Although the input Stem.1 looked like a Trigger Message, Stem.0 was too small for the message to originate from an Initiation Queue, and so cannot be splitup

-19 0 0 RXMQTM Too short for a TMC (<n> bytes!)

Explanation Although the input data looked like Trigger Params (MQTMC2), the data was too small (not 732 bytes long) for it to be a 'Triggered Process' Parameter and so cannot be splitup

-20 0 0 RXMQTM malloc failure, RC(<errno>)

Explanation An attempt to acquire Storage for the number of bytes specified in the input Stem.0 (1st parameter) variable failed. The return code is errno code from the malloc call.

-21 0 0 RXMQPUT1 Data length is not equal to specified value

Explanation The Stem.0 (1st parameter), the Stem variable containing the length of message to be splitup, is not equal to Stem.1 actual message length.

-99 0 0 RXMQTM UNKNOWN FAILURE

Explanation Some unknown error has occurred!

Trigger information

Table 9. Trigger Component (MQTM/MQTMC2) Mappings

Structure name	Stem. Component	Input, Output or Both	Format
<i>QName</i>	.QN	O	MQCHAR48
<i>ProcessName</i>	.PN	O	MQCHAR48
<i>TriggerData</i>	.TD	O	MQCHAR64
<i>ApplType</i>	.AT	O	MQLONG
<i>ApplId</i>	.AID	O	MQCHAR256
<i>EnvData</i>	.ED	O	MQCHAR128
<i>UserData</i>	.UD	O	MQCHAR128
<i>QMgrName</i>	.QM	O	MQCHAR48
<i>MQTMC2 parameter</i>	.PL	O	MQTMC2

Notes:

- Input, Output and Both show how the field is used
- Format shows the type of the field
- **ZLIST** is set to relevant existing field Stem. Component names
- Text items which are all Blanks (or start with a Binary Zero) are not generated
- **.AT** and **.PL** are only available in Message Mode
- **.QM** is only available in Data Mode
- **.PL** is not placed in **ZLIST**

Examples

```
/* A message has been obtained from an Initiation Queue */

message.0 = 684
message.1 = <MQTM>

/* Clear the result variable */

drop t.

/* Split the message */

rcc = RXMQTM( 'message.', 't.' )

/* on return, the following are set */

say t.QN      /* L3N1      */
say t.PN      /* P3T046N   */

/* Truncated non-parm areas for usage */

do j=1 to words(t.zlist)
  item = word(t.zlist,j)
  t.item = strip(t.item,'B')
end

/* Some processing to decide on something to do */

/* Start a Process to service the Queue      */

'@START CMD someproc.CMD' t.pl exit
```

Figure 3. A Trigger Monitor

This example shows how a message obtained from an Initiation Queue is splitup, showing how the PL component is used to start a process to service the Queue which generated the Trigger. Note that all the parameters passed in the Message can be used however one wants when one codes ones own Trigger Monitor.

```
/* Get the parm */

parse arg parm

/* Clear the result variable */

drop p.

/* Split the parm */

rcc = RXMQTM( parm, 'p.' )

/* on return, the following are set */

say p.QM      /* QM1      */
say p.QN      /* L3N1      */
say p.PN      /* P3T046N   */

/* Truncate areas for usage */

do j=1 to words(p.zlist)
  item = word(p.zlist,j)
  p.item = strip(p.item,'B')
end
```

Figure 4. A Rexx Triggered Process

This example shows how a Rexx Exec being initiated via a Trigger Monitor accesses its passed data.

Appendix A. ISPF Interface

As **RXMQC** operates within the Rexx environment, all the Rexx variables used are available for use within ISPF in the normal fashion. Figure 5. ISPF Exec (MA95T1) on page 72 shows an ISPF exec using **RXMQC** to issue a Command via the panel shown in Figure 6. ISPF Panel (MA95P1) on page 73. If everything works, then Figure 7. ISPF Panel (success) on page 74 is displayed, or else something like Figure 8. ISPF Panel (failure) on page 74.

```

/* REXX ***** */
"ISPQRY"                                /* Check ISPF availability */
lastrc = rc
if lastrc = 20 then do                  /* rc=20 is no ISPF,so start ISPF */
    address TSO 'ISPSTART CMD(MA95T1) NEWAPPL(MA95)'
    exit
end

address ispexec

qm          = '' ; com          = '' ; rcm          = '' ; rcc          = '' ; rcc1 = ''
reply1 = '' ; reply2          = '' ; reply3          = '' ; reply4          = '' ; rcc2 = ''

"VPUT (qm com rcm rcc rcc1 rcc2 reply1 reply2 reply3 reply4 ) SHARED"

Do mainloop = 1 by 1                  /* Do mainloop forever */

    "DISPLAY PANEL("MA95P1")" ; lastrc = rc

    if ( lastrc <> 0 ) then leave mainloop

    "VGET (qm com rcm rcc rcc1 rcc2 reply1 reply2 reply3 reply4) SHARED"

    rcm = '' ; rcc          = '' ; rcc1 = ''
    reply1 = '' ; reply2 = '' ; reply3 = '' ; reply4          = '' ; rcc2 = ''

drop rep.

rcci = RXMQINIT
rcc = RXMQC(qm, com, 'rep.' )
rcct = RXMQTERM

rcm = strip(word(rcc,1),'B')
rcm = rxmq.rcmap.rcm
rcc1 = rep.cc
rcc2 = rep.ac

if ( (rep.0 <> 'REP.0') & (rep.0 <> 0) ) then do
    if (rep.0 >= 1) then reply1 = rep.1
    if (rep.0 >= 2) then reply2 = rep.2
    if (rep.0 >= 3) then reply3 = rep.3
    if (rep.0 >= 4) then reply4 = rep.4
end

end

exit 0
/***** End of MA95T1 *****/

```

Figure 5. ISPF Exec (MA95T1)

```

)ATTR
/***** */
    # TYPE(INPUT) COLOR(WHITE)
    @ TYPE(OUTPUT) COLOR(TURQ)
    { TYPE(OUTPUT) CAPS(OFF)
)BODY SMSG(MSG)
%------ MA95 Rexx/MQ/MVS/ -----
%COMMAND ==> #ZCMD                                +

                                +MA95P1 Commands+

+QM+#qm                                +
+Command+#com                            +
+Reply1:#{reply1

```

```

+Reply2: +{reply2

+Reply3: +{reply3

+Reply4: +{reply4

+Com.cc {rcc1      +Com.rc {rcc2      +
+rc      {rcm
+rcc     {rcc

{msg
)INIT
/*****/
/*  INITIALIZATION SECTION */
/*****/
)PROC
/*****/
/*  PROCESSING SECTION */
/*****/
)END

```

Figure 6. ISPF Panel (MA95P1)

```

----- MA95 Rexx/MQ/MVS/ -----
                                  MA95P1 Commands

QM QM1
Command DISPLAY QUEUE(N1) ALL
Reply1: CSQM401I  QUEUE(N1) TYPE(LOCAL)
DESCR(Notpersist)
PUT(ENABLED  ) DEFPRTY(      0) DEFPSIST(NO      ) OPPOCS(
Reply2: CSQ9022I > CSQMDRTS ' DISPLAY QUEUE' NORMAL COMPLETION

Reply3:

Reply4:

Com.cc 00000000      Com.rc 00000000
Rc      MQCC_OK
rcc      0 0 0 RXMQC OK

```

Figure 7. ISPF Panel (success)

```

----- MA95 Rexx/MQ/MVS/ -----
                                  MA95P1 Commands

QM QM1
Command DISPLAY QZ  (N1) ALL
Reply1: CSQ9021E > VERB ' DISPLAY' REQUIRED KEYWORD IS MISSING

Reply2: CSQ9023E > CSQ9SCND 'DISPLAY ' ABNORMAL COMPLETION

Reply3:

Reply4:

Com.cc 00000008      Com.rc FFFFFFFF
Rc      MQCC_OK
rcc      0 0 0 RXMQC OK

```

Figure 8. ISPF Panel (failure)

Appendix B. Sample REXX execs

MA95\samples directory contains a number of useful examples related to both z/OS and Windows environments. They are generally self-explaining and show how to exploit the functions provided by SupportPac. In order to run them you can use sample jobs in **MA95\MVS\JCL** or batch files in **MA95\Windows\Runttests**. Before running these samples please update Queue Manager and Queue names to comply with your WebSphere MQ installation names.

- **RXMQCMND.REXX** – sends a command to Queue Manager, receives a response and presents it to the REXX caller.
- **RXMQEVNT.REXX** – illustrates EVENT queue interface. Requests data from one of EVENT queues, interprets it by calling **RXMQEVNT**, and presents the results.
- **RXMQHXT.REXX** – illustrates transmission queue message header parsing. Gets a message from transmission queue, interprets it by calling **RXMQHXT**, and presents the results.
- **RXMQMA95.REXX** – perform basic WebSphere MQ functions, showing how to process the results and return/reason codes.
- **RXMQTM.REXX** – shows how to use Trigger Monitor function of SupportPac. Gets message from Initiation Queue, calls **RXMQTM** to interpret the message, and presents the results.

Appendix C. Message properties

This appendix documents the MA95 Rexx interfaces for IBM MQ message handles and message properties. These calls are wrappers around MQCRTMH, MQDLTMH, MQSETMP, MQINQMP, MQDLTMP, MQMHBUF and MQBUFMH. They complement RXMQPUT and RXMQGET; they do not replace them.

Tested platform: z/OS with IBM MQ. The message-handle and message-property extensions, including HANDLE1 through HANDLE10, have not been tested on Windows. No claim of Windows compatibility is made for this version.

A message handle is an IBM MQ object associated with the current queue-manager connection. Properties can be added to the handle before a put, received into a handle during a get, queried or deleted, and converted to or from an MQRFH2 representation when an application needs an explicit message buffer.

The normal direct path is: create a handle with RXMQMH, add properties with RXMQSMP, pass the handle in MQPMO.NewMsgHandle during RXMQPUT, create another handle, pass it in MQGMO.MsgHandle during RXMQGET, and read the received properties with RXMQIMP. RXMQMBF and RXMQBMH are only required when an explicit MQRFH2 buffer conversion is needed.

Message handle usage

Putting a message with properties:

```
rcc = RXMQMH('mhput','cmho.')
rcc = RXMQSMP('mhput','smo.','usr.colour','pd.',MQTYPE_STRING,'blue')
ipmo.VER = MQPMO_VERSION_3
ipmo.NMH = mhput
rcc = RXMQPUT(hobj,'message.','imd.','omd.','ipmo.','opmo.')
```

Getting a message and receiving its properties into a handle:

```
rcc = RXMQMH('mhget','cmho.')
igmo.VER = MQGMO_VERSION_4
igmo.MH = mhget
rcc = RXMQGET(hobj,'message.','imd.','omd.','igmo.','ogmo.')
pvalue.0 = 4096
rcc = RXMQIMP('mhget','impo.','usr.colour','pdout.','ptype','pvalue.','pnameout')
```

Message handles should be deleted with RXMQDMH when no longer required. A handle is valid only for the queue-manager connection on which it was created.

Message property option and descriptor structures

The following sections describe the IBM MQ structures represented by Rexx stems in the MA95 message-handle and message-property interfaces. The tables show the fields that MA95 accepts or returns, the preferred stem component names, and the initial values supplied by the IBM MQ default structures.

Fields such as StrucId are initialized internally by MA95 and are therefore not represented by Rexx stem components. When a component is omitted, MA95 begins with the corresponding IBM MQ *_DEFAULT structure and applies only the components supplied by the Rexx program.

The Create Message Handle Options Structure (MQCMHO)

Description

MQCMHO controls creation of a message handle. MA95 initializes the structure from MQCMHO_DEFAULT, overlays the supplied stem components, calls MQCRTMH, and returns the resulting Version and Options values to the same stem.

This structure is used by RXMQMH.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQCMHO_VERSION_1
<i>Options</i>	.OPT	B	MQCMHO_DEFAULT_VALIDATION

Notes:

- B means that the component can be supplied as input and is returned to the same stem.
- If the optional MQCMHO stem is omitted from RXMQMH, MA95 uses MQCMHO_DEFAULT.
- MQCMHO_DEFAULT_VALIDATION currently performs the same property-name validation as MQCMHO_VALIDATE.
- ZLIST contains VER and OPT on return.

Options

Value	Meaning
MQCMHO_DEFAULT_VALIDATION	Use the queue manager default validation level for property names.
MQCMHO_VALIDATE	Validate property names when properties are subsequently set in the handle.
MQCMHO_NO_VALIDATION	Create the handle without property-name validation.
MQCMHO_NONE	No explicit option. The numeric value is zero and default processing applies.

Example

```
cmho1.VER = MQCMHO_CURRENT_VERSION
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1','cmho1.')
```

The Delete Message Handle Options Structure (MQDMHO)

Description

MQDMHO controls deletion of a message handle. MA95 initializes the structure from MQDMHO_DEFAULT, overlays the supplied stem components, calls MQDLTMH, and returns the resulting values to the same stem.

This structure is used by RXMQDMH.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQDMHO_VERSION_1
<i>Options</i>	.OPT	B	MQDMHO_NONE

Notes:

- If the optional MQDMHO stem is omitted from RXMQDMH, MA95 uses MQDMHO_DEFAULT.
- After a successful RXMQDMH call, the Rexx variable containing the message handle is reset to MQHM_NONE.
- ZLIST contains VER and OPT on return.

Options

Value	Meaning
MQDMHO_NONE	No delete-message-handle options are defined.

Example

```
dmho1.VER = MQDMHO_CURRENT_VERSION
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1', 'dmho1.')
```

The Set Message Property Options Structure (MQSMPO)

Description

MQSMPO controls how MQSETMP locates or creates a property. MA95 exposes Version and Options in the Rexx stem. ValueEncoding and ValueCCSID are initialized from MQSMPO_DEFAULT and are not currently exposed as stem components.

This structure is used by RXMQSMP.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQSMPO_VERSION_1
<i>Options</i>	.OPT	B	MQSMPO_SET_FIRST / MQSMPO_NONE
<i>ValueEncoding</i>	Not exposed	I	MQENC_NATIVE
<i>ValueCCSID</i>	Not exposed	I	MQCCSI_APPL

Notes:

- RXMQSMP supplies the property value as Rexx bytes. IBM MQ interprets those bytes according to the MQTYPE_* parameter and the MQSMPO default encoding and CCSID.
- The property cursor used by the cursor-relative options is established by RXMQIMP using MQIMPO_INQ_FIRST or MQIMPO_INQ_NEXT.
- ZLIST contains VER and OPT on return.

Options

Value	Meaning
MQSMPO_SET_FIRST	Set the first matching property; create it when no matching property exists.
MQSMPO_SET_PROP_UNDER_CURSOR	Set the property currently identified by the property cursor.

MQSMPO_SET_PROP_AFTER_CURSOR	Insert a new property after the property under the cursor.
MQSMPO_SET_PROP_BEFORE_CURSOR	Insert a new property before the property under the cursor.
MQSMPO_APPEND_PROPERTY	Append a new property after existing properties with the same name hierarchy.
MQSMPO_NONE	No explicit option. This has the same zero value as MQSMPO_SET_FIRST.

Example

```

smpos1.VER = MQSMPO_CURRENT_VERSION
smpos1.OPT = MQSMPO_NONE
rcc = RXMQSMP('mh1', 'smpos1.', 'usr.color', 'pd1.', MQTYPE_STRING, 'red')

```

The Property Descriptor Structure (MQPD)

Description

MQPD describes the attributes of a property. It is an input/output structure for MQSETMP and an output structure for MQINQMP. MA95 accepts both the preferred long stem component names and the original short compatibility names.

This structure is used by RXMQSMP and RXMQIMP.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQPD_VERSION_1
<i>Options</i>	.OPT	B	MQPD_NONE
<i>Support</i>	.SUPPORT	B	MQPD_SUPPORT_OPTIONAL
<i>Context</i>	.CONTEXT	B	MQPD_NO_CONTEXT
<i>CopyOptions</i>	.COPYOPTIONS	B	MQCOPY_DEFAULT

Notes:

- Use SUPPORT, CONTEXT and COPYOPTIONS in new programs. SUP, CTX and CO remain supported as compatibility aliases.
- On return, MA95 populates both the long and short aliases. ZLIST uses the canonical compatibility components VER OPT SUP CTX CO.
- For user-defined usr.* properties, MQPD_SUPPORT_OPTIONAL, MQPD_NO_CONTEXT and MQCOPY_DEFAULT are the normal settings.
- IBM MQ can correct descriptor attributes for recognized IBM MQ-defined properties.

Field details

Options. MQPD_NONE is the only currently defined value.

Support. Specifies whether an application that does not understand the property may accept the message. Common values are MQPD_SUPPORT_OPTIONAL, MQPD_SUPPORT_REQUIRED and MQPD_SUPPORT_REQUIRED_IF_LOCAL.

Context. MQPD_NO_CONTEXT associates the property with no special message context. MQPD_USER_CONTEXT associates it with user context and affects how the property is propagated when context is passed.

CopyOptions. Controls copying to forwarded, published, reply or report messages. MQCOPY_DEFAULT includes the standard forward, publish and report behavior; MQCOPY_NONE prevents ordinary copy propagation.

Example

```
pd1.VER          = MQPD_CURRENT_VERSION
pd1.OPT          = MQPD_NONE
pd1.SUPPORT      = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT      = MQPD_NO_CONTEXT
pd1.COPYOPTIONS  = MQCOPY_DEFAULT
```

The Inquire Message Property Options Structure (MQIMPO)

Description

MQIMPO controls direct inquiry and enumeration of properties. MA95 exposes the encoding and CCSID fields in the stem. The returned property name and the numeric MQTYPE_* value are returned through separate RXMQIMP parameters rather than through MQIMPO.ReturnedName and MQIMPO.TypeString.

This structure is used by RXMQIMP.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQIMPO_VERSION_1
<i>Options</i>	.OPT	B	MQIMPO_INQ_FIRST
<i>RequestedEncoding</i>	.RQENC	B	MQENC_NATIVE
<i>RequestedCCSID</i>	.RQCCSI	B	MQCCSI_APPL
<i>ReturnedEncoding</i>	.RTENC	O	MQENC_NATIVE
<i>ReturnedCCSID</i>	.RTCCSI	O	0
<i>ReturnedName</i>	Separate parameter	O	MQCHARV_DEFAULT
<i>TypeString</i>	Type variable	O	Blank

Notes:

- REQENC and REQUESTEDENCODING are aliases for RQENC. REQCCSI and REQUESTEDCCSID are aliases for RQCCSI.
- RETENC and RETURNEDENCODING are aliases for RTENC. RETCCSI and RETURNEDCCSID are aliases for RTCCSI.
- ZLIST contains VER OPT RQENC RQCCSI RTENC RTCCSI.
- Set value.0 to the maximum property-value buffer length before every RXMQIMP call.

Options

Value	Meaning
MQIMPO_INQ_FIRST	Inquire the first property matching the supplied name or wildcard pattern and establish the property cursor.
MQIMPO_INQ_NEXT	Continue from the property cursor and return the next matching property.
MQIMPO_INQ_PROP_UNDER_CURSOR	Inquire the property currently identified by the cursor.
MQIMPO_CONVERT_TYPE	Allow IBM MQ to convert the property to the requested MQTYPE_* when supported.
MQIMPO_QUERY_LENGTH	Return the required value length without returning the value bytes.
MQIMPO_CONVERT_VALUE	Convert a returned value to RequestedEncoding and RequestedCCSID when supported.
MQIMPO_NONE	No explicit option. This has the same zero value as MQIMPO_INQ_FIRST.

Example

```
impo1.OPT      = MQIMPO_NONE
impo1.RQENC    = MQENC_NATIVE
impo1.RQCCSI   = MQCCSI_APPL
```

```
pvalue.0 = 4096
rcc = RXMQIMP('mh1','impo1.','usr.color','pdout.','ptype','pvalue.','pnameout')
```

The Delete Message Property Options Structure (MQDMPO)

Description

MQDMPO controls which matching property is deleted by MQDLTMP. MA95 initializes the structure from MQDMPO_DEFAULT, overlays the stem, and returns the resulting Version and Options values.

This structure is used by RXMQDMP.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQDMPO_VERSION_1
<i>Options</i>	.OPT	B	MQDMPO_DEL_FIRST / MQDMPO_NONE

Notes:

- MQDMPO_DEL_PROP_UNDER_CURSOR requires a property cursor previously established by RXMQIMP.
- The cursor is reset when the message handle is reused by relevant MQPUT or MQGET processing.
- ZLIST contains VER and OPT on return.

Options

Value	Meaning
MQDMPO_DEL_FIRST	Delete the first property matching the supplied name.
MQDMPO_DEL_PROP_UNDER_CURSOR	Delete the property currently identified by the property cursor.
MQDMPO_NONE	No explicit option. This has the same zero value as MQDMPO_DEL_FIRST.

Example

```
dmpo1.VER = MQDMPO_CURRENT_VERSION
dmpo1.OPT = MQDMPO_DEL_FIRST
rcc = RXMQDMP('mh1','dmpo1.','usr.color')
```

The Message Handle To Buffer Options Structure (MQMHBO)

Description

MQMHBO controls conversion of selected message-handle properties into an explicit buffer by MQMHBUF. The required representation is an MQRFH2 header. MA95 initializes the structure from MQMHBO_DEFAULT.

This structure is used by RXMQMBF.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQMHBO_VERSION_1
<i>Options</i>	.OPT	B	MQMHBO_PROPERTIES_IN_ MQRFH2

Notes:

- MQMHBO_PROPERTIES_IN_MQRFH2 is required for MQMHBUF and is present in MQMHBO_DEFAULT.
- RXMQMBF returns the updated MQMD, the generated buffer, and the IBM MQ DataLength.
- ZLIST contains VER and OPT on return.

Options

Value	Meaning
MQMHBO_PROPERTIES_IN_MQRFH2	Represent selected properties in MQRFH2 format in the output buffer.
MQMHBO_DELETE_PROPERTIES	Delete from the handle the properties successfully added to the buffer.
MQMHBO_NONE	No optional flags. MQMHBO_PROPERTIES_IN_MQRFH2 is still required by the IBM MQ call.

Example

```
mhbo1.VER = MQMHBO_CURRENT_VERSION
mhbo1.OPT = MQMHBO_PROPERTIES_IN_MQRFH2
rcc = RXMQMBF('mh1','mhbo1.','usr.%','md1.','buf',8192,'blen')
```

The Buffer To Message Handle Options Structure (MQBMHO)

Description

MQBMHO controls parsing of an explicit message buffer into a message handle by MQBUFMH. MA95 initializes the structure from MQBMHO_DEFAULT, which requests deletion of extracted property data from the supplied buffer representation.

This structure is used by RXMQBMH.

Stem mapping and initial values

Structure field	Stem component	Use	Initial value
<i>Version</i>	.VER	B	MQBMHO_VERSION_1
<i>Options</i>	.OPT	B	MQBMHO_DELETE_PROPERTIES

Notes:

- The destination message handle must already exist before RXMQBMH is called.
- Use MQBMHO_NONE to retain the property representation in the input buffer; use MQBMHO_DELETE_PROPERTIES to remove the extracted property representation as IBM MQ processes the buffer.
- ZLIST contains VER and OPT on return.

Options

Value	Meaning
MQBMHO_DELETE_PROPERTIES	Delete property data from the buffer as the properties are added to the handle.
MQBMHO_NONE	Add properties to the handle without requesting deletion from the buffer.

Example

```
rcc = RXMQMH('mh2','cmho2.')
bmho1.VER = MQBMHO_CURRENT_VERSION
bmho1.OPT = MQBMHO_DELETE_PROPERTIES
```

```
rcc = RXMQBMH('mh2','bmho1.','md1.',buf)
```

RXMQMH

Description

This call creates an IBM MQ message handle by issuing MQCRTMH. The resulting MQHMSG value is written to the Rexx variable whose name is supplied in the first parameter. The options stem is optional; if omitted, the MA95 MQCMHO defaults are used. The MQCMHO stem is described in “The Create Message Handle Options Structure (MQCMHO)”.

Parameters

Parameter	Direction	Description
handle-variable	Output	Name of the Rexx variable that receives the MQHMSG value, for example mh1.
cmho-stem	Input/Output, optional	Name of an MQCMHO stem, for example cmho1. The trailing period is required. The updated structure is returned to the same stem.

Call

```
rcc = RXMQMH( handle-variable [, cmho-stem ] )
```

Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null CMHO
-5	Zero length CMHO
-99	UNKNOWN FAILURE

Usage Notes

One or two parameters are accepted.

On success, MA95 also updates RXMQ.HMSG and the environment-specific compatibility HMSG variable.

If no queue-manager connection exists, MQRC_HCONN_ERROR is returned as an IBM MQ reason code.

Example

```
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1','cmho1.')
say 'MH1 =' rcc mh1
```

Related samples

HANDLE1 through HANDLE10 demonstrate message-handle creation with RXMQMH.

RXMQDMH

Description

This call deletes an IBM MQ message handle by issuing MQDLTMH. The handle variable is reset to MQHM_NONE after a successful call. The MQDMHO stem is described in “The Delete Message Handle Options Structure (MQDMHO)”.

Parameters

Parameter	Direction	Description
handle-variable	Input/Output	Name of the Rexx variable containing the MQHMSG value. It is reset after successful deletion.
dmho-stem	Input/Output, optional	Name of an MQDMHO stem, for example dmho1. The trailing period is required.

Call

```
rcc = RXMQDMH( handle-variable [, dmho-stem ] )
```

Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null DMHO
-5	Zero length DMHO
-99	UNKNOWN FAILURE

Usage Notes

Delete every message handle that is no longer required.

Do not reuse the previous handle value after a successful call.

Example

```
dmho1.OPT = MQDMHO_NONE  
rcc = RXMQDMH('mh1', 'dmho1.')  
say 'DMH mh1 =' rcc
```

Related samples

HANDLE1 through HANDLE10 demonstrate message-handle deletion with RXMQDMH.

RXMQSMP

Description

This call creates or sets a property in a message handle by issuing MQSETMP. The property exists in the handle; it is not placed on a queue until the handle is associated with an RXMQPUT operation, normally through MQPMO.NewMsgHandle. The MQSMPO and MQPD stems are described in “The Set Message Property Options Structure (MQSMPO)” and “The Property Descriptor Structure (MQPD)”.

Parameters

Parameter	Direction	Description
handle-variable	Input	Name of the Rexx variable containing the MQHMSG value.
smpos-stem	Input/Output	Name of an MQSMPO stem.
property-name	Input	Property name, for example usr.colour.
pd-stem	Input/Output	Name of an MQPD property-descriptor stem.
property-type	Input	An MQTYPE_* value, for example MQTYPE_STRING.
property-value	Input	The property value. Its Rexx string length is passed to MQSETMP.

Call

```
rcc = RXMQSMP( handle-variable, smpos-stem, property-name, pd-stem, property-type, property-value )
```


Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null SMPO
-5	Zero length SMPO
-6	Null property name
-7	Zero length property name
-8	Null PD
-9	Zero length PD
-10	Null property type
-11	Zero length property type
-12	Null property value
-99	UNKNOWN FAILURE

Usage Notes

Exactly six parameters are required.

The property descriptor and set-property options are returned to their original stems after a successful call.

The value is supplied as Rexx bytes. The selected MQTYPE_* determines how IBM MQ interprets those bytes.

Example

```
smpos1.OPT      = MQSMPO_NONE
pd1.OPT         = MQPD_NONE
pd1.SUPPORT     = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT     = MQPD_NO_CONTEXT
pd1.COPYOPTIONS = MQCOPY_DEFAULT
rcc = RXMQSMP('mh1','smpos1','usr.color','pd1.',MQTYPE_STRING,'red')
```

Related samples

HANDLE1 through HANDLE10 demonstrate setting properties with RXMQSMP.

RXMQIMP

Description

This call reads a property from a message handle by issuing MQINQMP. It supports both a direct inquiry using a specific property name and enumeration using MQIMPO_INQ_FIRST followed by MQIMPO_INQ_NEXT. The current interface uses a Rexx value stem so that the caller can provide the maximum buffer size and receive both the actual DataLength and the returned property bytes. The MQIMPO and returned MQPD stems are described in “The Inquire Message Property Options Structure (MQIMPO)” and “The Property Descriptor Structure (MQPD)”.

Parameters

Parameter	Direction	Description
handle-variable	Input	Name of the Rexx variable containing the MQHMSG value.
impo-stem	Input/Output	Name of an MQIMPO stem.
property-name	Input	A specific property name, or an enumeration pattern such as usr.%.
pd-stem	Output	Name of the stem that receives the MQPD property descriptor.
type-variable	Output	Name of the Rexx variable that receives the MQTYPE_* value.
value-stem	Input/Output	Stem used for the property value. Before the call value.0 contains the maximum buffer size. After the call value.0 contains DataLength and

Parameter	Direction	Description
returned-name-variable	Output	value.1 contains the returned bytes. Name of the Rexx variable that receives the actual property name returned by MQINQMP. For usr.% enumeration, the returned name is normally the matching name without the usr. prefix.

Call

```
rcc = RXMQIMP( handle-variable, impo-stem, property-name, pd-stem, type-variable, value-stem, returned-name-variable )
```

Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null IMPO
-5	Zero length IMPO
-6	Null property name
-7	Zero length property name
-8	Null PD
-9	Zero length PD
-10	Null type variable
-11	Zero length type variable
-12	Null value stem
-13	Zero length value stem
-14	Null returned property name variable
-15	Zero length returned property name variable
-16	Invalid value buffer length in value.0
-17	Value buffer allocation failed
-18	Returned property name buffer allocation failed
-99	UNKNOWN FAILURE

Usage Notes

Exactly seven parameters are required. This replaces the older scalar-value form of RXMQIMP.

Set value.0 to a positive maximum buffer length before every call. The implementation allocates this buffer for MQINQMP.

After a successful call, value.0 is the actual DataLength returned by MQINQMP and value.1 contains the property bytes.

For a direct inquiry, MQIMPO_NONE may be used. For enumeration, use MQIMPO_INQ_FIRST on the first call, then MQIMPO_INQ_NEXT on subsequent calls.

MQRC_PROPERTY_NOT_AVAILABLE marks the end of an enumeration. The sample loop tests RXMQ.LASTAC for this reason code.

MQIMPO_CONVERT_VALUE can be combined with the inquiry option when MQ should convert the returned value to the requested encoding and CCSID.

Example

```
/* Direct inquiry */
impo1.OPT = MQIMPO_NONE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
pvalue.0 = 4096
rcc = RXMQIMP('mh2', 'impo1.', 'usr.color', 'pdout.', 'ptype', 'pvalue.', 'pnameout')
say 'name=' pnameout 'type=' ptype 'length=' pvalue.0 'value=' pvalue.1

/* Enumeration */
impo1.OPT = MQIMPO_INQ_FIRST
pname = 'usr.%'
do until RXMQ.LASTAC = MQRC_PROPERTY_NOT_AVAILABLE
  pvalue.0 = 4096
```

```

pvalue.1 = ''
pnameout = ''
rcc = RXMQIMP('mh2','impo1.',pname,'pdout.','ptype','pvalue.','pnameout')
if word(rcc,1) <> 0 then leave
say pnameout '=' pvalue.1
impo1.OPT = MQIMPO_INQ_NEXT
end

```

Related samples

HANDLE1, HANDLE2, and HANDLE4 through HANDLE10 demonstrate direct inquiry or enumeration with RXMQIMP.

RXMQDMP

Description

This call deletes a property from a message handle by issuing MQDLTMP. The MQDMPO stem is described in “The Delete Message Property Options Structure (MQDMPO)”.

Parameters

Parameter	Direction	Description
handle-variable	Input	Name of the Rexx variable containing the MQHMSG value.
dmpo-stem	Input/Output	Name of an MQDMPO stem.
property-name	Input	Name of the property to delete.

Call

```
rcc = RXMQDMP( handle-variable, dmpo-stem, property-name )
```

Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null DMPO
-5	Zero length DMPO
-6	Null property name
-7	Zero length property name
-99	UNKNOWN FAILURE

Usage Notes

Exactly three parameters are required.

The updated MQDMPO structure is returned to the supplied stem.

A subsequent RXMQIMP for the deleted property normally returns MQRC_PROPERTY_NOT_AVAILABLE.

Example

```

dmpo1.OPT = MQDMPO_NONE
rcc = RXMQDMP('mh1','dmpo1.','usr.color')

```

Related samples

HANDLE6 demonstrates property deletion with RXMQDMP.

RXMQMBF

Description

This call converts selected properties in a message handle to an explicit message buffer by issuing MQMHBUF. The output commonly contains an MQRFH2 representation and the MQMD is updated accordingly. The MQMHBO stem is described in “The Message Handle To Buffer Options Structure (MQMHBO)”.

Parameters

Parameter	Direction	Description
handle-variable	Input	Name of the Rexx variable containing the MQHMSG value.
mhbo-stem	Input/Output	Name of an MQMHBO stem.
property-name	Input	Property name or pattern to include, for example usr.%.
md-stem	Input/Output	Name of an MQMD stem. MQMHBUF can update Format, Encoding and CodedCharSetId.
buffer-variable	Output	Name of the Rexx variable that receives the generated bytes.
buffer-length	Input	Maximum number of bytes available for the generated buffer.
data-length-variable	Output	Name of the Rexx variable that receives the DataLength returned by MQMHBUF.

Call

```
rcc = RXMQMBF( handle-variable, mhbo-stem, property-name, md-stem, buffer-variable, buffer-length, data-length-variable )
```

Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null MHBO
-5	Zero length MHBO
-6	Null name
-7	Zero length name
-8	Null MD
-9	Zero length MD
-10	Null output buffer variable
-11	Zero length output buffer variable
-12	Null buffer length
-13	Zero length buffer length
-14	Null data length variable
-15	Zero length data length variable
-16	Buffer length must be greater than zero
-17	Unable to allocate output buffer
-99	UNKNOWN FAILURE

Usage Notes

Exactly seven parameters are required.

Unlike message data used by RXMQPUT and RXMQGET, the output buffer is a scalar Rexx variable; the returned length is placed in the separate data-length variable.

On success, only DataLength bytes are returned in the buffer variable.

This API is not required for the normal NewMsgHandle/MsgHandle PUT and GET path.

Example

```
mhbo1.OPT = MQMHBO_NONE
```

```

rcc = RXMQMBF('mh1', 'mhbo1.', 'usr.%', 'md1.', 'buf', 8192, 'blen')
say 'MBF =' rcc
say 'buffer length =' length(buf)
say 'DataLength    =' blen
say 'MD.Format     =' md1.FORM

```

Related samples

HANDLE3 and HANDLE8 demonstrate conversion from a message handle to an MQRFH2 buffer with RXMQMBF.

RXMQBMH

Description

This call parses an explicit message buffer into a message handle by issuing MQBUFMH. It is typically used to reconstruct message properties from an MQRFH2 buffer previously produced by RXMQMBF or received by another application. The MQBMHO stem is described in “The Buffer To Message Handle Options Structure (MQBMHO)”.

Parameters

Parameter	Direction	Description
handle-variable	Input	Name of the Rexx variable containing the destination MQHMSG value.
bmho-stem	Input/Output	Name of an MQBMHO stem.
md-stem	Input/Output	Name of an MQMD stem describing the supplied buffer.
buffer-variable	Input	Rexx variable containing the bytes to parse.

Call

```

rcc = RXMQBMH( handle-variable, bmho-stem, md-stem, buffer-variable )

```

Additional Interface Return Codes and Messages

If the underlying IBM MQ verb is called, its Completion Code and Reason Code are returned in the normal MA95 return string. The following negative values are detected by the Rexx interface before, or while preparing, the MQ call:

Return code	Message
-1	Bad number of parameters
-2	Null handle name
-3	Zero length handle name
-4	Null BMHO
-5	Zero length BMHO
-6	Null MD
-7	Zero length MD
-8	Null input buffer
-9	Zero length input buffer
-99	UNKNOWN FAILURE

Usage Notes

Exactly four parameters are required.

The destination message handle must already have been created with RXMQMH.

After successful reconstruction, use RXMQIMP to inquire or enumerate the properties in the destination handle.

Example

```

rcc = RXMQMH('mh2', 'cmho2.')
bmho1.OPT = MQBMHO_NONE
rcc = RXMQBMH('mh2', 'bmho1.', 'md1.', buf)

```

Related samples

HANDLE3 and HANDLE8 demonstrate reconstruction of message properties from an MQRFH2 buffer with RXMQBMH.

Appendix D. Message property samples

This appendix contains the ten complete Rexx samples supplied with the message-handle and message-property extensions. The executable statements are taken from the validated HANDLE1 through HANDLE10 members; comments and diagnostic text have been standardized in English for this edition. The samples use the generic queue manager QM1 and queue DEV.QUEUE.1. Change these values before execution to match the local z/OS environment. The examples contain no passwords, credentials, private host names, IP addresses, personal data, or site-specific dataset names. Each sample calls RXMQINIT and performs explicit cleanup of the resources it creates.

Sample overview

HANDLE1 - Put and get one property using message handles

HANDLE2 - Enumerate several received properties

HANDLE3 - Convert properties to MQRFH2 and reconstruct them

HANDLE4 - Use several IBM MQ property data types

HANDLE5 - Forward a message and copy properties with MQACTP_FORWARD

HANDLE6 - Delete a property with MQDLTMP

HANDLE7 - Enumerate properties with MQINQMP

HANDLE8 - Convert a message handle to and from an MQRFH2 buffer

HANDLE9 - Demonstrate MQPD_USER_CONTEXT and context passing

HANDLE10 - Select messages with MQOD.SelectionString

HANDLE1 - Put and get one property using message handles

Purpose

Demonstrates the preferred message-handle path for a single user property. The program sets usr.color=red in a source handle, supplies that handle to RXMQPUT, receives the property into a second handle with RXMQGET, and verifies it with RXMQIMP.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

```
RXMQMH(mh1) -> RXMQSMP -> RXMQPUT(NewMsgHandle=mh1)
RXMQMH(mh2) -> RXMQGET(MsgHandle=mh2) -> RXMQIMP
```

Complete Rexx sample

```
/* REXX */
/*
RXMQVTRACE = 'CONN OPEN PUT GET CLOSE DISC MH SMP MBF BMH IMP DMH'
*/
TRACE 0
qmgr = 'QM1'
qname = 'DEV.QUEUE.1'
say "QMGR =" qmgr
say "QUEUE=" qname
/* ----- */
/* 0. Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
say time('l') 'initialization problem rc ==> ' rcc
interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
say "                               ==> "msgerr
```

```

        exit 8
    end
/* Connection */
rcc = RXMQCONN(qmgr)
say "CONN =" rcc
/* OPEN */
/* OPEN */
iod1.OT = MQOT_Q
iod1.ON = qname
iod1.OM = ''
oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF QUIESCING
rcc = RXMQOPEN('iod1.', oo1, 'h1', 'ood1.')
say "OPEN =" rcc " h1=" h1
/* MH source */
/* cmho1.OPT = MQCMHO_NONE */
/* cmho1.OPT = MQCMHO_NO_VALIDATION */
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1', 'cmho1.')
say "MH1 =" rcc mh1
/* Property usr.color = red */
smpo1.OPT = MQSMP_O_NONE
pd1.OPT = MQPD_NONE
pd1.SUPPORT = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT = MQPD_NO_CONTEXT
pd1.COPYOPTIONS = MQCOPY_DEFAULT
rcc = RXMQSMP('mh1', 'smpo1.', 'usr.color', 'pd1.', MQTYPE_STRING, 'red')
say "SMP =" rcc
if word(rcc,1) <> 0 then signal cleanup_mh1
/* Simple application data */
d.0 = 5
d.1 = 'hello'
/* MQMD PUT */
imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING
imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF
/* PMO: the important field is NMH = mh1 */
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF QUIESCING + MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED
ipmo2.OMH = 0
ipmo2.NMH = mh1
ipmo2.ACT = MQACTP_NEW
ipmo2.PL = 0
rcc = RXMQPUT(h1, 'd.', 'imd2.', 'omd2.', 'ipmo2.', 'opmo2.')
say "PUT =" rcc
if word(rcc,1) <> 0 then signal cleanup_mh1
/* MH1 can be deleted after the PUT */
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1', 'dmho1.')
say "DMH mh1 =" rcc
/* MH2: handle that receives the properties during GET */
cmho2.OPT = MQCMHO_NONE
rcc = RXMQMH('mh2', 'cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then signal cleanup_close
/* Prepare the GET */
msg2.0 = 8192
igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE
/* GMO: the important field is MH = mh2 */
igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT + MQGMO_FAIL_IF QUIESCING
igmo2.WAIT = 5000
igmo2.MOPT = MQMO_NONE
igmo2.MH = mh2
rcc = RXMQGET(h1, 'msg2.', 'igmd2.', 'ogmd2.', 'igmo2.', 'ogmo2.')
say "GET =" rcc
say "msg2.0 =" msg2.0
say "msg2.1 length =" length(msg2.1)
say "msg2.1 =" msg2.1
if word(rcc,1) <> 0 then signal cleanup_mh2

```



```

/* IMP: inquire the property directly from mh2 */
impo1.OPT = MQIMPO_NONE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
pname = 'usr.color'
pvalue.1 = ''
pvalue.0 = 4096
rcc = RXMQIMP('mh2', 'impo1.', pname, 'pdout.', 'ptype', 'pvalue.', 'pnameout')
say "IMP      =" rcc
say "ptype    =" ptype
say "pvalue   =" pvalue.1
say "datalength =" pvalue.0
say "pnameout  =" pnameout
if pvalue.1 = 'red' then
    say "OK: property usr.color = red"
else
    say "FAILED: unexpected property value"
/* Normal cleanup */
cleanup_mh2:
    dmho2.OPT = MQDMHO_NONE
    rcc = RXMQDMH('mh2', 'dmho2.')
    say "DMH mh2 =" rcc
cleanup_close:
    rcc = RXMQCLOS(h1,0)
    say "CLOSE =" rcc
cleanup_disc:
    rcc = RXMQDISC()
    say "DISC =" rcc
exit 0
cleanup_mh1:
    dmho1.OPT = MQDMHO_NONE
    rcc = RXMQDMH('mh1', 'dmho1.')
    say "DMH mh1 =" rcc
    signal cleanup_close

```

Discussion

1. MQPMO.NewMsgHandle identifies the handle whose properties are associated with the message being put.
2. MQGMO.MsgHandle identifies the handle that receives properties when the message is retrieved.
3. The seven-parameter RXMQIMP interface returns the property type, value length, value bytes, property descriptor, and returned property name.
4. Both message handles are deleted explicitly after use.

Expected result

RXMQIMP completes with MQCC_OK.

ptype is MQTYPE_STRING.

pvalue.1 contains red and pvalue.0 contains 5.

pnameout identifies color.

HANDLE2 - Enumerate several received properties

Purpose

Extends the normal put/get path by setting usr.color and usr.country, then enumerating all received properties that match usr.%. It demonstrates the transition from MQIMPO_INQ_FIRST to MQIMPO_INQ_NEXT and the normal end-of-enumeration condition.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

```
RXMQSMP(color) + RXMQSMP(country) -> RXMQPUT -> RXMQGET  
RXMQIMP(INQ_FIRST) -> RXMQIMP(INQ_NEXT) -> MQRC_PROPERTY_NOT_AVAILABLE
```

Complete Rexx sample

```
/* REXX */  
/*  
RXMQVTRACE = 'CONN OPEN PUT GET CLOSE DISC MH SMP MBF BMH IMP DMH'  
*/  
TRACE 0  
qmgr = 'QM1'  
qname = 'DEV.QUEUE.1'  
say "QMGR =" qmgr  
say "QUEUE=" qname  
/* ----- */  
/* 0. Initialization */  
/* ----- */  
rcc = rxmqinit()  
if word(rcc,1) /= 0 then do  
  say time('l') 'initialization problem rc ==> ' rcc  
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)  
  say "                                     ==> "msgerr  
  exit 8  
end  
/* Connection */  
rcc = RXMQCONN(qmgr)  
say "CONN =" rcc  
/* OPEN */  
/* OPEN */  
iod1.OT = MQOT_Q  
iod1.ON = qname  
iod1.OM = ''  
oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF QUIESCING  
rcc = RXMQOPEN('iod1.', oo1, 'h1', 'ood1.')  
say "OPEN =" rcc " h1=" h1  
/* MH source */  
/* cmho1.OPT = MQCMHO_NONE */  
/* cmho1.OPT = MQCMHO_NO_VALIDATION */  
cmho1.OPT = MQCMHO_VALIDATE  
rcc = RXMQMH('mh1','cmho1.')  
say "MH1 =" rcc mh1  
/* First property: usr.color = red */  
smpo1.OPT = MQSMPO_NONE  
pd1.OPT = MQPD_NONE  
pd1.SUPPORT = MQPD_SUPPORT_OPTIONAL  
pd1.CONTEXT = MQPD_NO_CONTEXT  
pd1.COPYOPTIONS = MQCOPY_DEFAULT  
rcc = RXMQSMP('mh1','smpo1.','usr.color','pd1.',MQTYPE_STRING,'red')  
say "SMP usr.color =" rcc  
if word(rcc,1) <> 0 then  
  signal cleanup_mh1  
/* Second property: usr.country = France */  
rcc = RXMQSMP('mh1','smpo1.','usr.country','pd1.',MQTYPE_STRING,'France')  
say "SMP usr.country =" rcc  
if word(rcc,1) <> 0 then signal cleanup_mh1  
/* Simple application data */  
d.0 = 5  
d.1 = 'hello'  
/* MQMD PUT */
```

```

imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING
imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF
/* PMO: the important field is NMH = mh1 */
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF QUIESCING + MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED
ipmo2.OMH = 0
ipmo2.NMH = mh1
ipmo2.ACT = MQACTP_NEW
ipmo2.PL = 0
rcc = RXMQPUT(h1, 'd.', 'imd2.', 'omd2.', 'ipmo2.', 'opmo2.')
say "PUT =" rcc
if word(rcc,1) <> 0 then signal cleanup_mh1
/* MH1 can be deleted after the PUT */
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1', 'dmho1.')
say "DMH mh1 =" rcc
/* MH2: handle that receives the properties during GET */
cmho2.OPT = MQCMHO_NONE
rcc = RXMQMH('mh2', 'cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then signal cleanup_close
/* Prepare the GET */
msg2.0 = 8192
igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE
/* GMO: the important field is MH = mh2 */
igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT + MQGMO_FAIL_IF QUIESCING
igmo2.WAIT = 5000
igmo2.MOPT = MQMO_NONE
igmo2.MH = mh2
rcc = RXMQGET(h1, 'msg2.', 'igmd2.', 'ogmd2.', 'igmo2.', 'ogmo2.')
say "GET =" rcc
say "msg2.0 =" msg2.0
say "msg2.1 length =" length(msg2.1)
say "msg2.1 =" msg2.1
if word(rcc,1) <> 0 then signal cleanup_mh2
/* IMP: inquire the property directly from mh2 */
/* Enumerate all usr.* properties */
impo1.OPT = MQIMPO_INQ_FIRST
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
pname = 'usr.%'
say "Enumerating properties matching" pname
do until RXMQ.LASTAC = MQRC_PROPERTY_NOT_AVAILABLE /* end property */
  pvalue.1 = ''
  pvalue.0 = 4096
  pnameout = ''
  ptype = ''
  rcc = RXMQIMP('mh2', 'impo1.', pname, 'pdout.', 'ptype', 'pvalue.', 'pnameout')
  if word(rcc,1) <> 0 then
    leave
  say "Name =" pnameout
  say "Type =" ptype
  say "Value =" pvalue.1
  say "rcc=" rcc
  say "-----"
  impo1.OPT = MQIMPO_INQ_NEXT
end
say "End of enumeration"
/* Normal cleanup */
cleanup_mh2:
  dmho2.OPT = MQDMHO_NONE
  rcc = RXMQDMH('mh2', 'dmho2.')
  say "DMH mh2 =" rcc
cleanup_close:
  rcc = RXMQCLOS(h1,0)
  say "CLOSE =" rcc
cleanup_disc:

```

```
rcc = RXMQDISC()
say "DISC =" rcc
exit 0
cleanup_mh1:
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1', 'dmho1.')
say "DMH mh1 =" rcc
signal cleanup_close
```

Discussion

1. The first inquiry uses MQIMPO_INQ_FIRST and every subsequent inquiry uses MQIMPO_INQ_NEXT.
2. The wildcard name usr.% selects the user properties created by the sample.
3. MQRC_PROPERTY_NOT_AVAILABLE is the expected terminator after the last matching property.
4. The returned property name normally omits the usr. prefix for this enumeration form.

Expected result

Two properties are enumerated.

The returned names identify color and country.

The returned values are red and France.

Enumeration ends with MQRC_PROPERTY_NOT_AVAILABLE.

HANDLE3 - Convert properties to MQRFH2 and reconstruct them

Purpose

Demonstrates explicit conversion between a message handle and an MQRFH2 representation. RXMQMBF serializes selected properties into a buffer and RXMQBMH reconstructs those properties in another message handle.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQMBF, RXMQBMH, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

```
RXMQMh(mh1) -> RXMQSMP -> RXMQMBF -> MQRFH2 buffer  
RXMQMH(mh2) -> RXMQBMH -> RXMQIMP enumeration
```

Complete Rexx sample

```
/* REXX */  
/*  
RXMQVTRACE = 'CONN OPEN PUT GET CLOSE DISC MH SMP MBF BMH IMP DMH'  
*/  
TRACE 0  
call RXMQCONS  
qmgr = 'QM1'  
qname = 'DEV.QUEUE.1'  
say "QMGR =" qmgr  
say "QUEUE=" qname  
/* ----- */  
/* 0. Initialization */  
/* ----- */  
rcc = rxmqinit()  
if word(rcc,1) /= 0 then do  
  say time('l') 'initialization problem rc ==> ' rcc  
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)  
  say "                               ==> "msgerr  
  exit 8  
end  
/* Connection */  
rcc = RXMQCONN(qmgr)  
say "CONN =" rcc  
/* OPEN */  
/* OPEN */  
iod1.OT = MQOT_Q  
iod1.ON = qname  
iod1.OM = ''  
oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF_QUIESCING  
rcc = RXMQOPEN('iod1.', oo1, 'h1', 'ood1.')  
say "OPEN =" rcc " h1=" h1  
/* MH source */  
/* cmho1.OPT = MQCMHO_NONE */  
/* cmho1.OPT = MQCMHO_NO_VALIDATION */  
cmho1.OPT = MQCMHO_VALIDATE  
rcc = RXMQMH('mh1','cmho1.')  
say "MH1 =" rcc mh1  
/* Property usr.color = red */  
smpo1.OPT = MQSMP0_NONE  
pd1.OPT = MQPD_NONE  
pd1.SUPPORT = MQPD_SUPPORT_OPTIONAL  
pd1.CONTEXT = MQPD_NO_CONTEXT  
pd1.COPYOPTIONS = MQCOPY_DEFAULT  
/* Property 1 */  
rcc = RXMQSMP('mh1','smpo1.','usr.color','pd1.',MQTYPE_STRING,'red')  
if word(rcc,1) <> 0 then signal cleanup_mh1  
say "SMP color =" rcc  
/* Property 2 */  
rcc = RXMQSMP('mh1','smpo1.','usr.shape','pd1.',MQTYPE_STRING,'square')  
if word(rcc,1) <> 0 then signal cleanup_mh1  
say "SMP shape =" rcc  
/* MH -> buffer RFH2 */
```

```

mhbo1.OPT = MQMHBO_NONE
md1.VER = MQMD_VERSION_2
md1.REP = MQRO_NONE
md1.MSG = MQMT_DATAGRAM
md1.EXP = MQEI_UNLIMITED
md1.FBK = MQFB_NONE
md1.ENC = MQENC_NATIVE
md1.CCSI = MQCCSI_Q_MGR
md1.FORM = MQFMT_STRING
md1.PRI = MQPRI_PRIORITY_AS_Q_DEF
md1.PER = MQPER_PERSISTENCE_AS_Q_DEF
rcc = RXMQMBF('mh1', 'mhbo1.', 'usr.%', 'md1.', 'buf', 8192, 'blen')
say "MBF =" rcc
say "MD.CCSI =" md1.CCSI
say "MD.ENC =" md1.ENC
say "MD.FORM =" md1.FORM
say "buf length =" length(buf)
say "blen =" blen
say "buf hex =" c2x(substr(buf,1,64))
if word(rcc,1) <> 0 then signal cleanup_mh1
/* Create the handle that receives the properties from the buffer */
cmho2.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh2', 'cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then signal cleanup_mh1
/* Convert the MQRFH2 buffer to message handle mh2 */
bmho1.OPT = MQBMHO_NONE
rcc = RXMQBMH('mh2', 'bmho1.', 'md1.', buf)
say "BMH =" rcc
if word(rcc,1) <> 0 then signal cleanup_mh2
/* Inquire the properties reconstructed in mh2 */
impo1.OPT = MQIMPO_INQ_FIRST + MQIMPO_CONVERT_VALUE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
pname = 'usr.%'
do until RXMQ.LASTAC = MQRC_PROPERTY_NOT_AVAILABLE /* end property */
  pvalue.1 = ''
  pvalue.0 = 4096
  pnameout = ''
  ptype = ''
rcc = RXMQIMP('mh2', 'impo1.', pname, 'pdout.', 'ptype', 'pvalue.', 'pnameout')
  if word(rcc,1) <> 0 then
    leave
  say "Name =" pnameout
  say "Type =" ptype
  say "Longueur =" pvalue.0
  say "Value =" pvalue.1
  say "rcc =" rcc
  say "-----"
  impo1.OPT = MQIMPO_INQ_NEXT + MQIMPO_CONVERT_VALUE
end
/* Normal cleanup */
cleanup_close:
  rcc = RXMQCLOS(h1,0)
  say "CLOSE =" rcc
cleanup_disc:
  rcc = RXMQDISC()
  say "DISC =" rcc
exit 0
cleanup_mh1:
  dmho1.OPT = MQDMHO_NONE
  rcc = RXMQDMH('mh1', 'dmho1.')
  say "DMH mh1 =" rcc
  signal cleanup_close
cleanup_mh2:
  dmho2.OPT = MQDMHO_NONE
  rcc = RXMQDMH('mh2', 'dmho2.')
  say "DMH mh2 =" rcc
  signal cleanup_close

```

Discussion

1. RXMQMBF calls MQMHBUF and returns the generated bytes and DataLength.
2. The returned MQMD identifies the generated MQRFH2 representation.
3. RXMQBMH calls MQBUFMH to rebuild properties in the destination handle.
4. MQIMPO_CONVERT_VALUE requests conversion to the requested encoding and CCSID during

inquiry.

Expected result

RXMQMBF and RXMQBMH complete successfully.

The returned MQMD format identifies MQRFH2.

usr.color=red and usr.shape=square are reconstructed in mh2.

HANDLE4 - Use several IBM MQ property data types

Purpose

Creates six properties with different MQTYPE_* values, transports them through a normal put/get cycle, and verifies both the returned type and value for every property.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

SET STRING, INT32, INT64, BOOLEAN, FLOAT64, and FLOAT32 properties in mh1
RXMQPUT(NewMsgHandle=mh1) -> RXMQGET(MsgHandle=mh2) -> verify all six properties

Complete Rexx sample

```
/* REXX */
/*
 * HANDLE4 - IBM MQ properties with multiple data types
 *
 * This sample deliberately follows the structure of HANDLE1:
 * CONN -> OPEN -> create MH1 -> SETMP -> PUT
 * -> create MH2 -> GET -> INQMP -> cleanup
 *
 * Properties tested:
 * usr.color      MQTYPE_STRING
 * usr.quantity   MQTYPE_INT32
 * usr.grandnombre MQTYPE_INT64
 * usr.prioritaire MQTYPE_BOOLEAN
 * usr.amount     MQTYPE_FLOAT64
 * usr.temperature MQTYPE_FLOAT32
 *
 * RXMQVTRACE = 'CONN OPEN PUT GET CLOSE DISC MH SMP IMP DMH'
 */
TRACE 0

qmgr = 'QM1'
qname = 'DEV.QUEUE.1'

mh1cree = 0
mh2cree = 0
ouvert = 0
connecte = 0
rcode = 0

say "QMGR =" qmgr
say "QUEUE=" qname
say ""
/* ----- */
/* 0. Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
  say time('l') 'initialization problem rc ==> ' rcc
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
  say "                               ==> "msgerr
  exit 8
end

/* ----- */
/* Connection */
/* ----- */
rcc = RXMQCONN(qmgr)
say "CONN =" rcc
if word(rcc,1) <> 0 then do
  rcode = 8
  signal cleanup
end
connecte = 1

/* ----- */
```



```

/* Open the queue for input and output */
/* ----- */
iod1.OT = MQOT_Q
iod1.ON = qname
iod1.OM = ''

oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF QUIESCING

rcc = RXMQOPEN('iod1.',oo1,'h1','ood1.')
say "OPEN =" rcc " h1=" h1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
ouvert = 1

/* ----- */
/* Create the source message handle */
/* ----- */
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1','cmho1.')
say "MH1 =" rcc mh1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 1

/* Common descriptor for the usr.* application properties */
smo1.OPT = MQSMPO_NONE
pd1.OPT = MQPD_NONE
pd1.SUPPORT = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT = MQPD_NO_CONTEXT
pd1.COPYOPTIONS = MQCOPY_DEFAULT

/* ----- */
/* 1. STRING property */
/* ----- */
rcc = RXMQSMP('mh1','smo1.','usr.color','pd1.',MQTYPE_STRING,'red')
say "SMP usr.color STRING red =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* ----- */
/* 2. INT32 property */
/* ----- */
rcc = RXMQSMP('mh1','smo1.','usr.quantity','pd1.',MQTYPE_INT32,25)
say "SMP usr.quantity INT32 25 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* ----- */
/* 3. INT64 property */
/* ----- */
rcc = RXMQSMP('mh1','smo1.','usr.grandnombre','pd1.',MQTYPE_INT64,5000000000)
say "SMP usr.grandnombre INT64 5000000000 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* ----- */
/* 4. BOOLEAN property */
/* MQBOOL: 0 = FALSE, any other value = TRUE */
/* ----- */
rcc = RXMQSMP('mh1','smo1.','usr.prioritaire','pd1.',MQTYPE_BOOLEAN,1)
say "SMP usr.prioritaire BOOLEAN TRUE(1) =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* ----- */
/* 5. FLOAT64 property */
/* ----- */
rcc = RXMQSMP('mh1','smo1.','usr.amount','pd1.',MQTYPE_FLOAT64,123.45)
say "SMP usr.amount FLOAT64 123.45 =" rcc
if word(rcc,1) <> 0 then do

```

```

        rcode = 8
        signal cleanup
    end

/* ----- */
/* 6. FLOAT32 property */
/* ----- */
rcc=RXMQSMP('mh1','smpo1.','usr.temperature','pd1.',MQTYPE_FLOAT32,12.5)
say "SMP usr.temperature FLOAT32 12.5 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* ----- */
/* Message body */
/* ----- */
d.1 = 'Test command: color=red quantity=25 amount=123.45'
d.0 = length(d.1)

/* MQMD for the PUT */
imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING
imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF

/* PMO: properties are supplied by handle mh1 */
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT+MQPMO_FAIL_IF QUIESCING+MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED
ipmo2.OMH = 0
ipmo2.NMH = mh1
ipmo2.ACT = MQACTP_NEW
ipmo2.PL = 0

rcc = RXMQPUT(h1,'d.','imd2.','omd2.','ipmo2.','opmo2.')
say "PUT =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* The source handle is no longer required after the PUT */
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1','dmho1.')
say "DMH mh1 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 0

/* ----- */
/* Create the receiving handle */
/* ----- */
cmho2.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh2','cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh2cree = 1

/* GET: message properties are placed in mh2 */
msg2.0 = 8192

igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE

igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT +,

```

```

MQGMO_FAIL_IF QUIESCING
igmo2.WAIT = 5000
igmo2.MOPT = MQGMO_NONE
igmo2.MH = mh2

rcc = RXMQGET(h1,'msg2.','igmd2.','ogmd2.','igmo2.','ogmo2.')
say "GET =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

say "Received body =" msg2.1
say ""

/* ----- */
/* Read and verify the six properties individually */
/* ----- */
impo1.OPT = MQIMPO_NONE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL

call verifprop 'usr.color',MQTYPE_STRING,'red'
call verifprop 'usr.quantity',MQTYPE_INT32,'25'
call verifprop 'usr.grandnombre',MQTYPE_INT64,'5000000000'
call verifprop 'usr.prioritaire',MQTYPE_BOOLEAN,'1'
call verifprop 'usr.amount',MQTYPE_FLOAT64,'123.45'
call verifprop 'usr.temperature',MQTYPE_FLOAT32,'12.5'

if rcode = 0 then do
    say ""
    say "=====
    say "TEST PASSED: all six types were preserved"
    say "=====
end
else do
    say ""
    say "=====
    say "TEST FAILED: at least one property is incorrect"
    say "=====
end

signal cleanup

/* ----- */
/* Inquire a property and verify its type and value */
/* ----- */
verifprop:
    parse arg nomatt,typeatt,valatt

    pvalue.0 = 4096
    pvalue.1 = ''
    pnameout = ''
    ptype = ''

rcc = RXMQIMP('mh2','impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')

say "IMP" nomatt "=" rcc

if word(rcc,1) <> 0 then do
    say "FAILED: property inquiry failed"
    rcode = 8
    return
end

say " name      =" pnameout
say " type      =" ptype "(expected" typeatt")"
say " longueur =" pvalue.0
say " value     =" pvalue.1 "(expected" valatt)"

if ptype <> typeatt then do
    say " FAILED: unexpected type"
    rcode = 8
end
else if pvalue.1 <> valatt then do
if typeatt = MQTYPE_FLOAT32 ! typeatt = MQTYPE_FLOAT64 then do
if pvalue.1 = valatt + 0 then
    say " OK: numerically equivalent value"
else do
    say " FAILED: unexpected value"

```

```

        rcode = 8
    end
end
else do
    if pvalue.1 = valatt then
        say " OK"
    else do
        say " FAILED: unexpected value"
        rcode = 8
    end
end
end
else
    say " OK"

    say "-----"
return

/* ----- */
/* Single cleanup path, including error cases */
/* ----- */
cleanup:
    if mh2cree = 1 then do
        dmho2.OPT = MQDMHO_NONE
        rcc2 = RXMQDMH('mh2', 'dmho2.')
        say "DMH mh2 =" rcc2
        mh2cree = 0
    end

    if mh1cree = 1 then do
        dmho1.OPT = MQDMHO_NONE
        rcc2 = RXMQDMH('mh1', 'dmho1.')
        say "DMH mh1 =" rcc2
        mh1cree = 0
    end

    if ouvert = 1 then do
        rcc2 = RXMQCLOS(h1,0)
        say "CLOSE =" rcc2
        ouvert = 0
    end

    if connecte = 1 then do
        rcc2 = RXMQDISC()
        say "DISC =" rcc2
        connecte = 0
    end

    exit rcode

```

Discussion

1. The same MQPD descriptor is reused for all application properties.
2. The sample validates MQTYPE_STRING, MQTYPE_INT32, MQTYPE_INT64, MQTYPE_BOOLEAN, MQTYPE_FLOAT64, and MQTYPE_FLOAT32.
3. Numeric values are checked by numeric equivalence so that harmless textual formatting differences do not cause a false failure.
4. A single cleanup path deletes any handles that were created before an error occurred.

Expected result

All six properties are returned with the expected MQTYPE_* value.

All six values are preserved.

The program reports TEST PASSED when every comparison succeeds.

HANDLE5 - Forward a message and copy properties with MQACTP_FORWARD

Purpose

Demonstrates how MQPD.CopyOptions controls property propagation when a message is forwarded with MQPMO.Action set to MQACTP_FORWARD and both OriginalMsgHandle and NewMsgHandle are supplied.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

Create properties using MQCOPY_DEFAULT, FORWARD, REPLY, FORWARD|REPLY, ALL, and NONE
GET original into mh2 -> PUT with OMH=mh2, NMH=mh3, ACT=MQACTP_FORWARD -> GET forwarded message into mh4

Complete Rexx sample

```
/* REXX */
/*
 * HANDLE5 - Copying properties with MQACTP_FORWARD
 *
 * Structure based on HANDLE4:
 *
 * CONN
 * -> OPEN
 * -> create MH1
 * -> SETMP
 * -> PUT initial
 * -> create MH2
 * -> GET initial
 * -> create MH3
 * -> PUT FORWARD with OMH=MH2 and NMH=MH3
 * -> create MH4
 * -> GET the forwarded message
 * -> INQMP
 * -> cleanup
 *
 * RXMQVTRACE = 'CONN OPEN PUT GET CLOSE DISC MH SMP IMP DMH'
 */
TRACE o
qmgr = 'QM1'
qname = 'DEV.QUEUE.1'
mh1cree = 0
mh2cree = 0
mh3cree = 0
mh4cree = 0
ouvert = 0
connecte = 0
rcode = 0
say "QMGR =" qmgr
say "QUEUE=" qname
say ""
/* ----- */
/* 0. Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
  say time('l') 'initialization problem rc ==> ' rcc
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
  say "                                     ==> "msgerr
  exit 8
end
/* ----- */
/* Connection */
/* ----- */
rcc = RXMQCONN(qmgr)
say "CONN =" rcc
if word(rcc,1) <> 0 then do
  rcode = 8
```

```

    signal cleanup
end
connecte = 1
/* ----- */
/* Open the queue for input and output */
/* ----- */
iod1.OT = MQOT_Q
iod1.ON = qname
iod1.OM = ''
oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF_QUIESCING
rcc = RXMQOPEN('iod1.',oo1,'h1','ood1.')
say "OPEN =" rcc " h1=" h1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
ouvert = 1
/* ----- */
/* Create source message handle MH1 */
/* ----- */
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1','cmho1.')
say "MH1 =" rcc mh1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 1
/* ----- */
/* Common property descriptor */
/* ----- */
smpo1.OPT = MQSMP0_NONE
pd1.OPT = MQPD_NONE
pd1.SUPPORT = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT = MQPD_NO_CONTEXT
/* ----- */
/* Property with MQCOPY_DEFAULT */
/* ----- */
pd1.COPYOPTIONS = MQCOPY_DEFAULT
rcc=RXMQSMP('mh1','smpo1.','usr.default','pd1.',MQTYPE_STRING,'blue')
say "SMP usr.default MQCOPY_DEFAULT =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Property with MQCOPY_FORWARD */
/* ----- */
pd1.COPYOPTIONS = MQCOPY_FORWARD
rcc=RXMQSMP('mh1','smpo1.','usr.forward','pd1.',MQTYPE_INT32,25)
say "SMP usr.forward MQCOPY_FORWARD =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Property with MQCOPY_REPLY */
/* ----- */
pd1.COPYOPTIONS = MQCOPY_REPLY
rcc=RXMQSMP('mh1','smpo1.','usr.reply','pd1.',MQTYPE_BOOLEAN,1)
say "SMP usr.reply MQCOPY_REPLY =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Property with MQCOPY_FORWARD and MQCOPY_REPLY */
/* ----- */
pd1.COPYOPTIONS = MQCOPY_FORWARD + MQCOPY_REPLY
rcc=RXMQSMP('mh1','smpo1.','usr.forwrep','pd1.',MQTYPE_FLOAT64,123.45)
say "SMP usr.forwrep FORWARD + REPLY =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Property with MQCOPY_ALL */
/* ----- */
pd1.COPYOPTIONS = MQCOPY_ALL
rcc=RXMQSMP('mh1','smpo1.','usr.toujours','pd1.',MQTYPE_INT64,5000000000)

```

```

say "SMP usr.toujours      MQCOPY_ALL =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Property with MQCOPY_NONE */
/* ----- */
pd1.COPYOPTIONS = MQCOPY_NONE
rcc=RXMQSMP('mh1','smp01.','usr.jamais','pd1.',MQTYPE_FLOAT32,12.5)
say "SMP usr.jamais      MQCOPY_NONE =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Initial message body */
/* ----- */
d.1 = 'Initial message for the MQACTP_FORWARD'
d.0 = length(d.1)
/* MQMD for the first PUT */
imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING
imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF
/* ----- */
/* First PUT: new message with the properties from MH1 */
/* ----- */
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF QUIESCING+MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED
ipmo2.OMH = 0
ipmo2.NMH = mh1
ipmo2.ACT = MQACTP_NEW
ipmo2.PL = 0
rcc = RXMQPUT(h1,'d.','imd2.','omd2.','ipmo2.','opmo2.')
say "PUT initial =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Delete MH1 */
/* ----- */
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1','dmho1.')
say "DMH mh1 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 0
/* ----- */
/* Create message handle MH2 for the first GET */
/* ----- */
cmho2.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh2','cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh2cree = 1
/* ----- */
/* GET the initial message */
/* ----- */
msg2.0 = 8192
igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE
igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT +,
            MQGMO_FAIL_IF QUIESCING

```

```

igmo2.WAIT = 5000
igmo2.MOPT = MQMO_NONE
igmo2.MH = mh2
rcc = RXMQGET(h1,'msg2.','igmd2.','ogmd2.','igmo2.','ogmo2.')
say "GET initial =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
say "Initial body =" msg2.1
say ""
/* ----- */
/* Verify properties present in MH2 */
/* before the PUT FORWARD */
/* ----- */
impo1.OPT = MQIMPO_NONE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
say "Verify properties before FORWARD"
say ""
handleinq = 'mh2'
call verifpresent 'usr.default',MQTYPE_STRING,'blue'
call verifpresent 'usr.forward',MQTYPE_INT32,'25'
call verifpresent 'usr.reply',MQTYPE_BOOLEAN,'1'
call verifpresent 'usr.forwrep',MQTYPE_FLOAT64,'123.45'
call verifpresent 'usr.toujours',MQTYPE_INT64,'5000000000'
call verifpresent 'usr.jamais',MQTYPE_FLOAT32,'12.5'
if rcode <> 0 then signal cleanup
/* ----- */
/* Create MH3: NewMsgHandle for the PUT FORWARD */
/* ----- */
cmho3.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh3','cmho3.')
say "MH3 =" rcc mh3
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh3cree = 1
/* ----- */
/* Second PUT with MQACTP_FORWARD */
/* ----- */
/* MH2 is the original message handle. */
/* MH3 is the new message handle. */
/* MQ copies properties from MH2 to MH3 according to MQPD.CopyOptions. */
/* ----- */
msgf.1 = msg2.1
msgf.0 = length(msgf.1)
imd3.VER = MQMD_VERSION_2
imd3.REP = MQRO_NONE
imd3.MSG = MQMT_DATAGRAM
imd3.EXP = MQEI_UNLIMITED
imd3.FBK = MQFB_NONE
imd3.ENC = MQENC_NATIVE
imd3.CCSI = MQCCSI_Q_MGR
imd3.FORM = MQFMT_STRING
imd3.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd3.PER = MQPER_PERSISTENCE_AS_Q_DEF
ipmo3.VER = MQPMO_VERSION_3
ipmo3.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF QUIESCING+MQPMO_DEFAULT_CONTEXT
ipmo3.TIME = MQWI_UNLIMITED
ipmo3.OMH = mh2
ipmo3.NMH = mh3
ipmo3.ACT = MQACTP_FORWARD
ipmo3.PL = 0
say ""
say "PUT FORWARD"
say " OriginalMsgHandle =" ipmo3.OMH
say " NewMsgHandle      =" ipmo3.NMH
say " Action            =" ipmo3.ACT
rcc = RXMQPUT(h1,'msgf.','imd3.','omd3.','ipmo3.','opmo3.')
say "PUT forward =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Delete the handles used by the PUT FORWARD */
/* ----- */
dmho2.OPT = MQDMHO_NONE

```



```

rcc = RXMQDMH('mh2','dmho2.')
say "DMH mh2 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh2cree = 0
dmho3.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh3','dmho3.')
say "DMH mh3 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh3cree = 0
/* ----- */
/* Create MH4 to receive the forwarded message */
/* ----- */
cmho4.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh4','cmho4.')
say "MH4 =" rcc mh4
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh4cree = 1
/* ----- */
/* GET the forwarded message */
/* ----- */
msg4.0 = 8192
igmd4.VER = MQMD_VERSION_2
igmd4.ENC = MQENC_NATIVE
igmd4.CCSI = MQCCSI_Q_MGR
igmd4.FORM = MQFMT_NONE
igmo4.VER = MQGMO_VERSION_4
igmo4.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT +,
           MQGMO_FAIL_IF_QUIESCING
igmo4.WAIT = 5000
igmo4.MOPT = MQMO_NONE
igmo4.MH = mh4
rcc = RXMQGET(h1,'msg4.','igmd4.','ogmd4.','igmo4.','ogmo4.')
say "GET forward =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
say "Forwarded body =" msg4.1
say ""
/* ----- */
/* Verify properties after MQACTP_FORWARD */
/* ----- */
say "Verify properties after MQACTP_FORWARD"
say ""
handleinq = 'mh4'
say "MQCOPY_DEFAULT : expected property is present"
call verifpresent 'usr.default',MQTYPE_STRING,'blue'
say "MQCOPY_FORWARD : expected property is present"
call verifpresent 'usr.forward',MQTYPE_INT32,'25'
say "MQCOPY_REPLY : expected property is absent"
call verifabsente 'usr.reply'
say "MQCOPY_FORWARD + MQCOPY_REPLY : expected property is present"
call verifpresent 'usr.forwrep',MQTYPE_FLOAT64,'123.45'
say "MQCOPY_ALL : expected property is present"
call verifpresent 'usr.toujours',MQTYPE_INT64,'5000000000'
say "MQCOPY_NONE : expected property is absent"
call verifabsente 'usr.jamais'
/* ----- */
/* Summary */
/* ----- */
say ""
if rcode = 0 then do
    say "=====
    say "TEST PASSED: MQACTP_FORWARD"
    say "=====
end
else do
    say "=====
    say "TEST FAILED: MQACTP_FORWARD"
    say "=====
end
end

```

```

signal cleanup
/* ----- */
/* Verify a property that is expected to be present */
/* ----- */
verifpresent:
  parse arg nomatt,typeatt,valatt
  pvalue.0 = 4096
  pvalue.1 = ''
  pnameout = ''
  ptype = ''
  rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
  say "IMP" nomatt "=" rcc
  if word(rcc,1) <> 0 then do
    say " FAILED: expected property is present"
    rcode = 8
    say "-----"
    return
  end
  say " name      =" pnameout
  say " type      =" ptype "(expected" typeatt")"
  say " longueur =" pvalue.0
  say " value     =" pvalue.1 "(expected" valatt)"
  if ptype <> typeatt then do
    say " FAILED: unexpected type"
    rcode = 8
  end
  else if pvalue.1 <> valatt then do
    if typeatt = MQTYPE_FLOAT32 then do
      if pvalue.1 = valatt + 0 then
        say " OK: numerically equivalent value"
      else do
        say " FAILED: unexpected value"
        rcode = 8
      end
    end
    else if typeatt = MQTYPE_FLOAT64 then do
      if pvalue.1 = valatt + 0 then
        say " OK: numerically equivalent value"
      else do
        say " FAILED: unexpected value"
        rcode = 8
      end
    end
    else do
      say " FAILED: unexpected value"
      rcode = 8
    end
  end
  say " OK"
  say "-----"
return
/* ----- */
/* Verify a property that is expected to be absent */
/* ----- */
verifabsente:
  parse arg nomatt
  pvalue.0 = 4096
  pvalue.1 = ''
  pnameout = ''
  ptype = ''
  rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
  say "IMP" nomatt "=" rcc
  if word(rcc,3) = MQRC_PROPERTY_NOT_AVAILABLE then do
    say " OK: property is absent"
  end
  else do
    if word(rcc,1) = 0 then do
      say " FAILED: property is present"
      say " type      =" ptype
      say " value     =" pvalue.1
    end
    else do
      say " FAILED: unexpected error"
      say " return =" rcc
    end
    rcode = 8
  end
  say "-----"
return

```

```

/* ----- */
/* Single cleanup path */
/* ----- */
cleanup:
  if mh4cree = 1 then do
    dmho4.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh4','dmho4.')
    say "DMH mh4 =" rcc2
    mh4cree = 0
  end
  if mh3cree = 1 then do
    dmho3.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh3','dmho3.')
    say "DMH mh3 =" rcc2
    mh3cree = 0
  end
  if mh2cree = 1 then do
    dmho2.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh2','dmho2.')
    say "DMH mh2 =" rcc2
    mh2cree = 0
  end
  if mh1cree = 1 then do
    dmho1.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh1','dmho1.')
    say "DMH mh1 =" rcc2
    mh1cree = 0
  end
  if ouvert = 1 then do
    rcc2 = RXMQCLOSE(h1,0)
    say "CLOSE =" rcc2
    ouvert = 0
  end
  if connecte = 1 then do
    rcc2 = RXMQDISC()
    say "DISC =" rcc2
    connecte = 0
  end
end
exit rcodes

```

Discussion

1. Before forwarding, every property is present in the original message handle.
2. MQACTP_FORWARD copies properties according to each property descriptor CopyOptions value.
3. MQCOPY_DEFAULT, MQCOPY_FORWARD, MQCOPY_FORWARD|MQCOPY_REPLY, and MQCOPY_ALL are expected to propagate.
4. MQCOPY_REPLY and MQCOPY_NONE are expected not to propagate during a forward operation.

Expected result

usr.default, usr.forward, usr.forwrep, and usr.toujours are present after forwarding.

usr.reply and usr.jamais are absent after forwarding.

The program reports TEST PASSED: MQACTP_FORWARD.

HANDLE6 - Delete a property with MQDLTMP

Purpose

Receives three properties into a message handle, deletes usr.id with RXMQDMP, and confirms that the other properties remain unchanged while usr.id is no longer available.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQDMP, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

Create and receive usr.color, usr.id, and usr.client in mh2
RXMQDMP(mh2, usr.id) -> verify usr.id absent and other properties present

Complete Rexx sample

```
/* REXX */
/*
 * HANDLE6 - Deleting a property with MQDLTMP
 */
* CONN
* -> OPEN
* -> create MH1
* -> SETMP three properties
* -> PUT initial
* -> delete MH1
* -> create MH2
* -> GET into MH2
* -> verify the three properties
* -> DLTMP usr.id
* -> verify the deletion
* -> cleanup
*/
TRACE 0
qmgr = 'QM1'
qname = 'DEV.QUEUE.1'
mh1cree = 0
mh2cree = 0
ouvert = 0
connecte = 0
rcode = 0
say "QMGR =" qmgr
say "QUEUE=" qname
say ""
/* ----- */
/* 0. Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
  say time('l') 'initialization problem rc ==> ' rcc
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
  say "                                     ==> "msgerr
  exit 8
end
/* ----- */
/* Connection */
/* ----- */
rcc = RXMQCONN(qmgr)
say "CONN =" rcc
if word(rcc,1) <> 0 then do
  rcode = 8
  signal cleanup
end
connecte = 1
/* ----- */
/* Open the queue for input and output */
/* ----- */
iod1.OT = MQOT_Q
iod1.ON = qname
iod1.OM = ''
oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF QUIESCING
```

```

rcc = RXMQOPEN('iod1.', 'oo1', 'h1', 'ood1.')
say "OPEN =" rcc " h1=" h1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
ouvert = 1
/* ----- */
/* Create message handle MH1 */
/* ----- */
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1', 'cmho1.')
say "MH1 =" rcc mh1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 1
/* ----- */
/* Define properties in MH1 */
/* ----- */
smo1.OPT      = MQSMPD_NONE
pd1.OPT      = MQPD_NONE
pd1.SUPPORT  = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT  = MQPD_NO_CONTEXT
pd1.COPYOPTIONS = MQCOPY_DEFAULT
rcc=RXMQSMP('mh1', 'smo1.', 'usr.color', 'pd1.', MQTYPE_STRING, 'blue')
say "SMP usr.color =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1', 'smo1.', 'usr.id', 'pd1.', MQTYPE_INT32, 1234)
say "SMP usr.id      =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1', 'smo1.', 'usr.client', 'pd1.', MQTYPE_STRING, 'IBM')
say "SMP usr.client =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Initial message body */
/* ----- */
d.1 = 'Initial message for the MQDLTMP'
d.0 = length(d.1)
/* ----- */
/* MQMD for the PUT */
/* ----- */
imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING
imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF
/* ----- */
/* PUT the message with the properties from MH1 */
/* ----- */
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF_QUIESCING+MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED
ipmo2.OMH = 0
ipmo2.NMH = mh1
ipmo2.ACT = MQACTP_NEW
ipmo2.PL = 0
rcc = RXMQPUT(h1, 'd.', 'imd2.', 'omd2.', 'ipmo2.', 'opmo2.')
say "PUT initial =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Delete MH1 */

```

```

/* ----- */
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1', 'dmho1.')
say "DMH mh1 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 0
/* ----- */
/* Create MH2 for the GET */
/* ----- */
cmho2.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh2', 'cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh2cree = 1
/* ----- */
/* GET the message */
/* ----- */
msg2.0 = 8192
igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE
igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT +,
            MQGMO_FAIL_IF_QUIESCING
igmo2.WAIT = 5000
igmo2.MOPT = MQMO_NONE
igmo2.MH = mh2
rcc = RXMQGET(h1, 'msg2.', 'igmd2.', 'ogmd2.', 'igmo2.', 'ogmo2.')
say "GET initial =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
say "Received body =" msg2.1
say ""
/* ----- */
/* MQINQMP options */
/* ----- */
impo1.OPT = MQIMPO_NONE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
handleinq = 'mh2'
/* ----- */
/* Verification before MQDLTMP */
/* ----- */
say "====="
say "PROPERTIES BEFORE MQDLTMP"
say "====="
say ""
call verifpresent 'usr.color', MQTYPE_STRING, 'blue'
call verifpresent 'usr.id', MQTYPE_INT32, '1234'
call verifpresent 'usr.client', MQTYPE_STRING, 'IBM'
if rcode <> 0 then signal cleanup
/* ----- */
/* Delete usr.id from MH2 */
/* ----- */
dmpo1.OPT = MQDMPO_NONE
say ""
say "====="
say "DELETE usr.id"
say "====="
rcc = RXMQDMP('mh2', 'dmpo1.', 'usr.id')
say "DMP usr.id =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Verification after MQDLTMP */
/* ----- */
say ""
say "====="
say "PROPERTIES AFTER MQDLTMP"

```

```

say "=====
say ""
call verifpresent 'usr.color',MQTYPE_STRING,'blue'
call verifabsente 'usr.id'
call verifpresent 'usr.client',MQTYPE_STRING,'IBM'
/* ----- */
/* Summary */
/* ----- */
say ""
if rcode = 0 then do
  say "=====
  say "TEST PASSED: MQDLTMP"
  say "=====
end
else do
  say "=====
  say "TEST FAILED: MQDLTMP"
  say "=====
end
signal cleanup
/* ----- */
/* Verify a property that is expected to be present */
/* ----- */
verifpresent:
  parse arg nomatt,typeatt,valatt
  pvalue.0 = 4096
  pvalue.1 = ''
  pnameout = ''
  ptype = ''
  rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
  say "Verifying" nomatt
  say " Expected result = PRESENT"
  say " IMP = " rcc
  if word(rcc,1) <> 0 then do
    say " Actual result = ABSENT OR ERROR"
    say " FAILED: expected property is present"
    rcode = 8
    say "-----"
    return
  end
  say " Returned name = " pnameout
  say " Returned type = " ptype
  say " Expected type = " typeatt
  say " Returned value = " pvalue.1
  say " Expected value = " valatt
  if ptype <> typeatt then do
    say " FAILED: unexpected type"
    rcode = 8
  end
  else if pvalue.1 <> valatt then do
    say " FAILED: unexpected value"
    rcode = 8
  end
  else
    say " OK: property is present"
    say "-----"
return
/* ----- */
/* Verify a property that is expected to be absent */
/* ----- */
verifabsente:
  parse arg nomatt
  pvalue.0 = 4096
  pvalue.1 = ''
  pnameout = ''
  ptype = ''
  rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
  say "Verifying" nomatt
  say " Expected result = ABSENT"
  say " IMP = " rcc
  if word(rcc,3) = MQRC_PROPERTY_NOT_AVAILABLE then do
    say " Actual result = ABSENT"
    say " OK: property was deleted"
  end
  else do
    if word(rcc,1) = 0 then do
      say " Actual result = PRESENT"
      say " Type = " ptype
      say " Value = " pvalue.1
      say " FAILED: property is still present"
    end
  end

```

```

        end
    else do
        say " Actual result  = UNEXPECTED ERROR"
        say " FAILED: unexpected return"
    end
    rcode = 8
end
say "-----"
return
/* ----- */
/* Single cleanup path */
/* ----- */
cleanup:
    if mh2cree = 1 then do
        dmho2.OPT = MQDMHO_NONE
        rcc2 = RXMQDMH('mh2','dmho2.')
        say "DMH mh2 =" rcc2
        mh2cree = 0
    end
    if mh1cree = 1 then do
        dmho1.OPT = MQDMHO_NONE
        rcc2 = RXMQDMH('mh1','dmho1.')
        say "DMH mh1 =" rcc2
        mh1cree = 0
    end
    if ouvert = 1 then do
        rcc2 = RXMQCLOS(h1,0)
        say "CLOSE =" rcc2
        ouvert = 0
    end
    if connecte = 1 then do
        rcc2 = RXMQDISC()
        say "DISC =" rcc2
        connecte = 0
    end
end
exit rcode

```

Discussion

1. The property is deleted from the message handle, not from a queue object or a previously stored message.
2. The sample verifies all three properties before deletion to establish the initial state.
3. After RXMQDMP, MQRC_PROPERTY_NOT_AVAILABLE is the expected result for usr.id.
4. The surrounding properties are queried again to demonstrate that deletion is selective.

Expected result

RXMQDMP completes successfully.

usr.id is absent after deletion.

usr.color and usr.client remain present with their original values.

The program reports TEST PASSED: MQDLTMP.

HANDLE7 - Enumerate properties with MQINQMP

Purpose

Creates five properties, receives them into a second handle, and enumerates the complete property set with the % wildcard and the MQIMPO_INQ_FIRST/MQIMPO_INQ_NEXT sequence.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

Create five properties -> PUT -> GET into mh2
RXMQIMP(%, INQ_FIRST) -> repeated RXMQIMP(%, INQ_NEXT) -> count results

Complete Rexx sample

```
/* REXX */
/*
 * HANDLE7 - Enumerating properties with MQINQMP
 *
 * CONN
 * -> OPEN
 * -> create MH1
 * -> SETMP several properties
 * -> PUT initial
 * -> delete MH1
 * -> create MH2
 * -> GET into MH2
 * -> enumerate with %
 * -> MQIMPO_INQ_FIRST
 * -> MQIMPO_INQ_NEXT
 * -> cleanup
 */
TRACE 0
qmgr = 'QM1'
qname = 'DEV.QUEUE.1'
mh1cree = 0
mh2cree = 0
ouvert = 0
connecte = 0
rcode = 0
say "QMGR =" qmgr
say "QUEUE=" qname
say ""
/* ----- */
/* 0. Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
  say time('l') 'initialization problem rc ==> ' rcc
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
  say "                                     ==> "msgerr
  exit 8
end
/* ----- */
/* Connection */
/* ----- */
rcc = RXMQCONN(qmgr)
say "CONN =" rcc
if word(rcc,1) <> 0 then do
  rcode = 8
  signal cleanup
end
connecte = 1
/* ----- */
/* Open the queue for input and output */
/* ----- */
iod1.OT = MQOT_Q
iod1.ON = qname
iod1.OM = ''
oo1 = MQOO_OUTPUT + MQOO_INPUT_SHARED + MQOO_FAIL_IF QUIESCING
```

```

rcc = RXMQOPEN('iod1.', 'oo1', 'h1', 'ood1.')
say "OPEN =" rcc " h1=" h1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
ouvert = 1
/* ----- */
/* Create message handle MH1 */
/* ----- */
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1', 'cmho1.')
say "MH1 =" rcc mh1
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 1
/* ----- */
/* Define properties in MH1 */
/* ----- */
smo1.OPT      = MQSMO_NONE
pd1.OPT      = MQPD_NONE
pd1.SUPPORT  = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT  = MQPD_NO_CONTEXT
pd1.COPYOPTIONS = MQCOPY_DEFAULT
rcc=RXMQSMP('mh1', 'smo1.', 'usr.color', 'pd1.', MQTYPE_STRING, 'blue')
say "SMP usr.color =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1', 'smo1.', 'usr.quantity', 'pd1.', MQTYPE_INT32, 25)
say "SMP usr.quantity =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1', 'smo1.', 'usr.prioritaire', 'pd1.', MQTYPE_BOOLEAN, 1)
say "SMP usr.prioritaire=" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1', 'smo1.', 'usr.amount', 'pd1.', MQTYPE_FLOAT64, 123.45)
say "SMP usr.amount =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1', 'smo1.', 'usr.client', 'pd1.', MQTYPE_STRING, 'IBM')
say "SMP usr.client =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Initial message body */
/* ----- */
d.1 = 'Initial message for the enumeration MQINQMP'
d.0 = length(d.1)
/* ----- */
/* MQMD for the PUT */
/* ----- */
imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING
imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF
/* ----- */
/* PUT the message with the properties from MH1 */
/* ----- */
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF_QUIESCING+MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED

```

```

ipmo2.OMH      = 0
ipmo2.NMH      = mh1
ipmo2.ACT      = MQACTP_NEW
ipmo2.PL       = 0
rcc = RXMQPUT(h1,'d.','imd2.','omd2.','ipmo2.','opmo2.')
say "PUT initial =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Delete MH1 */
/* ----- */
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1','dmho1.')
say "DMH mh1 =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh1cree = 0
/* ----- */
/* Create MH2 for the GET */
/* ----- */
cmho2.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh2','cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh2cree = 1
/* ----- */
/* GET the message */
/* ----- */
msg2.0 = 8192
igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE
igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT +,
           MQGMO_FAIL_IF QUIESCING
igmo2.WAIT = 5000
igmo2.MOPT = MQMO_NONE
igmo2.MH   = mh2
rcc = RXMQGET(h1,'msg2.','igmd2.','ogmd2.','igmo2.','ogmo2.')
say "GET initial =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
say "Received body =" msg2.1
say ""
/* ----- */
/* Enumerate properties with % */
/* ----- */
say "====="
say "ENUMERATE PROPERTIES %"
say "====="
say ""
pnam = '%'
nbprop = 0
impo1.OPT = MQIMPO_INQ_FIRST
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
do forever
    pvalue.0 = 4096
    pvalue.1 = ''
    pnameout = ''
    ptype    = ''
    rcc = RXMQIMP('mh2','impo1.',pnam,'pdout.','ptype','pvalue.','pnameout')
    if word(rcc,1) = 0 then do
        nbprop = nbprop + 1
        say "Property number" nbprop
        say "  Name    =" pnameout
        say "  Type   =" ptype
        say "  Longueur =" pvalue.0
        say "  Value  =" pvalue.1
        say "-----"
    end
end

```

```

        impo1.OPT = MQIMPO_INQ_NEXT
    end
else do
    if word(rcc,3) = MQRC_PROPERTY_NOT_AVAILABLE then do
        leave
    end
    else do
        say "Error during enumeration"
        say "  IMP =" rcc
        rcode = 8
        leave
    end
end
end
/* ----- */
/* Summary */
/* ----- */
say ""
say "Number of % properties found =" nbprop
say ""
if rcode = 0 then do
    if nbprop = 5 then do
        say "=====
        say "TEST PASSED: MQINQMP ENUMERATION"
        say "=====
    end
    else do
        say "=====
        say "TEST FAILED: UNEXPECTED PROPERTY COUNT"
        say "Expected =" 5
        say "Actual  =" nbprop
        say "=====
        rcode = 8
    end
end
else do
    say "=====
    say "TEST FAILED: MQINQMP ENUMERATION"
    say "=====
end
signal cleanup
/* ----- */
/* Single cleanup path */
/* ----- */
cleanup:
    if mh2cree = 1 then do
        dmho2.OPT = MQDMHO_NONE
        rcc2 = RXMQDMH('mh2','dmho2.')
        say "DMH mh2 =" rcc2
        mh2cree = 0
    end
    if mh1cree = 1 then do
        dmho1.OPT = MQDMHO_NONE
        rcc2 = RXMQDMH('mh1','dmho1.')
        say "DMH mh1 =" rcc2
        mh1cree = 0
    end
    if ouvert = 1 then do
        rcc2 = RXMQCLOS(h1,0)
        say "CLOSE =" rcc2
        ouvert = 0
    end
    if connecte = 1 then do
        rcc2 = RXMQDISC()
        say "DISC =" rcc2
        connecte = 0
    end
end
exit rcode

```

Discussion

1. The % pattern enumerates all properties visible in the handle.
2. value.0 is reset before every inquiry because it supplies the maximum value-buffer length.
3. The loop treats MQRC_PROPERTY_NOT_AVAILABLE as normal completion and any other nonzero result as an error.
4. The final count provides an additional validation that no property was skipped or duplicated.

Expected result

Exactly five properties are enumerated.

Each property name, type, DataLength, and value is displayed.

The program reports TEST PASSED: MQINQMP ENUMERATION.

HANDLE8 - Convert a message handle to and from an MQRFH2 buffer

Purpose

Performs a complete MQMHBUFF/MQBUFMH round trip for five typed properties without putting a message to a queue. It verifies that the properties reconstructed in the destination handle retain their type and value.

APIs used

RXMQINIT, RXMQCONN, RXMQMH, RXMQSMP, RXMQMBF, RXMQBMH, RXMQIMP, RXMQDMH, and RXMQDISC.

Execution flow

Create typed properties in mh1 -> RXMQMBF -> MQRFH2 buffer
Create mh2 -> RXMQBMH(buffer) -> verify reconstructed properties

Complete Rexx sample

```
/* REXX */
/*
 * HANDLE8 - Message handle / MQRFH2 buffer conversion
 *
 * CONN
 * -> create MH1
 * -> SETMP several properties in MH1
 * -> MQMHBUFF: MH1 to MQRFH2 buffer
 * -> create MH2
 * -> MQBUFMH: MQRFH2 buffer to MH2
 * -> verify properties reconstructed in MH2
 * -> cleanup
 */
TRACE 0
qmgr = 'QM1'
mh1cree = 0
mh2cree = 0
connecte = 0
rcode = 0
say "QMGR =" qmgr
say ""
/* ----- */
/* 0. Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
  say time('l') 'initialization problem rc ==> ' rcc
  interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
  say "                                     ==> "msgerr
  exit 8
end
/* ----- */
/* Connection */
/* ----- */
rcc = rxmqconn(qmgr)
say "CONN =" rcc
if word(rcc,1) <> 0 then do
  rcode = 8
  signal cleanup
end
connecte = 1
/* ----- */
/* Create message handle MH1 */
/* ----- */
cmho1.OPT = MQCMHO_VALIDATE
rcc = rxmqmh('mh1','cmho1.')
say "MH1 =" rcc mh1
if word(rcc,1) <> 0 then do
  rcode = 8
  signal cleanup
end
mh1cree = 1
```

```

/* ----- */
/* Define properties in MH1 */
/* ----- */
smpt1.OPT      = MQSMPT_NONE
pd1.OPT        = MQPD_NONE
pd1.SUPPORT    = MQPD_SUPPORT_OPTIONAL
pd1.CONTEXT    = MQPD_NO_CONTEXT
pd1.COPYOPTIONS = MQCOPY_DEFAULT
rcc=RXMQSMP('mh1','smpt1.','usr.color','pd1.',MQTYPE_STRING,'blue')
say "SMP usr.color      =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1','smpt1.','usr.quantity','pd1.',MQTYPE_INT32,25)
say "SMP usr.quantity  =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1','smpt1.','usr.prioritaire','pd1.',MQTYPE_BOOLEAN,1)
say "SMP usr.prioritaire =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1','smpt1.','usr.amount','pd1.',MQTYPE_FLOAT64,123.45)
say "SMP usr.amount    =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
rcc=RXMQSMP('mh1','smpt1.','usr.client','pd1.',MQTYPE_STRING,'IBM')
say "SMP usr.client     =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* Convert message handle MH1 to an MQRFH2 buffer */
/* ----- */
mhbo1.OPT = MQMHBO_NONE
md1.VER   = MQMD_VERSION_2
md1.REP   = MQRO_NONE
md1.MSG   = MQMT_DATAGRAM
md1.EXP   = MQEI_UNLIMITED
md1.FBK   = MQFB_NONE
md1.ENC   = MQENC_NATIVE
md1.CCSI  = MQCCSI_Q_MGR
md1.FORM  = MQFMT_STRING
md1.PRI   = MQPRI_PRIORITY_AS_Q_DEF
md1.PER   = MQPER_PERSISTENCE_AS_Q_DEF
pnam = '%'
buf  = ''
blen = 0
say ""
say "=====
say "MQMHBUF: MH1 TO MQRFH2 BUFFER"
say "=====
rcc = RXMQMBF('mh1','mhbo1.',pnam,'md1.','buf',8192,'blen')
say "MBF =" rcc
say "MD.CCSI  =" md1.CCSI
say "MD.ENC   =" md1.ENC
say "MD.FORM  =" md1.FORM
say "buf length=" length(buf)
say "blen     =" blen
say "buf hex   =" c2x(substr(buf,1,64))
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
if blen = 0 then do
    say "FAILED: no MQRFH2 buffer returned"
    rcode = 8
    signal cleanup
end
/* ----- */
/* Create message handle MH2 */
/* ----- */
cmho2.OPT = MQCMHO_VALIDATE

```

```

rcc = RXMQMH('mh2','cmho2.')
say ""
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
mh2cree = 1
/* ----- */
/* Convert the MQRFH2 buffer to message handle MH2 */
/* ----- */
bmho1.OPT = MQBMHO_NONE
say ""
say "=====
say "MQBUFMH: MQRFH2 BUFFER TO MH2"
say "=====
rcc = RXMQBMH('mh2','bmho1.','md1.',buf)
say "BMH =" rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
/* ----- */
/* MQINQMP options */
/* ----- */
impo1.OPT = MQIMPO_NONE + MQIMPO_CONVERT_VALUE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
handleinq = 'mh2'
/* ----- */
/* Verify properties reconstructed in MH2 */
/* ----- */
say ""
say "=====
say "VERIFY PROPERTIES IN MH2"
say "=====
say ""
call verifprop 'usr.color',MQTYPE_STRING,'blue'
call verifprop 'usr.quantity',MQTYPE_INT32,'25'
call verifprop 'usr.prioritaire',MQTYPE_BOOLEAN,'1'
call verifprop 'usr.amount',MQTYPE_FLOAT64,'123.45'
call verifprop 'usr.client',MQTYPE_STRING,'IBM'
/* ----- */
/* Summary */
/* ----- */
say ""
if rcode = 0 then do
    say "=====
    say "TEST PASSED: MQMHBUF AND MQBUFMH"
    say "THE MH1 PROPERTIES WERE RECONSTRUCTED"
    say "IN MH2 FROM THE MQRFH2 BUFFER"
    say "=====
end
else do
    say "=====
    say "TEST FAILED: MQMHBUF AND MQBUFMH"
    say "=====
end
signal cleanup
/* ----- */
/* Verify one property */
/* ----- */
verifprop:
    parse arg nomatt,typeatt,valatt
    pvalue.0 = 4096
    pvalue.1 = ''
    pnameout = ''
    ptype = ''
    rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
    say "Verifying" nomatt
    say " IMP      =" rcc
    if word(rcc,1) <> 0 then do
        say " FAILED: property is absent or inquiry failed"
        rcode = 8
        say "-----"
        return
    end
    say " Returned name      =" pnameout
    say " Returned type       =" ptype
    say " Expected type        =" typeatt

```



```

say " Returned value =" pvalue.1
say " Expected value =" valatt
if ptype <> typeatt then do
  say " FAILED: unexpected type"
  rcode = 8
end
else if pvalue.1 <> valatt then do
  say " FAILED: unexpected value"
  rcode = 8
end
else
  say " OK: property was reconstructed"
  say "-----"
return
/* ----- */
/* Single cleanup path */
/* ----- */
cleanup:
  if mh2cree = 1 then do
    dmho2.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh2','dmho2.')
    say "DMH mh2 =" rcc2
    mh2cree = 0
  end
  if mh1cree = 1 then do
    dmho1.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh1','dmho1.')
    say "DMH mh1 =" rcc2
    mh1cree = 0
  end
  if connecte = 1 then do
    rcc2 = RXMQDISC()
    say "DISC =" rcc2
    connecte = 0
  end
end
exit rcode

```

Discussion

1. This sample isolates the conversion APIs from queue put/get processing.
2. RXMQMBF returns the generated buffer, its actual DataLength, and the updated MQMD.
3. RXMQBMH consumes the same MQMD and buffer to populate mh2.
4. RXMQIMP with MQIMPO_CONVERT_VALUE verifies the reconstructed values in application encoding and CCSID.

Expected result

A nonempty MQRFH2 buffer is returned.

RXMQBMH completes successfully.

All five properties are reconstructed with the expected types and values.

The program reports TEST PASSED: MQMHBUFF AND MQBUFMH.

HANDLE9 - Demonstrate MQPD_USER_CONTEXT and context passing

Purpose

Separates message-context propagation from normal property copying. Four cases vary MQOO_SAVE_ALL_CONTEXT and the MQPMO context-passing options while MQCOPY_NONE prevents Action/CopyOptions from copying either test property.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQCLOS, and RXMQDISC.

Execution flow

Create usr.normal with MQPD_NO_CONTEXT and usr.contexte with MQPD_USER_CONTEXT
Run four GET/PUT combinations from no saved context through SAVE_ALL_CONTEXT + PASS_ALL_CONTEXT

Complete REXX sample

```
/* REXX */
/*
 * HANDLE9 - Testing MQPD_USER_CONTEXT
 *
 * Objective: clearly distinguish two mechanisms:
 *
 * 1. Message context:
 * MQOO_SAVE_ALL_CONTEXT
 * MQOO_PASS_ALL_CONTEXT
 * MQPMO_PASS_ALL_CONTEXT
 * MQPMO.Context = handle of the queue used for MQGET
 *
 * 2. Normal property copying:
 * OriginalMsgHandle
 * NewMsgHandle
 * MQPMO.Action
 * MQPD.CopyOptions
 *
 * This sample does NOT use OriginalMsgHandle, NewMsgHandle, or Action
 * during the second PUT. It therefore isolates only the mechanism
 * used to pass context.
 *
 * Two properties are placed in the initial message:
 *
 * usr.normal
 * Context = MQPD_NO_CONTEXT
 *
 * usr.contexte
 * Context = MQPD_USER_CONTEXT
 *
 * Both properties use MQCOPY_NONE so that no normal copy
 * through Action/CopyOptions can occur.
 *
 * Cases tested:
 *
 * 1 - GET without SAVE_ALL_CONTEXT
 * PUT without PASS_ALL_CONTEXT
 *
 * 2 - GET with SAVE_ALL_CONTEXT
 * PUT without PASS_ALL_CONTEXT
 *
 * 3 - GET with SAVE_ALL_CONTEXT
 * PUT with PASS_IDENTITY_CONTEXT
 *
 * 4 - GET with SAVE_ALL_CONTEXT
 * PUT with PASS_ALL_CONTEXT
 *
 * MA95 stems used:
 * MQPMO : .VER .OPT .TIME .CON .OMH .NMH .ACT .PL
 * MQGMO : .VER .OPT .WAIT .MOPT .MH
 * MQPD : .VER .OPT .SUPPORT .CONTEXT .COPYOPTIONS
 */
```

```

* Main expected result:
*
* usr.normal : absent from every copied message
* usr.contexte : present only in case 4
*/
TRACE 0
qmgr = 'QM1'
qname = 'DEV.QUEUE.1'
connecte = 0
rcode = 0
say "QMGR =" qmgr
say "QUEUE=" qname
say ""
/* ----- */
/* Initialization */
/* ----- */
rcc = rxmqinit()
if word(rcc,1) /= 0 then do
    say time('l') 'initialization problem rc ==> ' rcc
    interpret 'msgerr = rxmq.rcmap.'word(rcc,1)
    say "                                     ==> "msgerr
    exit 8
end
/* ----- */
/* Connection */
/* ----- */
rcc = RXMQCONN(qmgr)
say "CONN =" rcc
if word(rcc,1) <> 0 then exit 8
connecte = 1
/* ----- */
/* Run the four cases */
/* ----- */
call testcas 1,0,0,'NONE'
call testcas 2,1,0,'NONE'
call testcas 3,1,1,'IDENTITY'
call testcas 4,1,1,'ALL'
/* ----- */
/* Summary */
/* ----- */
say ""
if rcode = 0 then do
    say "=====
    say "TEST PASSED: MQPD_USER_CONTEXT"
    say "=====
end
else do
    say "=====
    say "TEST FAILED: MQPD_USER_CONTEXT"
    say "=====
end
signal cleanup
/* ----- */
/* Run one case */
/* ----- */
testcas:
    parse arg numcas,savecontext,passcontext,modepass
    h1ouvert = 0
    h2ouvert = 0
    mh1cree = 0
    mh2cree = 0
    mh3cree = 0
    casok = 1
    say ""
    say "=====
    say "CASE" numcas
    say "SAVE_ALL_CONTEXT =" savecontext
    say "PASS CONTEXT      =" modepass
    say "=====
    /* ----- */
    /* Open the input queue */
    /* ----- */
    iod1.OT = MQOT_Q
    iod1.ON = qname
    iod1.OM = ''
    oo1 = MQ00_INPUT_SHARED + MQ00_FAIL_IF QUIESCING
    if savecontext = 1 then
        oo1 = oo1 + MQ00_SAVE_ALL_CONTEXT
    rcc = RXMQOPEN('iod1.',oo1,'h1','ood1.')
    say "OPEN input =" rcc " h1=" h1

```

```

if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
h1ouvert = 1
/* ----- */
/* Open the output queue */
/* ----- */
iod2.OT = MQOT_Q
iod2.ON = qname
iod2.OM = ''
oo2 = MQOO_OUTPUT + MQOO_FAIL_IF QUIESCING
if passcontext = 1 then do
    if modepass = 'IDENTITY' then
        oo2 = oo2 + MQOO_PASS_IDENTITY_CONTEXT
    else if modepass = 'ALL' then
        oo2 = oo2 + MQOO_PASS_ALL_CONTEXT
    end
end
rcc = RXMQOPEN('iod2.',oo2,'h2','ood2.')
say "OPEN output =" rcc " h2=" h2
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
h2ouvert = 1
/* ----- */
/* Create MH1 for the initial message */
/* ----- */
cmho1.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh1','cmho1.')
say "MH1 =" rcc mh1
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
mh1cree = 1
/* ----- */
/* Normal property */
/* ----- */
drop smpo1. pdnormal.
smpo1.OPT = MQSMP0_NONE
pdnormal.VER = MQPD_VERSION_1
pdnormal.OPT = MQPD_NONE
pdnormal.SUPPORT = MQPD_SUPPORT_OPTIONAL
pdnormal.CONTEXT = MQPD_NO_CONTEXT
pdnormal.COPYOPTIONS = MQCOPY_NONE
rcc=RXMQSMP('mh1','smpo1.','usr.normal','pdnormal.',MQTYPE_STRING,'normale')
say "SMP usr.normal NO_CONTEXT =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
/* ----- */
/* Property belonging to user context */
/* ----- */
drop pdconte.
pdconte.VER = MQPD_VERSION_1
pdconte.OPT = MQPD_NONE
pdconte.SUPPORT = MQPD_SUPPORT_OPTIONAL
pdconte.CONTEXT = MQPD_USER_CONTEXT
pdconte.COPYOPTIONS = MQCOPY_NONE
rcc=RXMQSMP('mh1','smpo1.','usr.contexte','pdconte.',MQTYPE_STRING,'contexte')
say "SMP usr.contexte USER_CONTEXT =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
/* ----- */
/* First PUT: create the initial message */
/* ----- */
d.1 = 'Initial message for MQPD_USER_CONTEXT case 'numcas
d.0 = length(d.1)
imd2.VER = MQMD_VERSION_2
imd2.REP = MQRO_NONE
imd2.MSG = MQMT_DATAGRAM
imd2.EXP = MQEI_UNLIMITED
imd2.FBK = MQFB_NONE
imd2.ENC = MQENC_NATIVE
imd2.CCSI = MQCCSI_Q_MGR
imd2.FORM = MQFMT_STRING

```

```

imd2.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd2.PER = MQPER_PERSISTENCE_AS_Q_DEF
ipmo2.VER = MQPMO_VERSION_3
ipmo2.OPT = MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF QUIESCING +,
           MQPMO_DEFAULT_CONTEXT
ipmo2.TIME = MQWI_UNLIMITED
drop ipmo2.CON
ipmo2.OMH = 0
ipmo2.NMH = mh1
ipmo2.ACT = MQACTP_NEW
ipmo2.PL = 0
rcc = RXMQPUT(h2,'d.','imd2.','omd2.','ipmo2.','opmo2.')
say "PUT initial =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
dmho1.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh1','dmho1.')
say "DMH mh1 =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
mh1cree = 0
/* ----- */
/* GET the initial message */
/* ----- */
cmho2.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh2','cmho2.')
say "MH2 =" rcc mh2
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
mh2cree = 1
msg2.0 = 8192
igmd2.VER = MQMD_VERSION_2
igmd2.ENC = MQENC_NATIVE
igmd2.CCSI = MQCCSI_Q_MGR
igmd2.FORM = MQFMT_NONE
igmo2.VER = MQGMO_VERSION_4
igmo2.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT + MQGMO_FAIL_IF QUIESCING
igmo2.WAIT = 5000
igmo2.MOPT = MQMO_NONE
igmo2.MH = mh2
rcc = RXMQGET(h1,'msg2.','igmd2.','ogmd2.','igmo2.','ogmo2.')
say "GET initial =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
say "Initial body =" msg2.1
/* ----- */
/* Verify properties in MH2 */
/* ----- */
impo1.OPT = MQIMPO_NONE
impo1.RQENC = MQENC_NATIVE
impo1.RQCCSI = MQCCSI_APPL
handleinq = 'mh2'
say ""
say "Properties after the initial GET"
call verifpresent 'usr.normal',MQTYPE_STRING,'normale',MQPD_NO_CONTEXT
call verifpresent 'usr.contexte',MQTYPE_STRING,'contexte',MQPD_USER_CONTEXT
if casok = 0 then signal nettoiecas
/* ----- */
/* Second PUT: no message handle is supplied */
/* ----- */
msg3.1 = 'Message created from the context of case 'numcas
msg3.0 = length(msg3.1)
imd3.VER = MQMD_VERSION_2
imd3.REP = MQRO_NONE
imd3.MSG = MQMT_DATAGRAM
imd3.EXP = MQEI_UNLIMITED
imd3.FBK = MQFB_NONE
imd3.ENC = MQENC_NATIVE
imd3.CCSI = MQCCSI_Q_MGR
imd3.FORM = MQFMT_STRING
imd3.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd3.PER = MQPER_PERSISTENCE_AS_Q_DEF

```

```

ipmo3.VER      = MQPMO_VERSION_3
ipmo3.OPT=MQPMO_NO_SYNCPOINT + MQPMO_FAIL_IF QUIESCING +MQPMO_DEFAULT_CONTEXT
ipmo3.TIME     = MQWI_UNLIMITED
drop ipmo3.CON
ipmo3.OMH      = 0
ipmo3.NMH      = 0
ipmo3.ACT      = MQACTP_NEW
ipmo3.PL       = 0
if modepass = 'IDENTITY' then do
ipmo3.OPT=MQPMO_NO_SYNCPOINT+MQPMO_FAIL_IF QUIESCING+MQPMO_PASS_IDENTITY_CONTEXT
    ipmo3.CON = h1
end
else if modepass = 'ALL' then do
ipmo3.OPT =MQPMO_NO_SYNCPOINT+MQPMO_FAIL_IF QUIESCING+MQPMO_PASS_ALL_CONTEXT
    ipmo3.CON = h1
end
else do
ipmo3.OPT =MQPMO_NO_SYNCPOINT+MQPMO_FAIL_IF QUIESCING+MQPMO_DEFAULT_CONTEXT
end
say ""
say "Second PUT"
say " PMO.Options =" ipmo3.OPT
if symbol("ipmo3.CON") = "VAR" then,
    say " PMO.Context =" ipmo3.CON
else,
    say " PMO.Context = not set"
rcc = RXMQPUT(h2,'msg3.','imd3.','omd3.','ipmo3.','opmo3.')
say "Context PUT =" rcc
if word(rcc,1) <> 0 then do
    say "The PUT failed. Check context authorities."
    casok = 0
    signal nettoiecas
end
/* ----- */
/* The context saved in h1 is no longer required */
/* ----- */
dmho2.OPT = MQDMHO_NONE
rcc = RXMQDMH('mh2','dmho2.')
say "DMH mh2 =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
mh2cree = 0
/* ----- */
/* GET the message created by the second PUT */
/* ----- */
cmho3.OPT = MQCMHO_VALIDATE
rcc = RXMQMH('mh3','cmho3.')
say "MH3 =" rcc mh3
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
mh3cree = 1
msg4.0 = 8192
igmd4.VER = MQMD_VERSION_2
igmd4.ENC = MQENC_NATIVE
igmd4.CCSI = MQCCSI_Q_MGR
igmd4.FORM = MQFMT_NONE
igmo4.VER = MQGMO_VERSION_4
igmo4.OPT = MQGMO_NO_SYNCPOINT + MQGMO_WAIT + MQGMO_FAIL_IF QUIESCING
igmo4.WAIT = 5000
igmo4.MOPT = MQMO_NONE
igmo4.MH = mh3
rcc = RXMQGET(h1,'msg4.','igmd4.','ogmd4.','igmo4.','ogmo4.')
say "Context GET =" rcc
if word(rcc,1) <> 0 then do
    casok = 0
    signal nettoiecas
end
say "Context message body =" msg4.1
/* ----- */
/* Final verification */
/* ----- */
handleinq = 'mh3'
say ""
say "Final verification"
say "usr.normal must always be absent"
call verifabsente 'usr.normal'

```

```

    if modepass = 'ALL' then do
        say "usr.contexte must be present"
    call verifpresent 'usr.contexte',MQTYPE_STRING,'contexte',MQPD_USER_CONTEXT
    end
    else do
        say "usr.contexte must be absent"
        call verifabsente 'usr.contexte'
    end
    signal nettoiecas
/* ----- */
/* Verify a present property */
/* ----- */
verifpresent:
    parse arg nomatt,typeatt,valatt,contextatt
    pvalue.0 = 4096
    pvalue.1 = ''
    pnameout = ''
    ptype = ''
    rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
    say "IMP" nomatt "=" rcc
    if word(rcc,1) <> 0 then do
        say " FAILED: expected property is present"
        casok = 0
        say "-----"
        return
    end
    say " name      =" pnameout
    say " type      =" ptype "(expected" typeatt)"
    say " value      =" pvalue.1 "(expected" valatt)"
    say " PD.Context  =" pdout.CONTEXT "(expected" contextatt)"
    say " CopyOptions =" pdout.COPYOPTIONS
    if ptype <> typeatt then do
        say " FAILED: unexpected type"
        casok = 0
    end
    else if pvalue.1 <> valatt then do
        say " FAILED: unexpected value"
        casok = 0
    end
    else if pdout.CONTEXT <> contextatt then do
        say " FAILED: unexpected context"
        casok = 0
    end
    else
        say " OK"
        say "-----"
    return
/* ----- */
/* Verify an absent property */
/* ----- */
verifabsente:
    parse arg nomatt
    pvalue.0 = 4096
    pvalue.1 = ''
    pnameout = ''
    ptype = ''
    rcc=RXMQIMP(handleinq,'impo1.',nomatt,'pdout.','ptype','pvalue.','pnameout')
    say "IMP" nomatt "=" rcc
    if word(rcc,3) = MQRC_PROPERTY_NOT_AVAILABLE then do
        say " OK: property is absent"
    end
    else do
        if word(rcc,1) = 0 then do
            say " FAILED: property is present"
            say " type      =" ptype
            say " value      =" pvalue.1
            say " PD.Context  =" pdout.CONTEXT
            say " CopyOptions =" pdout.COPYOPTIONS
        end
        else do
            say " FAILED: unexpected error"
            say " return =" rcc
        end
        casok = 0
    end
    say "-----"
return
/* ----- */
/* Case cleanup */
/* ----- */

```

```

nettoiecas:
  if mh3cree = 1 then do
    dmho3.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh3','dmho3.')
    say "DMH mh3 =" rcc2
    mh3cree = 0
  end
  if mh2cree = 1 then do
    dmho2.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh2','dmho2.')
    say "DMH mh2 =" rcc2
    mh2cree = 0
  end
  if mh1cree = 1 then do
    dmho1.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mh1','dmho1.')
    say "DMH mh1 =" rcc2
    mh1cree = 0
  end
  if h2ouvert = 1 then do
    rcc2 = RXMQCLOS(h2,0)
    say "CLOSE h2 =" rcc2
    h2ouvert = 0
  end
  if h1ouvert = 1 then do
    rcc2 = RXMQCLOS(h1,0)
    say "CLOSE h1 =" rcc2
    h1ouvert = 0
  end
  if casok = 1 then
    say "CASE" numcas "OK"
  else do
    say "CASE" numcas "K0"
    rcode = 8
  end
return
/* ----- */
/* General cleanup */
/* ----- */
cleanup:
  if connecte = 1 then do
    rcc2 = RXMQDISC()
    say "DISC =" rcc2
    connecte = 0
  end
exit rcode

```

Discussion

1. No OriginalMsgHandle, NewMsgHandle, or MQPMO.Action value is used during the second put, so ordinary property copying cannot influence the result.
2. Both properties use MQCOPY_NONE.
3. usr.normal is not part of user context and is expected to be absent from every copied message.
4. usr.contexte is expected to appear only when the original GET saved all context and the second PUT passed all context.

Expected result

Cases 1, 2, and 3 contain neither test property after the second put/get cycle.

Case 4 contains usr.contexte with MQPD_USER_CONTEXT.

usr.normal is absent in every case.

The program reports TEST PASSED: MQPD_USER_CONTEXT.

HANDLE10 - Select messages with MQOD.SelectionString

Purpose

Creates five messages with identifying properties, opens queue handles with MQOD.SelectionString, retrieves two BLUE messages by property selection, then retrieves one exact RED message by combining Root.MQMD.MsgId with property predicates.

APIs used

RXMQINIT, RXMQCONN, RXMQOPEN, RXMQCLOS, RXMQMH, RXMQSMP, RXMQPUT, RXMQGET, RXMQIMP, RXMQDMH, RXMQDISC, and RXMQTERM.

Execution flow

Put five messages containing usr.testid, usr.color, usr.site, and usr.numero
Select two BLUE messages -> select one RED message by MsgId and properties -> delete remaining run-specific messages

Complete REXX sample

```
/* REXX */
/*
 * HANDLE10 - Selecting messages with MQOD.SelectionString
 *
 * This sample creates five messages on the same queue. Each message contains:
 *
 * usr.testid : unique identifier for this run
 * usr.color  : BLUE, RED, or GREEN
 * usr.site   : PARIS, LYON, or LILLE
 *
 * Two demonstrations are then pershaped:
 *
 * 1. Property-based selection:
 *   usr.testid = '<runid>' AND usr.color = 'BLUE'
 *   Two messages must be retrieved.
 *
 * 2. Combined MsgId and property selection:
 *   Root.MQMD.MsgId = "0x..."
 *   AND usr.testid = '<runid>'
 *   AND usr.color = 'RED'
 *   Exactly one specific message must be retrieved.
 *
 * The remaining messages from this run are then deleted with
 * a final selector usr.testid = '<runid>'.
 */
```

trace o

```
qmgr  = 'QM1'
qname = 'DEV.QUEUE.1'
runid = 'SEL' !! date('S') !! right(time('S'),5,'0')
```

```
connecte = 0
hputouvert = 0
hselouvert = 0
mhcrec = 0
rcode = 0
```

```
say '===== '
say 'HANDLE10 - MQ SELECTOR TEST'
say 'QMGR =' qmgr
say 'QUEUE =' qname
say 'RUNID =' runid
say '===== '
say ''
```

```
/* ----- */
/* Initialization and connection */
/* ----- */
rcc = RXMQINIT()
say 'INIT =' rcc
if word(rcc,1) <> 0 then exit 8
```

```

rcc = RXMQCONN(qmgr)
say 'CONN =' rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
connecte = 1

/* ----- */
/* Open the queue for output */
/* ----- */
drop iodput. oodput.
iodput.VER = MQOD_VERSION_4
iodput.OT = MQOT_Q
iodput.ON = qname

openput = MQOO_OUTPUT + MQOO_FAIL_IF QUIESCING
rcc = RXMQOPEN('iodput.',openput,'hput','oodput.')
say 'OPEN output =' rcc ' hput=' hput
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end
hputouvert = 1

/* ----- */
/* Create the five messages */
/* ----- */
say ''
say 'Create messages'
say '-----'

call putmessage 1, 'BLUE', 'PARIS', 'Message 1 : BLUE PARIS'
if rcode <> 0 then signal cleanup

call putmessage 2, 'RED', 'LYON', 'Message 2 : RED LYON'
if rcode <> 0 then signal cleanup

call putmessage 3, 'BLUE', 'LILLE', 'Message 3 : BLUE LILLE'
if rcode <> 0 then signal cleanup

call putmessage 4, 'GREEN', 'PARIS', 'Message 4 : GREEN PARIS'
if rcode <> 0 then signal cleanup

/* This fifth message is later targeted by its exact MsgId. */
call putmessage 5, 'RED', 'PARIS', 'Message 5: RED PARIS target MsgId'
if rcode <> 0 then signal cleanup

targetmsgid = msgid5
targetmsgidhex = c2x(targetmsgid)

say ''
say 'Target MsgId for message 5 = 0x' !! targetmsgidhex

/* The PUT handle is no longer required. */
rcc = RXMQCLOS(hput,MQCO_NONE)
say 'CLOSE output =' rcc
hputouvert = 0
if word(rcc,1) <> 0 then do
    rcode = 8
    signal cleanup
end

/* ===== */
/* TEST 1 - Property-based selection */
/* ===== */
say ''
say '===== '
say 'TEST 1 - PROPERTY-BASED SELECTION'
say '===== '

selector1 = "usr.testid = '" !! runid !!,
            '" AND usr.color = 'BLUE'"

say 'Selector = ' selector1

call openselector selector1
if rcode <> 0 then signal cleanup

count1 = 0

```

```

call createmh
if rcode <> 0 then signal cleanup

do forever
    call getselected 1000

    if getreason = MQRC_NO_MSG_AVAILABLE then leave
    if getcc <> MQCC_OK then do
        rcode = 8
        signal cleanup
    end

    count1 = count1 + 1
    call showproperties
end

call deletemh
call closeselector

say 'Number of BLUE messages retrieved =' count1
if count1 = 2 then
    say 'TEST 1 PASSED: exactly two messages selected'
else do
    say 'TEST 1 FAILED: expected 2, actual' count1
    rcode = 8
    signal cleanup
end

/* ===== */
/* TEST 2 - Combined MsgId and property selection */
/* ===== */
say ''
say '===== '
say 'TEST 2 - MsgId + PROPERTY SELECTION'
say '===== '

selector2 = 'Root.MQMD.MsgId = "0x' !! targetmsgidhex !!,
            '" AND usr.testid = ''' !! runid !!,
            ''' AND usr.color = 'RED''

say 'Selector = ' selector2

call openselector selector2
if rcode <> 0 then signal cleanup

call createmh
if rcode <> 0 then signal cleanup

call getselected 3000
if getcc <> MQCC_OK then do
    say 'TEST 2 FAILED: target message was not found'
    rcode = 8
    signal cleanup
end

say 'Target message retrieved'
say 'Received MsgId = 0x' !! c2x(ogmd.MSGID)
call showproperties

if c2x(ogmd.MSGID) <> targetmsgidhex then do
    say 'TEST 2 FAILED: MsgId differs from target MsgId'
    rcode = 8
    signal cleanup
end

/* A second GET must not find another message for this exact selector. */
call getselected 1000
if getreason = MQRC_NO_MSG_AVAILABLE then
    say 'No second message: OK'
else do
    say 'TEST 2 FAILED: more than one message matched the MsgId'
    rcode = 8
    signal cleanup
end

call deletemh
call closeselector
say 'TEST 2 PASSED: exactly one specific message selected'

/* ===== */

```

```

/* CLEANUP - delete the remaining messages from this run */
/* ===== */
say ''
say '===== '
say 'DELETE REMAINING MESSAGES'
say '===== '

selector3 = "usr.testid = '' !! runid !! ''"
say 'Selector = ' selector3

call openselector selector3
if rcode <> 0 then signal cleanup

call createmh
if rcode <> 0 then signal cleanup

reste = 0
do forever
    call getselected 500
    if getreason = MQRC_NO_MSG_AVAILABLE then leave
    if getcc <> MQCC_OK then do
        rcode = 8
        signal cleanup
    end
    reste = reste + 1
    say 'Remaining message deleted : ' msg.1
end

call deletemh
call closeselector
say 'Number of remaining messages deleted = ' reste

say ''
say '===== '
say 'TEST PASSED: MESSAGE SELECTION BY PROPERTIES AND MsgId'
say '===== '
signal cleanup

/* ===== */
/* Subroutine: create and PUT one message */
/* ===== */
putmessage:
    parse arg numero, color, site, corps

    drop cmho. smpo. pd. imd. omd. ipmo. opmo. data.

    cmho.OPT = MQCMHO_VALIDATE
    rcc = RXMQMH('mhput', 'cmho.')
    if word(rcc,1) <> 0 then do
        say 'MQCRTMH error = ' rcc
        rcode = 8
        return
    end

    smpo.OPT      = MQSMPO_NONE
    pd.VER        = MQPD_VERSION_1
    pd.OPT        = MQPD_NONE
    pd.SUPPORT    = MQPD_SUPPORT_OPTIONAL
    pd.CONTEXT    = MQPD_NO_CONTEXT
    pd.COPYOPTIONS = MQCOPY_NONE

    rcc = RXMQSMP('mhput', 'smpo.', 'usr.testid', 'pd.', MQTYPE_STRING, runid)
    if word(rcc,1) <> 0 then do
        say 'SET usr.testid error = ' rcc
        rcode = 8
        signal putmessage_cleanup
    end

    rcc = RXMQSMP('mhput', 'smpo.', 'usr.color', 'pd.', MQTYPE_STRING, color)
    if word(rcc,1) <> 0 then do
        say 'SET usr.color error = ' rcc
        rcode = 8
        signal putmessage_cleanup
    end

    rcc = RXMQSMP('mhput', 'smpo.', 'usr.site', 'pd.', MQTYPE_STRING, site)
    if word(rcc,1) <> 0 then do
        say 'SET usr.site error = ' rcc
        rcode = 8
        signal putmessage_cleanup
    end

```

```

end

rcc = RXMQSMP('mhput','smo.','usr.numero','pd.',MQTYPE_INT32,numero)
if word(rcc,1) <> 0 then do
    say 'SET usr.numero error =' rcc
    rcode = 8
    signal putmessage_cleanup
end

data.1 = corps
data.0 = length(data.1)

imd.VER = MQMD_VERSION_2
imd.REP = MQRO_NONE
imd.MSG = MQMT_DATAGRAM
imd.EXP = MQEI_UNLIMITED
imd.FBK = MQFB_NONE
imd.ENC = MQENC_NATIVE
imd.CCSI = MQCCSI_Q_MGR
imd.FORM = MQFMT_STRING
imd.PRI = MQPRI_PRIORITY_AS_Q_DEF
imd.PER = MQPER_PERSISTENCE_AS_Q_DEF
imd.MSGID = MQMI_NONE
imd.CID = MQCI_NONE

ipmo.VER = MQPMO_VERSION_3
ipmo.OPT = MQPMO_NO_SYNCPOINT +,
           MQPMO_FAIL_IF QUIESCING +,
           MQPMO_NEW_MSG_ID +,
           MQPMO_DEFAULT_CONTEXT
ipmo.OMH = MQHM_NONE
ipmo.NMH = mhput
ipmo.ACT = MQACTP_NEW

rcc = RXMQPUT(hput,'data.','imd.','omd.','ipmo.','opmo.')
say 'PUT' numero color site =' rcc
if word(rcc,1) <> 0 then do
    rcode = 8
    signal putmessage_cleanup
end

say ' Body = ' corps
say ' MsgId = 0x' !! c2x(omd.MSGID)

if numero = 5 then msgid5 = omd.MSGID

putmessage_cleanup:
    dmho.OPT = MQDMHO_NONE
    rcc2 = RXMQDMH('mhput','dmho.')
    if word(rcc2,1) <> 0 then do
        say 'MQDLTMH error =' rcc2
        rcode = 8
    end
    return

/* ===== */
/* Subroutine: open with MQOD.SelectionString */
/* ===== */
openselector:
    parse arg selecteur

    drop iodssel. oodssel.
    iodssel.VER = MQOD_VERSION_4
    iodssel.OT = MQOT_Q
    iodssel.ON = qname

    /* MQCHARV SelectionString representd by the SS sub-stem. */
    iodssel.SS.0 = length(selecteur)
    iodssel.SS.1 = selecteur
    iodssel.SS.CCSI = MQCCSI_APPL

    openin = MQOO_INPUT_SHARED + MQOO_FAIL_IF QUIESCING
    rcc = RXMQOPEN('iodssel.',openin,'hssel','oodssel.')
    say 'OPEN selector =' rcc ' hssel=' hssel
    if word(rcc,1) <> 0 then do
        rcode = 8
        return
    end
    hselouvert = 1
    return

```

```

/* ===== */
/* Subroutine: close the selector handle */
/* ===== */
closeselector:
    if hselouvert = 1 then do
        rcc = RXMQCLOS(hsel,MQCO_NONE)
        say 'CLOSE selector =' rcc
        if word(rcc,1) <> 0 then rcode = 8
        hselouvert = 0
    end
    return

/* ===== */
/* Subroutine: create the GET message handle */
/* ===== */
createmh:
    drop cmhoget.
    cmhoget.OPT = MQCMHO_VALIDATE
    rcc = RXMQMH('mhget','cmhoget.')
    say 'MH GET =' rcc ' mhget=' mhget
    if word(rcc,1) <> 0 then do
        rcode = 8
        return
    end
    mhcreate = 1
    return

/* ===== */
/* Subroutine: delete the GET message handle */
/* ===== */
deletemh:
    if mhcreate = 1 then do
        dmho.OPT = MQDMHO_NONE
        rcc = RXMQDMH('mhget','dmho.')
        say 'DMH GET =' rcc
        if word(rcc,1) <> 0 then rcode = 8
        mhcreate = 0
    end
    return

/* ===== */
/* Subroutine: GET a message matching the selector */
/* ===== */
getselected:
    parse arg attente

    drop msg. igmd. ogmd. igmo. ogmo.
    msg.0 = 8192

    igmd.VER = MQMD_VERSION_2
    igmd.ENC = MQENC_NATIVE
    igmd.CCSI = MQCCSI_Q_MGR
    igmd.FORM = MQFMT_NONE
    igmd.MSGID = MQMI_NONE
    igmd.CID = MQCI_NONE

    igmo.VER = MQGMO_VERSION_4
    igmo.OPT = MQGMO_NO_SYNCPOINT +,
              MQGMO_WAIT +,
              MQGMO_FAIL_IF QUIESCING +,
              MQGMO_PROPERTIES_IN_HANDLE
    igmo.WAIT = attente
    igmo.MOPT = MQMO_NONE
    igmo.MH = mhget

    rcc = RXMQGET(hsel,'msg.','igmd.','ogmd.','igmo.','ogmo.')
    getcc = RXMQ.LASTCC
    getreason = RXMQ.LASTAC

    if getcc = MQCC_OK then do
        say 'GET =' rcc
        say ' Body =' msg.1
    end
    else if getreason <> MQRC_NO_MSG_AVAILABLE then
        say 'GET error =' rcc

    return

/* ===== */

```

```

/* Subroutine: display properties of the selected message */
/* ===== */
showproperties:
  call inqstring 'usr.testid', 'testid'
  call inqstring 'usr.color', 'color'
  call inqstring 'usr.site', 'site'
  call inqstring 'usr.numero', 'numero'

  say ' MsgId      = 0x' !! c2x(ogmd.MSGID)
  say '  usr.testid =' testid
  say '  usr.color  =' color
  say '  usr.site   =' site
  say '  usr.numero =' numero
  say '-----'
  return

/* ===== */
/* Subroutine: inquire one property in the message handle */
/* ===== */
inqstring:
  parse arg propname, resultname

  drop impo. pdout. pvalue.
  impo.OPT  = MQIMPO_INQ_FIRST
  impo.RQENC = MQENC_NATIVE
  impo.RQCCSI = MQCCSI_APPL
  pvalue.0  = 4096
  pnameout  = ''
  ptype     = 0

  rcc = RXMQIMP('mhget', 'impo.', propname, 'pdout.',,
               'ptype', 'pvalue.', 'pnameout')
  if word(rcc,1) = 0 then
    interpret resultname '= pvalue.1'
  else do
    interpret resultname "= '<absent>'"
    say 'INQ' propname 'error =' rcc
    rcode = 8
  end
  return

/* ===== */
/* General cleanup */
/* ===== */
cleanup:
  if mhcreate = 1 then call deletemh
  if hselouvert = 1 then call closeselector

  if hputouvert = 1 then do
    rcc = RXMQCLOS(hput, MQCO_NONE)
    say 'CLOSE output cleanup =' rcc
    hputouvert = 0
  end

  if connecte = 1 then do
    rcc = RXMQDISC()
    say 'DISC =' rcc
    connecte = 0
  end

  rcc = RXMQTERM()
  say 'TERM =' rcc

  exit rcode

```

Discussion

1. A run-specific `usr.testid` value prevents messages from another execution from matching the selectors.
2. The first selector uses property predicates only and is expected to retrieve exactly two messages.
3. The second selector combines a hexadecimal `Root.MQMD.MsgId` predicate with `usr.testid` and `usr.color`.
4. A final selector removes any messages belonging to the current run so the test leaves the queue clean.

Expected result

The first selector retrieves exactly two BLUE messages.

The combined selector retrieves exactly one target RED message and no second message.

All remaining messages from the current run are deleted.

The program reports TEST PASSED: MESSAGE SELECTION BY PROPERTIES AND MsgId.